PD-LMI package
User Manual

Version v1.0.0

July 19, 2026

Abstract

The PD-LMI package is a MATLAB/YALMIP package for modelling and certifying parameter-dependent linear matrix inequalities (PD-LMIs). It represents coefficient-backed known matrices and decision matrices as cell-local tensor-product Bernstein polynomials on a physical parameter grid, with continuous shared-face storage for decision functions. This representation provides a finite, inspectable bridge from a continuous parameter domain to semidefinite constraints.

The manual develops that bridge through a reproducible differentiable PD-LMI workflow: define the parameter grid and known data, construct continuous decision matrices, differentiate them cell by cell when scheduling-rate terms are present, form a residual inequality, choose a direct or certificate-based positivity condition, export the resulting constraints to YALMIP, and solve them with an available SDP solver. It also makes explicit the distinction between shared-face continuity of the represented function and the generally discontinuous cellwise derivative data.

Version 1.0 presents the implemented and tested public surface as one supported workflow: installation and verification; known-data, decision-variable, and finite-certificate objects; MATLAB-style matrix operations and indexing; helper utilities; and direct, Pólya, Putinar, and full-box certificate assembly. The appendices provide the Bernstein-polynomial and positivity-certificate background needed to interpret those backends and their documented boundaries.

Contents

Release History

The entries below retain the current release and the latest release from each completed minor line.

v1.0.0 — July 19, 2026
Marks the first stable public release of the documented package workflow. It consolidates installation and test-suite verification, the `pdbase/pdmat/pdvar/pdlmi` object model, continuous arbitrary-degree decision storage and rate-vertex differentiation, MATLAB-style matrix operations and indexing, and direct, Pólya, Putinar, and full-box finite certificate assembly. The 1.0 documentation also makes the affine-expression, tensor-box, certificate-sufficiency, and solver-ownership boundaries explicit; this milestone does not add new runtime behavior beyond the implementation covered by the current code and tests.

v0.4.7 — July 18, 2026
Completed the v0.4 manual and Web explorer refresh around the LPV-to-cellwise-Bernstein-to-solver workflow, including installer guidance, the public API reference, mathematical cross-references, and storage-transformation examples. Released by commit `c6a2e25`.

v0.3.6 — July 16, 2026
Completed the public rename to `pdbase/pdmat/pdvar/pdlmi`, corrected forward Bernstein coordinates, added Full Box and Putinar certificates with independent SOS validation, hardened API boundaries, and defined entry-wise inequality behavior. Released by commit `fe74a65`.

v0.2.12 — July 13, 2026
Completed the `pdmat/pdvar/pdlmi` layers, rate-aware differentiation, display, plotting, structural algebra, direct finite assembly, Pólya selection, and deterministic reference examples and tests. Released by commit `98b63e5`.

v0.1.0 — July 3, 2026
Introduced the original grid-and-storage foundation and the first package manual, including Bernstein traversal, validation, and helper contracts. Released by commit `63c06ee`.

Chapter 1

From LPV Models to Finite Solver Constraints

This chapter introduces the purpose, object model, and complete solution path of the package. It begins with a continuous LPV control inequality, separates the physical grid from local polynomial labels, and ends with finite YALMIP constraints and recovery of the solved parameter-dependent matrix.

1.1 Motivation: LPV Systems and Parameter-Dependent LMIs

Many systems are usefully linear only after their operating condition is made explicit. An aircraft changes with altitude and speed, a chemical process changes with temperature and concentration, and an autonomous vehicle changes lane with forward speed and tire cornering conditions. Treating those conditions as unrelated LTI models discards their shared structure. Requiring one constant Lyapunov matrix or one constant stability criterion for all cases can be equally unsatisfactory, as the resulting test is often unnecessarily conservative.

A *linear parameter-varying (LPV) system* keeps the linear state-space structure while allowing its known matrices to depend on a measurable scheduling vector $\boldsymbol{\rho}(t) = (\rho_1(t), \ldots, \rho_\ell(t)) \in \mathcal{P} \subset \mathbb{R}^\ell$. For example, an autonomous model is written as

$$\dot{x}(t) = A\big(\boldsymbol{\rho}(t)\big)x(t),$$

where x is the state vector and $\boldsymbol{\rho}$ selects the current parameter of the known matrix family $A(\boldsymbol{\rho})$. The central question is therefore not whether each frozen model works in isolation, but whether one structured stability condition proves the required property throughout the full parameter set $\mathcal{P}$.

A *differentiable parameter-dependent linear matrix inequality* (DPD-LMI) supplies that condition by allowing the decision matrix itself to vary with $\boldsymbol{\rho}$. A standard LPV Lyapunov construction seeks a symmetric $P(\boldsymbol{\rho})$ such that the following inequalities hold throughout $(\rho, \dot{\rho}) \in \mathcal{P} \times \mathcal{V}$:

$$P(\boldsymbol{\rho}) \succeq I, \tag{1.1}$$

$$\dot{P}(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) + P(\boldsymbol{\rho})A(\boldsymbol{\rho}) + A(\boldsymbol{\rho})^{\mathsf{T}}P(\boldsymbol{\rho}) \preceq -\varepsilon I. \tag{1.2}$$

The first inequality keeps the Lyapunov function $V(t) = x^\top(t)P(\boldsymbol{\rho}(t))x(t)$ positive definite, while the second requires $\dot{V}$ to decrease along every admissible scheduling trajectory. When $\boldsymbol{\rho}$ changes with time, the time derivative term $\dot{P}$ matters. On the other hand, if $\boldsymbol{\rho}$ represents uncertainty of the system, which does not vary with respect to the time, removing $\dot{P}$, i.e., fixing $\dot{\boldsymbol{\rho}} = 0$, recovers an ordinary *parameter-dependent linear matrix inequality* (PD-LMI). After known data are fixed, every term must remain affine in the decision coefficients, and the displayed εI remains an explicit user choice rather than a hidden package margin.

1.2 Package roles and the end-to-end path

The package starts after the user has chosen an LPV model, a parameter box, any scheduling-rate bounds, and the matrix inequality to certify. Its role is to represent the known and decision matrices, convert the parameter continuum into finite certificate constraints, and hand those constraints to YALMIP. It does not generate the plant or controller structure, choose an objective or SDP solver, or interpret solver diagnostics on the user's behalf.

The main objects follow the same four-stage path used in the rest of the manual:

1. `pdmat` stores known parameter-dependent matrices, while `pdvar` creates continuous parameter-dependent decision matrices on a physical grid. Both inherit grid, degree, label, and cell-local storage mechanics from the backend class `pdbase`; ordinary users normally begin with `pdmat` and `pdvar`, not `pdbase`.

2. Matrix operations form an affine cell-local residual. When scheduling-rate terms are present, `rhodiff` differentiates a `pdvar` on each cell and stores one derivative row for every rate-box vertex.

3. A comparison such as `residual <= 0` creates a `pdlmi`. Direct Bernstein-coefficient constraints are the default; `applyPolya`, `applyPutinar`, and `applyFullBoxPreorder` replace that selection with an implemented opt-in certificate.

4. `toYalmip` exports the stored finite constraints. The user calls YALMIP `optimize` with an explicit objective and solver configuration, checks the returned diagnostics, and calls `value` to recover solved coefficient data.

Two finite reductions in this path have different logical status. Traversing all rate-box vertices is exact when the residual is affine in $\dot{\boldsymbol{\rho}}$. By contrast, the selected Bernstein or Bernstein–Gram certificate supplies a finite sufficient condition over the remaining $\boldsymbol{\rho}$-continuum; failure of that certificate does not prove that the original continuous inequality is infeasible. The mathematical comparison in sections B.9 and B.10 separates the implemented certificate choices from changes to the model itself. The next four sections explain the representation behind these reductions, and section 1.8 executes the complete path in MATLAB.

1.3 Version 1.0 Supported Scope

Version 1.0 is the package's public-release milestone for the workflow documented in this manual. The supported surface comprises the `pdbase` storage backend; `pdmat` known data; continuous arbitrary-degree `pdvar` decisions and their cellwise `rhodiff` derivatives; `pdlmi` direct, Pólya, Putinar, and full-box assembly; the documented MATLAB matrix-operation and indexing overloads; and the public `helper` utilities. The source-indexed reference chapters state the accepted forms, returned shapes, validation rules, and error boundaries for that surface.

This milestone stabilizes and documents the existing implementation; it does not introduce an additional certificate family or solver layer. The package remains restricted to tensor-product box grids, affine decision expressions, and the certificate constructions described here. Products that are nonlinear in decision or rate-dependent data are rejected, certificate failure remains inconclusive for the original continuum inequality, and YALMIP retains ownership of optimization objectives, solver selection, and diagnostics.

For release verification, run the root installer and `tests.run_all()` as described in Chapter 2. Examples involving `pdvar`, `rhodiff`, `pdlmi`, or solver state begin from a cleared YALMIP model so that their decision variables and constraints are reproducible in a fresh MATLAB session.

1.4 The Chain Rule in a DPD-LMI

The dot on P is a time derivative along the scheduling trajectory, not a new independent matrix. Applying the multivariable chain rule gives

$$\dot{P}(\boldsymbol{\rho}(t)) = \frac{dP}{dt} = \sum_{s=1}^{\ell} \frac{\partial P}{\partial \rho_s}(\boldsymbol{\rho}(t))\dot{\rho}_s(t).$$

Write $\boldsymbol{\nu} = \dot{\boldsymbol{\rho}}$ and give every scheduling direction independent finite rate bounds,

$$\mathcal{V} = \prod_{s=1}^{\ell} [\underline{\nu}_s, \overline{\nu}_s].$$

For a residual that is affine in $\boldsymbol{\nu}$, the rate box is handled exactly by traversing its 2^ℓ lower/upper vertices $\boldsymbol{v}^{(r)}$. The coefficient-level differentiation and rate-row construction are derived in section A.9. Thus, rather than retaining a rate continuum, the rate-dependent inequality can be written in the block-diagonal form

$$\operatorname{diag}_{r=1}^{2^\ell} \mathcal{F}(\boldsymbol{\rho}, \boldsymbol{v}^{(r)}) \prec 0, \qquad \boldsymbol{\rho} \in \mathcal{P}. \tag{1.3}$$

Each diagonal block is the residual at one rate vertex. The remaining continuum is only the scheduling parameter $\boldsymbol{\rho}$, which the package treats cellwise with its Bernstein certificates.

We therefore focus on the following general DPD-LMI.

General form of the differentiable parameter-dependent LMI

$$\mathcal{F}(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) = \sum_{i=1}^{N} R_i(\boldsymbol{\rho}) \sum_{j=1}^{\ell} \frac{\partial x_i}{\partial \rho_j}(\boldsymbol{\rho})\dot{\rho}_j + F_0(\boldsymbol{\rho}) + \sum_{i=1}^{N} F_i(\boldsymbol{\rho}) x_i(\boldsymbol{\rho}) \prec 0, \qquad \forall (\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) \in \mathcal{P} \times \mathcal{V}. \tag{1.4}$$

Here R_i, F_0, and F_i are known matrix functions, while the x_i are parameter-dependent decision functions. The affine dependence on $\dot{\boldsymbol{\rho}}$ reduces exactly to the finite rate vertices in (1.3), but $\boldsymbol{\rho} \in \mathcal{P}$ remains a continuum. The construction is dimension-generic in ℓ, subject to the cost of the resulting tensor data. The package handles the remaining continuum by representing each x_i as a continuous piecewise-polynomial function on a physical grid, with cell-local tensor-product Bernstein coefficients.

Setting $R_i \equiv 0$ for every i (collectively, $R = 0$) removes the derivative term and recovers an ordinary PD-LMI.

1.5 Grid the Scheduling-Parameter Space

The package currently considers only an axis-aligned box parameter domain,

$$\mathcal{P} = \prod_{s=1}^{\ell} [\underline{\rho}_s, \overline{\rho}_s].$$

This restriction is deliberate, as a compact box is easy to partition into hypercubes for cellwise modelling. General polytopic or curved parameter domains are outside the implemented model. Gridding starts independently in each parameter direction. For each direction, choose the following physical grid

$$\mathcal{G}_s = \{\underline{\rho}_s = \rho_s^{(1)}, \ldots, \rho_s^{(k_s)} = \overline{\rho}_s\}, \qquad s = 1, \ldots, \ell.$$

The full tensor grid is $\mathcal{G} = \mathcal{G}_1 \times \cdots \times \mathcal{G}_\ell$. It contains $\prod_{s=1}^{\ell} k_s$ physical nodes and partitions $\mathcal{P}$ into $\prod_{s=1}^{\ell}(k_s - 1)$ physical hypercubes. A hypercube is identified by the ℓ-dimensional multi-index $\mathbf{c} = (c_1, \ldots, c_\ell)$; section A.7 gives the exact physical-grid and local-label conventions:

$$\mathcal{H}_{\mathbf{c}} = \prod_{s=1}^{\ell} [\rho_s^{(c_s)}, \rho_s^{(c_s+1)}]. \tag{1.5}$$

1.6 Desired Behavior Across Cell Boundaries

The package represents coefficient-backed known data and decision matrices by local polynomials, so products, degree elevation, and differentiation can be performed one physical cell at a time. A Lyapunov matrix or comparable decision function must not jump when the scheduling trajectory crosses a shared face, so `pdvar` guarantees global C^0 continuity. It does not require C^1 continuity, that is, one-sided derivatives may differ, and `rhodiff` retains the corresponding cell-local derivative data. Known `pdmat` data are different: the object records whether adjacent numeric faces match, but it preserves explicitly supplied discontinuous local data rather than adding decision constraints to repair them.

A monomial parameterization makes the additional continuity condition explicit. For two adjacent one-dimensional cells, write

$$A^{(c)}(\alpha_c) = \sum_{i=0}^{m} \alpha_c^i A_i^{(c)}, \qquad \alpha_c \in [0, 1].$$

At their shared physical node, $\alpha_1 = 1$ in the left cell and $\alpha_2 = 0$ in the right cell. Enforcing C^0 continuity therefore requires the extra matrix equality

$$\underbrace{\sum_{i=0}^{m} A_i^{(1)}}_{A^{(1)}(1)} = \underbrace{A_0^{(2)}}_{A^{(2)}(0)}.$$

When $\ell > 1$, equality along a shared face is a polynomial identity in the tangential coordinates and produces a corresponding family of linear coefficient equalities. The number of such conditions grows with the polynomial degree and the number of shared faces.

The right side of Fig 1.1 shows the intended behaviour: the two polynomial pieces share their boundary value, while their one-sided slopes remain different. A globally C^0, piecewise-polynomial

matrix function on the compact parameter box is Lipschitz. At a shared face, its Clarke generalized derivative is the convex hull of the limiting derivatives from the adjacent cells. The package consequently imposes (1.4) using the relevant cellwise derivative and rate-vertex data without claiming that the derivative itself is globally continuous.

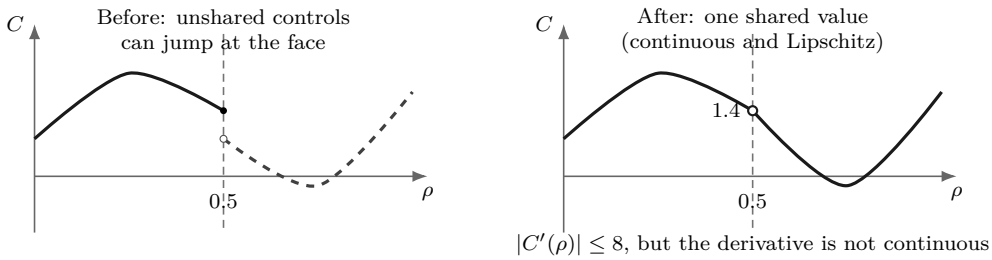


Figure 1.1: The required cell-interface behaviour. Enforcing one shared boundary value removes the jump while allowing unequal one-sided derivatives.

1.7 Bernstein Basis and Cell-Local Storage

For its cell-local polynomial representation, the package uses the Bernstein basis rather than the monomial basis in the preceding comparison. Bernstein endpoint identities expose each face polynomial directly through its face coefficients; shared decision storage can therefore impose C^0 continuity without adding separate cross-cell equalities. Inside a physical hypercube (1.5), the forward local coordinate in direction s is

$$\alpha_s = \frac{\rho_s - \rho_s^{(c_s)}}{\rho_s^{(c_s+1)} - \rho_s^{(c_s)}} \in [0,1].$$

On one local coordinate $\alpha \in [0,1]$, the normalized degree-m Bernstein basis is

$$B_i^m(\alpha) = \binom{m}{i}(1-\alpha)^{m-i}\alpha^i, \qquad i = 0, \dots, m.$$

The basis functions are nonnegative and sum to one. The first four nonnegative degrees are

$$\begin{aligned}
m = 0: &\quad B_0^0 = 1 \\
m = 1: &\quad B_0^1 = 1-\alpha, \qquad B_1^1 = \alpha, \\
m = 2: &\quad B_0^2 = (1-\alpha)^2, \quad B_1^2 = 2(1-\alpha)\alpha, \quad B_2^2 = \alpha^2, \\
m = 3: &\quad B_0^3 = (1-\alpha)^3, \quad B_1^3 = 3(1-\alpha)^2\alpha, \quad B_2^3 = 3(1-\alpha)\alpha^2, \quad B_3^3 = \alpha^3.
\end{aligned}$$

In ℓ parameter directions, the package uses the tensor-product expansion

$$C^{(\mathbf{c})}(\boldsymbol{\rho}) = \sum_{\mathbf{i} \in \{0,\dots,m\}^\ell} \left(\prod_{s=1}^{\ell} B_{i_s}^m(\alpha_s) \right) C^{(\mathbf{c})}[\mathbf{i}].$$

The matrix value is a convex combination of the local coefficient matrices. Exact degree elevation, multiplication, and differentiation operate on those coefficients without changing the identity of the physical cell. The detailed formulas appear in sections A.3, A.6, A.8 and A.9.

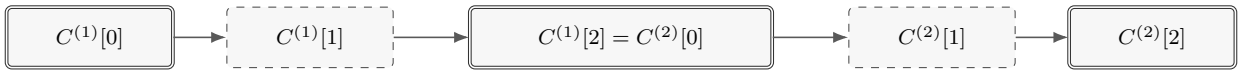
1.7.1 Physical nodes, local labels, and continuity by storage

The two index systems have different meanings:

- the physical node $\rho_s^{(j)}$ and the cell component c_s are one-based and identify geometry;
- the local Bernstein label $i_s \in \{0, \ldots, m\}$ is zero-based and selects one coefficient inside an already chosen cell.

A degree-m tensor polynomial stores $(m+1)^\ell$ local coefficient positions per physical cell. These positions are not another grid of point samples. The traversal order used by `cells`, `lbls`, `coeffs`, and rate-vertex rows varies later dimensions fastest; in two dimensions, `lbls()` lists $[0,0], [0,1], \ldots, [0,m], [1,0], \ldots, [m,m]$. The general tensor ordering is defined in section A.7.

The endpoint identities $B_0^m(0) = 1$ and $B_m^m(1) = 1$ make continuity easy to encode in coefficient data. For a `pdvar`, adjacent cells reuse the same YALMIP decision handles on their common face; in one dimension, $C^{(1)}[m]$ and $C^{(2)}[0]$ are the same decision matrix. Tensor cells similarly share every handle on a common face, so continuity requires no later equality constraint. For a coefficient-backed `pdmat`, `IsContinuous` instead reports whether the two numeric face-coefficient sets agree; explicitly mismatched local data remain cell-local and produce a warning.



shared value, independent interiors

1.7.2 A two-dimensional storage example

The first two-dimensional example uses $\mathcal{G}_1 = \{0, 0.5, 1\}$, $\mathcal{G}_2 = \{0, 1\}$, and degree two:

```
grid2 = {[0 .5 1], [0 1]};
A2 = pdmat(grid2, @(rho1,rho2) ...
    [1+rho1+rho2, rho1*rho2; ...
        rho1*rho2,   2+rho1^2], Degree=2);
physicalCells = A2.cells()
localLabels = A2.lbls()
secondCellCoefficients = A2.coeffs([2 1])
sameLeaf = A2.LocalValues{2}{1}
```

The grid has two physical hypercubes. The highlighted one below is $\mathbf{c} = (2, 1)$, whose domain is $[0.5, 1] \times [0, 1]$. Selecting it through `A2.coeffs([2 1])` reaches the same nested storage leaf as `A2.LocalValues{2}{1}`. The figure lists the nine matrices in `A2.lbls()` order, with i_1 varying more slowly than i_2.

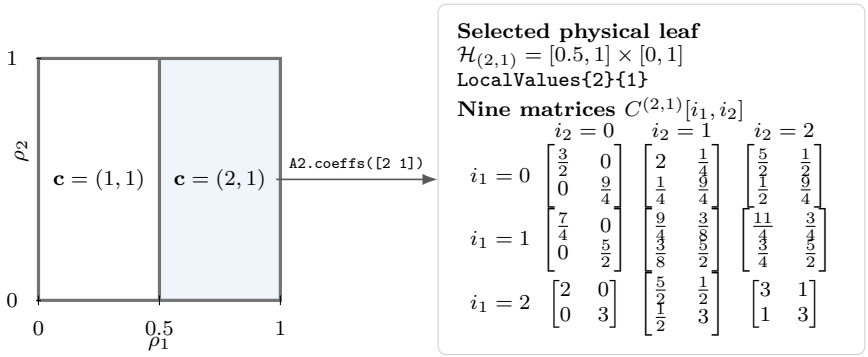


Figure 1.2: A physical hypercube selects one storage leaf. Within that leaf, `A2.lbls()` supplies the nine zero-based local labels in the displayed order. The matrices are Bernstein coefficients, not a 3-by-3 grid of point samples.

The selected leaf is a flat nine-entry coefficient cell: row q of `A2.lbls()` labels entry q. Sections A.4 and A.7 derive the ordering and conversion rules.

1.8 From Cellwise Algebra to Solver Constraints

This section executes a two-cell instance of the Lyapunov model in (1.1) and (1.2). It follows the object roles established above: known matrices are stored as `pdmat`, decision matrices as `pdvar`, cellwise rate derivatives are produced by `rhodiff`, and finite certificate selections are stored as `pdlmi`.

1.8.1 Store Known Matrices and Decision Matrices

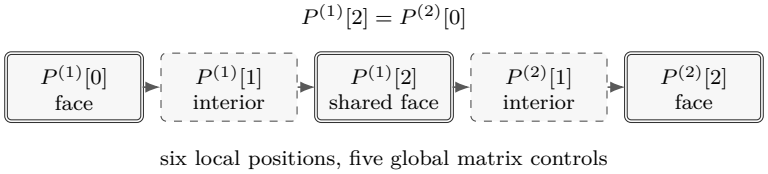
The known LPV matrix is affine in ρ on the two-cell grid $[0, 0.5] \cup [0.5, 1]$. Supplying `Degree=1` retains exact function evaluation together with cell-local Bernstein coefficient evidence.

```
yalmip('clear')
```

```
grid = {[0 0.5 1]};
A = pdmat(grid, @(rho) [−1, 0.5; −1, −2] + ...
    rho ∗ [−1.3, −20; 2, −10], Degree=1);
P = pdvar(2, grid, "symmetric", Degree=2);
matrixSize = size(P)
physicalCellCount = A.ncell()
knownDegree = A.Degree
decisionDegree = P.Degree
isContinuous = P.IsContinuous
```

```
matrixSize =
     2     2
physicalCellCount =
     2
knownDegree =
     1
decisionDegree =
     2
isContinuous =
  logical
   1
```

Each cell stores three local symmetric decision matrices. The upper face of cell 1 and lower face of cell 2 share one YALMIP decision handle, so six local positions contain five global controls. The interior controls remain independent.



six local positions, five global matrix controls

1.8.2 Multiply in Bernstein Coefficient Space

On one physical cell, write $P = \sum_i B_i^m P^{(c)}[i]$ and $A = \sum_j B_j^n A^{(c)}[j]$. The ordered product is

$$PA = \sum_{k=0}^{m+n} B_k^{m+n} R^{(c)}[k], \qquad R^{(c)}[k] = \sum_{i+j=k} \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}} P^{(c)}[i] A^{(c)}[j].$$

For quadratic P and affine A, the four result coefficients are

$$R[0] = P_0 A_0, \quad R[1] = (P_0 A_1 + 2P_1 A_0)/3,$$

$$R[2] = (2P_1 A_1 + P_2 A_0)/3, \quad R[3] = P_2 A_1.$$

The anti-diagonal grouping $k = i + j$ and binomial normalization are exact; this is not a pointwise sample product, and matrix order is preserved. Section A.8 derives the general tensor-product rule.

```
PA = P ∗ A;
ATP = A' ∗ P;
degreePA = PA.Degree
degreeATP = ATP.Degree
coefficientCountPA = numel(PA.coeffs(1))
```

```
degreePA =
     3
degreeATP =
     3
coefficientCountPA =
     4
```

1.8.3 Differentiate and Enumerate Rate Vertices

For a degree-m cell of width h_c, differentiation is the forward difference

$$\frac{\partial P}{\partial \rho} = \frac{m}{h_c} \sum_{i=0}^{m-1} (P^{(c)}[i+1] - P^{(c)}[i]) B_i^{m-1}(\alpha).$$

Because the chain-rule term is affine in the bounded rate, checking the two vertices -1 and $+1$ covers $\dot{\rho} \in [-1, 1]$. Section A.9 gives the coefficient derivation and its physical-cell scaling.

For ℓ parameters, let $h_{\mathbf{c},s} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)}$ be the width of physical hypercube $\mathbf{c}$ in direction s, and let $\mathbf{e}_s$ be the corresponding unit multi-index. Differentiating the tensor-product representation in section 1.7 gives

$$\frac{\partial P^{(\mathbf{c})}}{\partial \rho_s} = \frac{m}{h_{\mathbf{c},s}} \sum_{\substack{\mathbf{i} \in \{0,\dots,m\}^\ell \\ i_s < m}} (P^{(\mathbf{c})}[\mathbf{i} + \mathbf{e}_s] - P^{(\mathbf{c})}[\mathbf{i}]) B_{i_s}^{m-1}(\alpha_s) \prod_{r \neq s} B_{i_r}^m(\alpha_r), \qquad \dot{P}^{(\mathbf{c})} = \sum_{s=1}^{\ell} \dot{\rho}_s \frac{\partial P^{(\mathbf{c})}}{\partial \rho_s}.$$

Each partial derivative has degree $m-1$ in its differentiated direction and degree m in every other direction. For $\ell > 1$, `rhodiff` therefore elevates the shortened direction back to degree m before summing the rate-weighted partials. If $\dot{\rho}_s \in [\underline{v}_s, \overline{v}_s]$, the rate box has 2^ℓ Cartesian vertices $\mathbf{v}$; because $\dot{P}$ is affine in $\dot{\boldsymbol{\rho}}$, traversing those vertices is exact. For $m > 0$, each multidimensional cell consequently stores 2^ℓ rate rows with $(m+1)^\ell$ tensor-Bernstein coefficients per row.

```
D = rhodiff(P, [−1 1]);
derivativeDegree = D.Degree
isContinuous = D.IsContinuous
rateRowCount = size(D.coeffs(1),1)
coefficientsPerRow = size(D.coeffs(1),2)
```

```
derivativeDegree =
    1
isContinuous =
  logical
   0
rateRowCount =
    2
coefficientsPerRow =
    2
```

For example, a degree-one decision on one two-parameter cell has four rate vertices and four tensor-Bernstein labels:

```
rb2 = [−1 2; −3 5];
P2 = pdvar(1, {[0 2], [10 20]}, Degree=1, RateBounds=rb2);
D2 = rhodiff(P2);
tensorDerivativeDegree = D2.Degree
rateRowCount2D = size(D2.coeffs([1 1]), 1)
```

```
coefficientsPerRow2D = size(D2.coeffs([1 1]), 2)
```

```
tensorDerivativeDegree =
    1
rateRowCount2D =
    4
coefficientsPerRow2D =
    4
```

Thus a coefficient table is indexed independently by physical hypercube, rate-vertex row, and tensor-Bernstein label. Rate rows are neither physical cells nor additional Bernstein labels; a later comparison constrains every combination of these three indices.

1.8.4 Form a Residual on Every Hypercube

The MATLAB residual is a direct transcription of (1.1) and (1.2). Addition performs exact degree elevation before the cell-local terms are combined.

```
residual = D + P * A + A' * P;
Cdecay = residual <= -1e-6 * eye(2);
Cpositive = P >= eye(2);
residualDegree = residual.Degree
decayConstraintCount = numel(Cdecay.Constraints)
positivityConstraintCount = numel(Cpositive.Constraints)
```

```
residualDegree =
    3
decayConstraintCount =
    16
positivityConstraintCount =
    6
```

The decay count is two physical cells times two rate vertices times four degree-three labels. Positivity has no rate rows and has three degree-two labels per cell. Shared faces are visited from each adjoining cell because certification remains cell-local.

1.8.5 Choose a Finite Certificate

Direct Bernstein-coefficient constraints are the default. Three opt-in selectors rebuild from the same original residual and relation; selections replace rather than compound one another.

```
Cpolya = Cdecay.applyPolya(1);
Cputinar = Cdecay.applyPutinar();
Cbox = Cdecay.applyFullBoxPreorder();
counts = [numel(Cdecay.Constraints), ...
    numel(Cpolya.Constraints), ...
    numel(Cputinar.Constraints), ...
    numel(Cbox.Constraints)]
```

```
counts =
    16    20    24    24
```

Direct tests coefficient signs. Pólya elevates the residual before testing. For this one-parameter residual, the default Putinar selector delegates to the full-box parity construction, so both return 24 stored constraints. In multiple parameters, Putinar uses the singleton-generator box module, whereas Full Box uses all box-generator products. All four are finite sufficient certificates for the continuous inequality. Failure of a selected certificate to yield feasibility does not prove that the continuous inequality itself is infeasible. Sections B.4 to B.8 give the corresponding mathematical constructions, while section B.10 states the implemented boundary.

1.8.6 Export to YALMIP and Recover the Solution

`toYalmip` returns exactly the stored finite constraints. It does not select a solver, add an objective, or insert a strictness margin.

```
Fdecay = toYalmip(Cdecay);
Fpositive = Cpositive.toYalmip();
opts = sdpsettings('solver', 'sedumi', 'verbose', 0);
sol = optimize([Fdecay, Fpositive], [], opts);
if sol.problem == 0
    solvedP = value(P);
end
```

The solver choice and `sol.problem` semantics belong to YALMIP and the installed solver. After a successful solve, `value(P)` replaces the symbolic coefficient payloads with numeric values while preserving the grid, degree, matrix shape, and cell ordering. Complete solver examples appear in Chapter 8; certificate syntax and order boundaries appear in Chapter 6.

Chapter 2

Install and Verify the Package

2.1 One-Call Installation

Start MATLAB in the checked-out repository and run the root-level installer once. The installer adds the package source tree recursively while excluding documentation, the Web site, independent SOS validation material, Git/agent metadata, and generated build directories. It does not download dependencies or edit `startup.m`.

```
report = install_pd_lmi();
```

The returned structure records the package root, the YALMIP root discovered on the current MATLAB path, the working SDP solver, the paths added by this run, and whether the path was persisted:

```
report.PackageRoot
report.YALMIPRoot
report.Solver
report.AddedPaths
report.Persisted
```

YALMIP must already be on the active MATLAB path. The installer validates `yalmip`, `sdpvar`, `sdpsettings`, `optimize`, and `getavailablesolvers`; it then probes available SDP solvers in the fixed commercial-first order MOSEK, COPT, SeDuMi, SDPT3, and LMILAB using a tiny bounded SDP. A failed probe falls through to the next candidate. If any precondition, shadow check, solver probe, or path persistence operation fails, the installer restores the original MATLAB path and stops with one of these stable errors:

```
install_pd_lmi:MissingYALMIP
install_pd_lmi:IncompleteYALMIP
install_pd_lmi:NoWorkingSDPSolver
install_pd_lmi:PathConflict
install_pd_lmi:PathSaveFailed
```

Repeated successful calls are idempotent.

2.2 Validate the Installation

After the installer has selected and recorded a working solver, run the current MATLAB test entry point from the repository root:

```
results = tests.run_all();
```

The entry point covers installer rollback and persistence behavior, helper utilities, `pdbase`, `pdmat`, `pdvar`, and `pdlmi`. YALMIP-backed tests and the solver smoke tests use the same automatic commercial-first solver policy as the installer; their diagnostics identify the actual solver used.

2.3 Run a Minimal Solver Example

The installer already rejects shadowed package classes before it persists the path. To inspect the selected solver and exercise the solver-facing assembly boundary, use the report together with a small known-data example:

```
report.Solver
A = pdmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine")
```

```
    CellSubscript      Expression
    -------------    ---------------

        {[1]}        "(1-alpha)*1 + alpha*3"
```

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
F = toYalmip(C);
Cputinar = C.applyPutinar();
Fputinar = toYalmip(Cputinar);
Cbox = C.applyFullBoxPreorder();
Fbox = toYalmip(Cbox);
```

`F` is the direct coefficient certificate; `Fputinar` and `Fbox` are the default Putinar and full-box certificates for the same stored residual. Any one can be passed to ordinary YALMIP calls. Use the selected solver from the report when an explicit solver setting is useful:

```
opts = sdpsettings('solver', report.Solver, 'verbose', 0);
sol = optimize(F, objective, opts);
```

2.4 Navigate the Reference

Category	Entries
Backend	`pdbase`, constructor, `cells`, `lbls`, `coeffs`, `ncell`, `ncoeff`, `npar`, `size`
Known data	`pdmat`, construction, `evaluate`, `bernsteinTable`, `disp`/`display`, `plot`, operators, matrix methods
Decision variables	`pdvar`, construction, `rhodiff`, affine/mixed algebra, matrix methods, comparisons
LMIs	`pdlmi`, construction, direct coefficient assembly, `applyPolya`, `applyPutinar`, `applyFullBoxPreorder`, `toYalmip`, solver examples

Chapter 3

pdbase: Grid and Storage Backend

pdbase is the developer-facing backend parent for pdmat and pdvar. Ordinary users should model known data with pdmat and decision expressions with pdvar. This section documents pdbase because it explains the shared storage, traversal, and inspection contract used by both public modeling classes.

3.1 API Map

Task	Function or field
Create backend object	pdbase
Inspect cells/labels	cells, lbls, coeffs
Count sizes	ncell, ncoeff, npar, size
Elevate stored coefficients	elevVals
Backend algebra mechanisms	elevLocalValues, bernElev, bernProd, mergeGrid
Read storage fields	GridInfo, LocalValues, metadata fields

3.2 Construct pdbase Objects

Syntax

```
obj = pdbase(gridVectors, matrixSize, degree)
obj = pdbase(gridVectors, matrixSize, degree, localValues)
obj = pdbase(..., IsContinuous=tf)
obj = pdbase(..., ContainsDecision=tf)
obj = pdbase(..., HasRateDependence=tf)
obj = pdbase(..., RateBounds=rb)
obj = pdbase(..., SourceSummary=txt)
```

Arguments

- gridVectors is a cell vector with one strictly increasing node vector per parameter, such as {[0 1 2], [10 20]}.
- matrixSize is a positive two-element matrix size, such as [2 2].
- degree is a nonnegative integer Bernstein degree, such as 1.
- localValues is the nested physical-cell storage. When omitted, pdbase creates a zero coefficient-backed object.

- `IsContinuous`, `ContainsDecision`, and `HasRateDependence` are logical metadata scalars. They default to false; subclasses set them to describe their own payload contract.
- `RateBounds` is empty or an $\ell \times 2$ matrix of finite lower and upper rate bounds, such as `RateBounds=[-1 1; -2 2]`. Nonempty bounds also mark the object as rate-dependent.
- `SourceSummary` is inspectable origin text and defaults to `"coefficient-backed"`.

Examples and output

This backend example constructs a two-parameter parent object. The object has two physical cells because the first parameter has two intervals and the second parameter has one interval.

```
obj = pdbase({[0 1 2], [10 20]}, [2 2], 1);
objectClass = class(obj)
matrixSize = obj.MatrixSize
degree = obj.Degree
cellCount = obj.ncell()
coefficientCount = obj.ncoeff()
parameterCount = obj.npar()
objectSize = size(obj)
```

```
objectClass =
     'pdbase'

matrixSize =
2     2

degree =
1

cellCount =
2

coefficientCount =
4

parameterCount =
2

objectSize =
     2     2
```

The printed `ncoeff` value is $(\text{degree}+1)^{\text{parameterCount}} = 4$, one local Bernstein coefficient for each tensor-product label in the two parameter directions.

Remarks

- Malformed grids and node vectors raise `pdbase:InvalidGrid` or `pdbase:InvalidGridVector`.
- Invalid matrix sizes, degrees, or rate bounds raise `pdbase:InvalidMatrixSize`, `pdbase:InvalidDegree`, or `pdbase:InvalidRateBounds`.
- The nested tree must contain one leaf per physical cell and $(m+1)^{\ell}$ coefficient columns per leaf. Violations raise `pdbase:InvalidLocalValues` or `pdbase:InvalidCoefficientCell`.

- Payload matrices must have the declared size and be finite, real, and numeric or supported real YALMIP expressions. Malformed payloads raise `pdbase:InvalidCoefficientPayload`.
- Rate-affine payloads and `HasRateDependence=true` require compatible nonempty `RateBounds`.

3.3 Inspect Grid and Storage Properties

Every `pdbase` property below is publicly readable through dot access but has private set access. Users inspect `obj.Degree` or `obj.GridInfo.Vectors`; construction and package algebra create new objects instead of assigning these properties.

Property	Meaning
`GridInfo`	Grid vectors, node counts, and grid bounds.
`MatrixSize`	Size of each matrix coefficient or matrix expression.
`Degree`	Local Bernstein degree stored in every physical cell.
`LocalValues`	Nested cell tree indexed by physical-cell subscript; each leaf stores coefficient cells.
`IsContinuous`	Metadata flag; subclasses are responsible for boundary sharing if continuity is true.
`ContainsDecision`	True when coefficient payloads include YALMIP decision variables.
`HasRateDependence`	True when the object carries rate metadata or rate-vertex rows.
`RateBounds`	Parameter-rate lower and upper bounds used by `rhodiff` and `pdlmi`.
`SourceSummary`	Short origin label such as `coefficient-backed`, `decision`, or `derivative`.

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);
degree = obj.Degree
matrixSize = obj.MatrixSize
firstGrid = obj.GridInfo.Vectors{1}
source = obj.SourceSummary
```

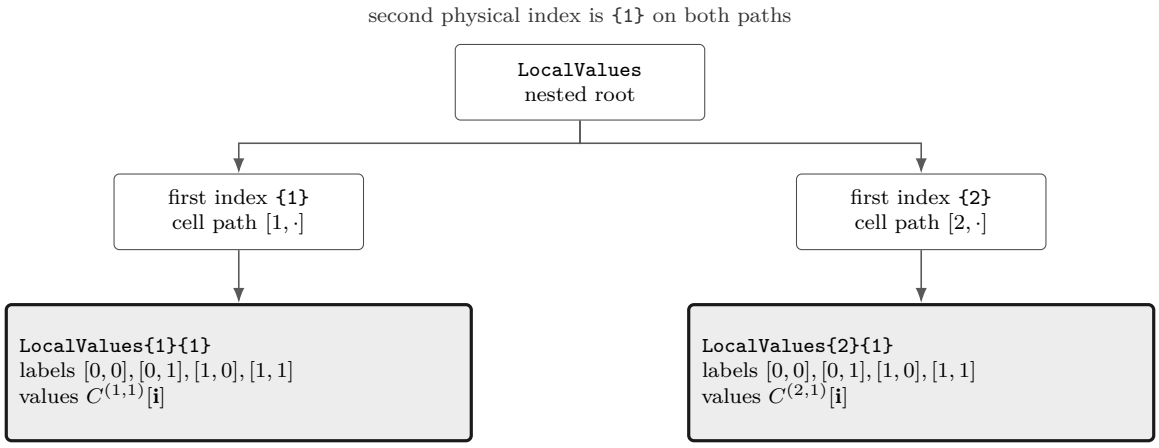


Figure 3.1: Nested cell-local storage for a two-parameter, degree-one object on `{[0 1 2], [10 20]}`. There are two physical cells, $[1, 1]$ and $[2, 1]$. Each leaf is a flat local coefficient cell in `lbls` order: $[0, 0]$, $[0, 1]$, $[1, 0]$, $[1, 1]$.

The nesting follows physical-cell subscripts, not the Bernstein labels. Start with `LocalValues{i1}`, then take one cell index per remaining parameter; this selects physical cell $[i_1, i_2, \ldots, i_\ell]$. Only after that selection does the leaf's flat cell array enumerate the $(m+1)^\ell$ local Bernstein coefficients. Thus `coeffs(obj, [2 1])` returns the lower leaf in Fig 3.1, while `lbls(obj)` explains the order inside that leaf.

3.4 Inspect `cells`, `lbls`, and `coeffs`

Syntax

```
subs = cells(obj)
subs = obj.cells()

labels = lbls(obj)
labels = obj.lbls()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)
```

Arguments

- `obj` is a `pdbase` object or a `pdmat`/`pdvar` subclass instance using the shared cell-local storage convention.
- `cellSubs` selects one physical cell for `coeffs`. Supply a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `{2,1}`.
- `cells` returns physical hypercube subscripts in the shared traversal order;
- `lbls` returns local Bernstein labels in the shared coefficient order.
- `c` is the flat coefficient cell array for the selected physical cell, ordered consistently with `lbls(obj)`. Rate-dependent leaves return one row per active rate vertex in the stored row order.

Remarks

`coeffs` requires one positive integer physical-cell index per parameter, within the corresponding cell count. Invalid numeric or cell-array selectors raise `pdbase:InvalidCellSubs`.

Examples and output

The label order is shared by `pdmat`, `pdvar`, and `pdlmi`. In this two-parameter degree-one object, each physical cell stores four coefficient entries with labels $[0,0]$, $[0,1]$, $[1,0]$, and $[1,1]$.

```
obj = pdbase({[0 1 2], [10 20]}, [1 1], 1);
labels = lbls(obj)
physicalCells = cells(obj)
```

```
c = coeffs(obj, [2 1]);
coefficientCount = numel(c)
```

```
labels =
     0     0
     0     1
     1     0
     1     1

physicalCells =
1     1
2     1

coefficientCount =
     4
```

3.5 Inspect Counts and Shape

Syntax

```
n = ncell(obj)
n = obj.ncell()

n = ncoeff(obj)
n = obj.ncoeff()

n = npar(obj)
n = obj.npar()

sz = size(obj)
d = size(obj, dim)
```

Arguments

- `obj` is a `pdbase` object or subclass instance.
- `dim` is a positive integer matrix dimension for `size`; it follows MATLAB's ordinary `size(obj,dim)` convention.
- `ncell` returns $\prod_s (k_s - 1)$, the number of physical hypercube cells.
- `ncoeff` returns $(m+1)^\ell$, the number of local Bernstein coefficients stored per physical cell.
- `npar` returns the parameter dimension ℓ.
- `size` returns the matrix payload size, never the physical grid size.

Examples and output

For a two-parameter ($\ell = 2$) degree-one backend object, the first grid has two physical intervals and the second has one. The matrix payload is 2-by-3, independently of the grid shape. Thus $n_{\mathrm{cell}} = (3-1)(2-1) = 2$ and each cell stores $n_{\mathrm{coeff}} = (1+1)^2 = 4$ entries, ordered by the explicit labels $[0,0], [0,1], [1,0], [1,1]$.

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);
physicalCellCount = obj.ncell()
coefficientCount = obj.ncoeff()
parameterCount = obj.npar()
matrixSize = size(obj)
rowCount = size(obj, 1)
columnCount = size(obj, 2)
thirdDimension = size(obj, 3)
```

```
physicalCellCount =
     2
coefficientCount =
     4
parameterCount =
     2
matrixSize =
     2     3
rowCount =
     2
columnCount =
     3
thirdDimension =
     1
```

3.6 Transform Bernstein Storage

The following entries separate whole-object elevation from the protected helpers used to align degrees, convolve coefficients, and refine grids. They change coefficient representations or temporary storage trees; they do not change the meanings of `ncell`, `ncoeff`, `npar`, or `size` described above.

3.6.1 `elevVals`

Syntax

```
vals = obj.elevVals(degreeIncrement)
```

Arguments

- `degreeIncrement` is a finite nonnegative integer scalar. It is added to `obj.Degree` in every parameter direction; zero returns coefficient evidence at the current degree.

Description

`vals` is a nested `LocalValues`-shaped cell tree. Each physical cell contains coefficients of degree `obj.Degree + degreeIncrement`, with the represented polynomial, physical-cell tree, and any rate-row ordering preserved. The method does not modify `obj` or its stored coefficient evidence.

Examples and output

```
A = pdmat({[0 1]}, {1, 3}, Degree=1);
elevated = A.elevVals(1)
```

```
elevated =
    {[1 2 3]}
```

Remarks

An invalid increment raises `pdbase:InvalidDegreeIncrement`. A function-only `pdmat` has no stored Bernstein coefficient evidence, so this method raises `pdbase:MissingCoefficientEvidence`.

Relationship to backend and public algebra. Unlike protected `bernElev`, which elevates one flat cell coefficient family for internal alignment, `elevVals` is a public whole-object query returning elevated `LocalValues`. Public coefficient algebra uses `bernElev` internally when compatible operands need degree alignment; it does not mutate either operand.

3.6.2 `elevLocalValues`

Internal/protected mechanism. This method elevates a temporary nested coefficient tree after public algebra has remapped operands to a common physical grid. It is documented as backend context, not as a supported user call form.

Syntax

```
vals = elevLocalValues(obj, vals, fromDeg, toDeg)
vals = elevLocalValues(obj, vals, fromDeg, toDeg, grid)
```

Each leaf must be a cell array with one column per degree-`fromDeg` Bernstein label. The method elevates each rate-vertex row independently to `toDeg`; it never mixes rows. The optional `grid` supplies the temporary physical-cell layout, and otherwise defaults to `obj.GridInfo.Vectors`. Equal source and target degrees return the input tree unchanged. A malformed leaf raises `pdbase:InvalidCoefficientCell`. Public users encounter this behavior through grid merging, assignment, concatenation, and arithmetic rather than by calling the protected method directly.

Examples and output

```
A = pdmat({[0 1]}, @(rho) rho, Degree=1);
B = pdmat({[0 .5 1]}, @(rho) 2*rho, Degree=2);
C = A + B;
mergedGrid = C.GridInfo.Vectors{1}
resultDegree = C.Degree
cell1Coefficients = cell2mat(C.coeffs(1))
cell2Coefficients = cell2mat(C.coeffs(2))
AGrid = A.GridInfo.Vectors{1}
ADegree = A.Degree
BGrid = B.GridInfo.Vectors{1}
BDegree = B.Degree
```

```
mergedGrid =
     0    0.5000    1.0000

resultDegree =
     2

cell1Coefficients =
     0    0.7500    1.5000

cell2Coefficients =
    1.5000    2.2500    3.0000

AGrid =
     0     1

ADegree =
     1

BGrid =
     0    0.5000    1.0000

BDegree =
     2
```

The example exercises the protected mechanism through public `pdmat` addition; users do not call protected `elevLocalValues` directly.

3.6.3 `bernElev`

Internal/protected mechanism. This signature documents the backend algorithm used by public algebra; it is not a supported user call form.

Syntax

```
out = bernElev(obj, coeffs, fromDeg, toDeg)
```

Arguments

- `coeffs` is a flat coefficient-cell family in the same label order as `lbls(obj)`.
- `fromDeg` and `toDeg` are nonnegative integer degrees, with `toDeg >= fromDeg`.
- `out` contains $(\texttt{toDeg}+1)^{\ell}$ elevated coefficients in the same label order.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
C = A + B;
degree = C.Degree
```

```
degree =
     2
```

The example invokes the protected helper through public addition; users do not call `bernElev` directly. Invalid degree values raise `pdbase:InvalidDegree`; a target below the source degree raises `pdbase:InvalidDegreeElevation`; a coefficient count inconsistent with the source degree and parameter dimension raises `pdbase:InvalidCoefficientCell`.

3.6.4 `bernProd`

Internal/protected mechanism. This signature documents the backend algorithm used by public matrix multiplication; it is not a supported user call form.

Syntax

```
out = bernProd(obj, lhs, lhsDeg, rhs, rhsDeg)
```

Arguments

- `lhs` and `rhs` are flat coefficient-cell families whose matrix payload inner dimensions are compatible for multiplication.
- `lhsDeg` and `rhsDeg` are their nonnegative local Bernstein degrees.
- `out` is the binomial-normalized coefficient convolution at degree `lhsDeg + rhsDeg`; payload products follow MATLAB matrix multiplication rules.

For one parameter, the protected helper implements

$$C[k] = \sum_{i+j=k} \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}} A[i]B[j].$$

It enumerates ordered payload products, groups them by the anti-diagonal $i + j = k$, and only then applies the binomial normalization. For tensor labels the same rule is applied componentwise. See the public `pdmat` multiplication entry in section 4.6.2, the `pdvar` known-data boundary in section 5.6, and the full derivation in Appendix A.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
P = A * A;
degree = P.Degree
coefficients = cell2mat(P.coeffs(1))
```

```
degree =
     2
coefficients =
     1     2     4
```

The example invokes the protected helper through public matrix multiplication; users do not call `bernProd` directly. Even for scalar data, the middle coefficient comes from two ordered contributions rather than from one sample.

3.6.5 `mergeGrid`

Internal/protected mechanism. This signature documents the backend grid-refinement algorithm used by public algebra; it is not a supported user call form.

Syntax

```
grid = obj.mergeGrid(errId, other1, other2, ...)
```

Arguments

- `errId` is the error identifier used when parameter counts or physical bounds are incompatible.
- `other1, other2, ...` are coefficient-backed operands, including function-backed operands constructed with an explicit `Degree`. They must have equal parameter counts and matching first and last nodes on every parameter axis.
- `grid` has the sorted union of the interior nodes. Public algebra re-expresses operands on these cells before degree elevation, coefficient-wise addition, or product convolution.

This protected helper is invoked by public algebra methods rather than called directly by users.

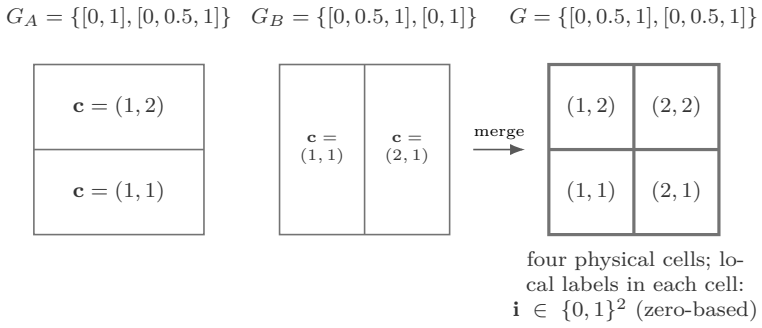
Examples and output

```
gridA = {[0 1], [0 0.5 1]};
gridB = {[0 0.5 1], [0 1]};
A = pdmat(gridA, @(rho1, rho2) rho1 + 2*rho2, Degree=1);
B = pdmat(gridB, @(rho1, rho2) 2*rho1 - rho2, Degree=1);
C = A + B;
rho1Grid = C.GridInfo.Vectors{1}
rho2Grid = C.GridInfo.Vectors{2}
```

```
rho1Grid =
0    0.5000    1.0000

rho2Grid =
0    0.5000    1.0000
```

The example invokes the protected helper through public addition; users do not call `mergeGrid` directly. Here $\ell = 2$. The physical-cell index $\mathbf{c} = (c_1, c_2)$ is one-based, while the degree-one local Bernstein label $\mathbf{i} = (i_1, i_2) \in \{0, 1\}^2$ is zero-based inside every cell.



Each axis uses the sorted union of its input nodes. The result is the unique minimal same-bound tensor grid containing both inputs.

3.7 Boundaries and Related APIs

`pdmat`, `pdvar`, `cells`, `coeffs`, `lbls`, `ncell`, `ncoeff`, `npar`, `size`, `elevVals`, `elevLocalValues`, `bernElev`, `bernProd`, `mergeGrid`.

Chapter 4

pdmat: Known Parameter-Dependent Matrices

`pdmat` stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle, but do not expose coefficient evidence for `bernsteinTable` or coefficient algebra.

4.1 API Map

Task	Entry
Construct data	`pdmat`
Evaluate	`evaluate`
Inspect/display	`bernsteinTable`, `disp`, `display`
Plot	`plot`
Arithmetic	`+`, `-`, unary `+/-`, `*`
Matrix/index methods	shape and structural methods, indexing and assignment

4.2 Construct pdmat Objects

Syntax

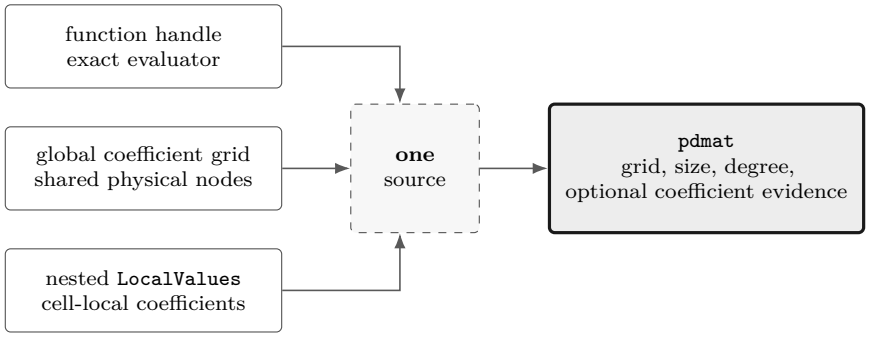
```
A = pdmat(gridVector, source)
A = pdmat(gridVector, source, Degree=m)
A = pdmat(gridVectors, source)
A = pdmat(gridVectors, source, Degree=m)
```

Arguments

- `gridVector` is a numeric vector used as a one-parameter grid shorthand, such as `[0 1 2]`.
- `gridVectors` is a cell vector with one numeric grid vector per parameter, such as `{[0 1], [10 20]}`.
- `source` may be:
    1. A function handle, e.g., `@(rho) [rho rho^2]` is a one-parameter function with two matrix entries.

2. A global cell grid of numeric Bernstein coefficients, e.g., `{[1 2], [3 4]}` is a two-cell degree-one grid with two 1×1 coefficients per cell.
3. A explicit nested `LocalValues`, e.g., `{1, 2, 3}` is scalar degree-two data on one cell.

- `Degree=m` sets the local Bernstein degree. For function handles, omitting `Degree` creates a function-only object; passing a value such as `Degree=2` validates and stores Bernstein coefficient evidence while retaining the exact function handle for `evaluate`.



Examples and output

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

```
A = pdmat([0 1 2], {10, 20, 30}, Degree=1);
disp(A)
matrixSize = size(A)
degree = A.Degree
sourceSummary = A.SourceSummary
```

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed

matrixSize =
1     1

degree =
1

sourceSummary =
    "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. The source cell array matches the global coefficient grid implied by the two parameter directions.

```
B = pdmat({[0 1], [10 20]}, {1 2; 3 4}, Degree=1);
disp(B)
secondGridVector = B.GridInfo.Vectors{2}
labels = B.lbls()
```

```
pdmat [1 1] over 2-D grid, degree 1, source coefficient-backed

secondGridVector =
10    20
```

```
labels =
     0     0
     0     1
     1     0
     1     1
```

Use explicit nested `LocalValues` when each physical cell must be written directly. Here each cell stores two 1×2 row-vector coefficients.

```
localVals = {{[1 0], [0 1]}, {[0 1], [0 2]}};
C = pdmat({[0 1 2]}, localVals, Degree=1);
disp(C)
matrixSize = size(C)
secondCellCoefficients = C.coeffs(2)
```

```
pdmat [1 2] over 1-D grid, degree 1, source coefficient-backed

matrixSize =
1     2

secondCellCoefficients =
  1x2 cell array
    {[2 0]}    {[0 2]}
```

When `source` is a function handle and `Degree` is omitted, the object keeps the exact evaluator but does not claim coefficient evidence.

```
F = pdmat({[0 1]}, @(rho) [rho rho^2]);
disp(F)
sourceSummary = F.SourceSummary
evaluatedValue = F.evaluate(0.5)
```

```
pdmat [1 2] over 1-D grid, degree 1, source function

sourceSummary =
"function"

evaluatedValue =
     0.5000    0.2500
```

Add `Degree` for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

```
G = pdmat({[0 1]}, @(rho) rho.^2, Degree=2);
disp(G)
sourceSummary = G.SourceSummary
firstCellCoefficients = G.coeffs(1)
```

```
pdmat [1 1] over 1-D grid, degree 2, source function-bernstein

sourceSummary =
"function-bernstein"

firstCellCoefficients =
  1x3 cell array
    {[0]}    {[0]}    {[1]}
```

These forms all create finite known-data objects, but only the coefficient- backed and function-Bernstein forms can participate in coefficient algebra.

Remarks

`FunctionHandle` retains the supplied function handle for function-only and function-Bernstein sources; it is empty for cell-backed sources. Its set access is private.

- Constructor option errors use `pdmat:InvalidOptions`, `pdmat:UnsupportedOption`, or `pdmat:UnknownOption`; grid errors use `pdmat:InvalidGrid` or `pdmat:InvalidGridVector`.
- Source and function validation uses `pdmat:InvalidSource`, `pdmat:InvalidFunctionHandle`, `pdmat:InvalidFunctionOutput`, or `pdmat:InvalidData`.
- Degree and storage failures use `pdmat:InvalidDegree`, `pdmat:InvalidLocalValues`, `pdmat:InvalidCoefficientCell`, or `pdmat:InvalidCoefficientPayload`.
- With explicit `Degree`, nonrepresentable functions raise `pdmat:NonBernsteinPolynomial`; invalid validation probes raise `pdmat:InvalidFunctionValue`.
- Mismatched shared faces are retained as discontinuous local data and emit warning `pdmat:DiscontinuousLocalValues`.

4.3 Inspect Object Properties

`pdmat` adds the read-only `FunctionHandle` property to the inherited `pdbase` properties in section 3.3. All use private set access. `FunctionHandle` contains the supplied handle for function and function-Bernstein sources and is empty for cell-backed sources. The inherited properties describe the grid, matrix size, Bernstein degree, cell-local evidence, continuity, and origin.

```
A = pdmat({[0 1]}, @(rho) rho.^2, Degree=2);
degree = A.Degree
source = A.SourceSummary
hasFunction = ~isempty(A.FunctionHandle)
firstCell = A.LocalValues{1}
```

Assigning `A.Degree`, `A.LocalValues`, or `A.FunctionHandle` is not a supported update operation; construct a new `pdmat` instead.

4.4 `evaluate`

Syntax

```
val = evaluate(A, point)
val = A.evaluate(point)
```

Arguments

- `A` is a `pdmat` object.
- `point` is a finite real scalar for a one-parameter object, such as `0.25`, or a finite real row vector with one entry per parameter, such as `[0.25 12]`. Every entry must lie within the corresponding grid bounds.
- `val` is a numeric matrix with `size(A)`. Coefficient-backed objects use local Bernstein interpolation; function-backed objects call their stored function handle.

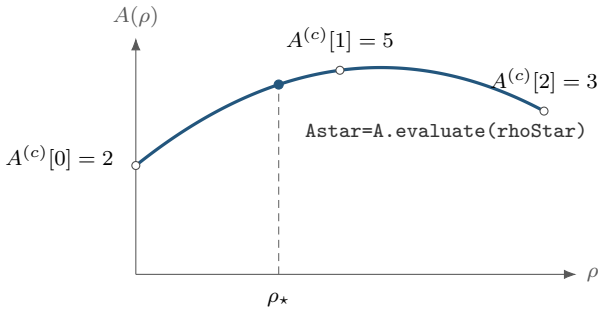
Remarks

A point with the wrong type, length, or finite-real shape raises `pdmat:InvalidPoint`; a point outside any grid bound raises `pdmat:PointOutOfBounds`. If a stored function fails or returns a nonfinite, complex, empty, or size-inconsistent matrix at the requested point, `evaluate` raises `pdmat:InvalidFunctionValue`.

For a degree-one scalar coefficient-backed object on $[\rho_1^{(1)}, \rho_1^{(2)}]$,

$$A(\rho) = (1-\alpha)A^{(c)}[0] + \alpha A^{(c)}[1], \qquad \alpha = \frac{\rho_1 - \rho_1^{(1)}}{\rho_1^{(2)} - \rho_1^{(1)}}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.



Evaluation follows the degree-two Bernstein curve with coefficients $[2, 5, 3]$*; the dashed line locates the requested physical point.*

Examples and output

Use the function-call form when the object is naturally one input among several MATLAB expressions.

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = evaluate(A, 0.25)
```

```
val =
     2.5000
```

At $\rho = 0.25$, the public local coordinate is $\alpha = 0.25$, so the interpolated value is $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$. The dot-call form is equivalent and can be convenient in command-window exploration.

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = A.evaluate(0.75)
```

```
val =
     3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

```
F = pdmat({[0 pi]}, @(rho) sin(rho));
val = F.evaluate(pi/2)
```

```
val =
     1
```

4.5 Inspect Coefficients and Plot Values

4.5.1 `bernsteinTable`

Syntax

```
T = bernsteinTable(A)
T = bernsteinTable(A, cellSubs)
T = bernsteinTable(A, "oneLine")
T = bernsteinTable(A, cellSubs, "oneLine")
```

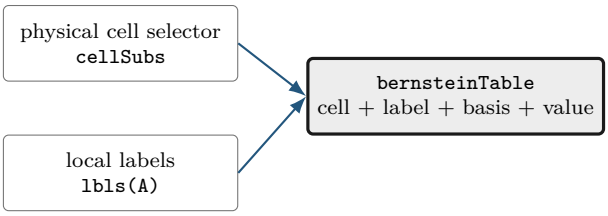
Arguments

- `cellSubs` selects one physical cell.
- `"oneLine"` returns one row per selected physical cell with a compact Bernstein expression.
- Without `"oneLine"`, the full table uses seven metadata columns; the output below shows their exact MATLAB names.

For one parameter, the `Basis` column reports

$$\binom{m}{j}(1-\alpha)^{m-j}\alpha^{j}.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate, using `alpha1`, `alpha2`, and so on. `LocalIndex` is the local Bernstein label. `CoeffSubscript` maps the local label back to the global coefficient subscript on the physical grid. `IsPhysicalNode` is true when that coefficient lies on a physical grid node.



The table reports stored coefficient evidence. It does not sample a function and does not create solver constraints.

Examples and output

```
A = pdmat({[0 0.2 1]}, {[0 1], [1 1], [1 2]}, Degree=1);
bernsteinTable(A, "oneLine")
bernsteinTable(A, 1, "oneLine")
```

```
    CellSubscript          Expression
    -------------    ------------------------

        {[1]}        "(1-alpha)*[0 1] + alpha*[1 1]"
        {[2]}        "(1-alpha)*[1 1] + alpha*[1 2]"

        CellSubscript          Expression
            -------------    ------------------------

            {[1]}        "(1-alpha)*[0 1] + alpha*[1 1]"
```

Use the full table when the metadata columns are the point of the example:

```
A = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
bernsteinTable(A)
```

```
    TermIndex    CellSubscript    CoeffSubscript    LocalIndex          Basis          IsPhysicalNode    Value
    ---------    -------------    --------------    ----------    -----------------    --------------    -----

        1            {[1]}            {[1]}           {[0]}       "(1-alpha)^2"            true          {[1]}
        2            {[1]}            {[2]}           {[1]}       "2(1-alpha)alpha"        false         {[2]}
        3            {[1]}            {[3]}           {[2]}       "alpha^2"                true          {[3]}
```

Remarks

A function-only object has no coefficient evidence and raises `pdmat:FunctionOnlyBernsteinTable`. Unknown text options, repeated selectors, or invalid physical-cell selectors raise `pdmat:InvalidBernsteinTableInput`.

4.5.2 `disp` And `display`

Syntax

```
disp(A)
display(A)
A
```

Arguments

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used when an expression is not terminated by a semicolon; it prints grid, degree, coefficient, and source metadata.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
display(A)
```

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed
A =
  pdmat with 1-by-1 matrix values
  Parameters: 1
    rho_1: [0, 1], 2 nodes
  Degree: 1
  Physical cells: 1
  Coefficients per cell: 2
  Continuous: true
  Source: coefficient-backed
  Function handle: false
```

4.5.3 `plot`

Syntax

```
h = plot(A)
h = plot(A, dims)
h = plot(A, SamplesPerCell=n)
h = plot(A, dims, SamplesPerCell=n)
h = plot(A, ..., Name, Value)
```

Arguments

- `dims` is one or two parameter indices. For one-parameter objects, the default is `1`; otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.
- `SamplesPerCell` is the package-owned sampling option. The default is `15`; use `SamplesPerCell=4` for a coarse diagnostic plot.

- Remaining name-value pairs pass through to MATLAB graphics. One- dimensional plots pass them to `plot`; two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

Remarks

An invalid parameter index, a repeated dimension, or a request for more than two plotting dimensions raises `pdmat:InvalidPlotDimensions`. Malformed plotting options or a nonpositive integer `SamplesPerCell` raise `pdmat:InvalidPlotOptions`. Errors from MATLAB graphics options remain MATLAB graphics errors. Every unselected parameter is fixed at its lower grid bound, `A.GridInfo.Bounds(:,1)`. A plot of a three-parameter object is therefore a one- or two-dimensional slice, not a volume plot.

Examples and output

Each source listing below is the complete source used by `generate_plot_figures.m` for the adjacent result.

One-dimensional entries.

```
A = pdmat({[-1 1]}, @(rho) [rho, rho.^2]);
h = plot(A, SamplesPerCell=80, LineWidth=2);
set(h(1), "Color", [35 86 125]/255, "LineStyle", "-")
set(h(2), "Color", [0.25 0.25 0.25], "LineStyle", "--")
grid on
xlabel("rho")
ylabel("matrix entry")
title("One-dimensional pdmat entries")
legend(["A(1,1)", "A(1,2)"], "Location", "eastoutside")
```

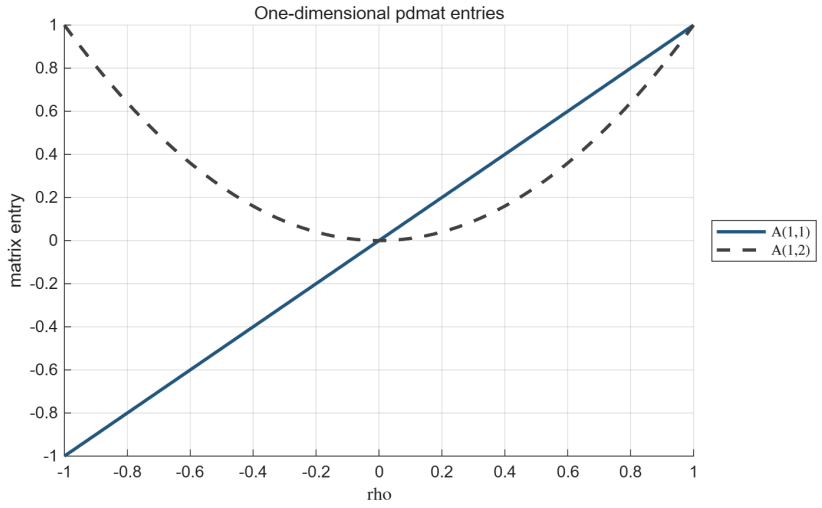


Figure 4.1: Result of the one-dimensional source above. The method returns one line handle per matrix entry.

Two-dimensional surface.

```
A = pdmat({[-1 1], [0 2]}, ...
    @(rho1, rho2) 1 + rho1.^2 + 0.5*rho2 + rho1.*rho2);
h = plot(A, [1 2], SamplesPerCell=45, ...
    EdgeColor="none", FaceAlpha=0.78);
set(h, "FaceColor", [35 86 125]/255)
grid on
view(35, 25)
xlabel("rho1", "Interpreter", "none")
ylabel("rho2", "Interpreter", "none")
zlabel("matrix entry")
title("Two-dimensional pdmat surface")
```

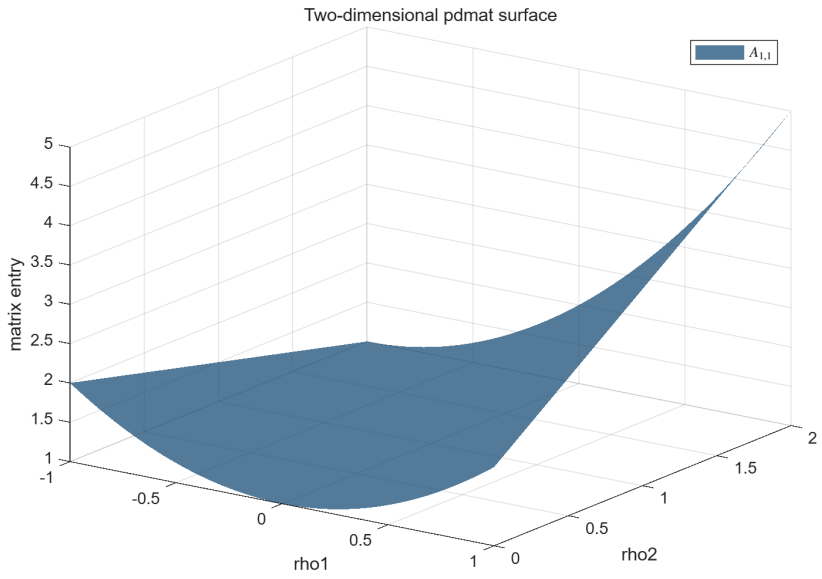


Figure 4.2: Result of the two-dimensional source above.

Two-dimensional matrix-valued surface.

```
A = pdmat({[-1 1], [0 2]}, @(rho1, rho2) [ ...
    1 + 0.8*rho1 + 0.25*rho2; ...
    1 - 0.8*rho1 + 0.25*rho2]);
h = plot(A, [1 2], SamplesPerCell=45, ...
    EdgeColor="none", FaceAlpha=0.62);
set(h(1), "FaceColor", [35 86 125]/255)
set(h(2), "FaceColor", [0.55 0.55 0.55])
grid on
view(35, 25)
xlabel("rho1", "Interpreter", "none")
ylabel("rho2", "Interpreter", "none")
zlabel("matrix entry")
title("Two entries of a 2-by-1 pdmat cross at rho1 = 0")
legend(h, ["A(1,1)", "A(2,1)"], ...
    "Location", "eastoutside")
```

The two entries are equal for every ρ_2 along $\rho_1 = 0$. Their ordering reverses on opposite sides: the first entry is lower when $\rho_1 < 0$ and higher when $\rho_1 > 0$. The crossing is therefore a property of the matrix function, not an artificial plotting offset.

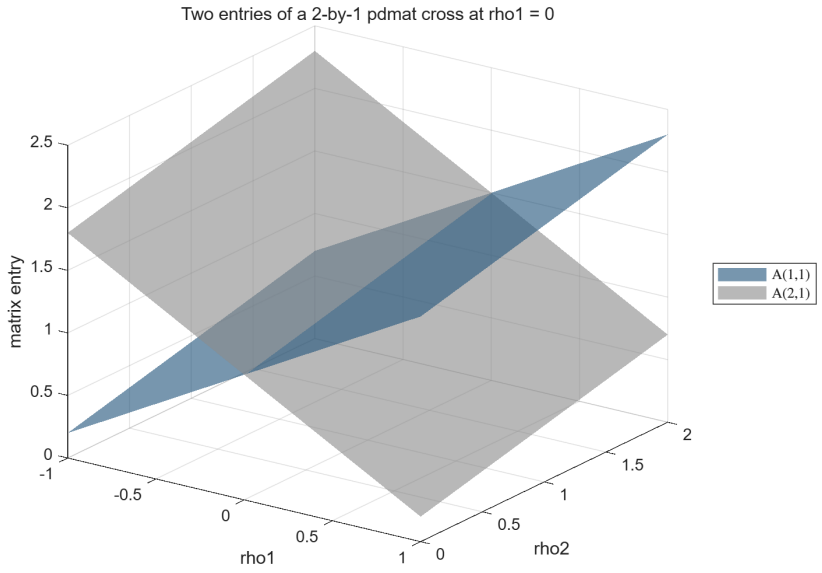


Figure 4.3: Two entry surfaces of a 2-by-1 matrix-valued function. The outside legend distinguishes the entries while their intersection remains visible along $\rho_1 = 0$.

4.6 Combine Known Matrices

A `pdmat` operation acts on complete matrix coefficients while the physical grid remains attached to the result. The blocks below deliberately show one operation family at a time: purpose, supported syntax, input, immediate output, and—when the result is spatial—a before/after picture.

4.6.1 Signs, Addition, And Subtraction

Use signs, sums, and differences to build known matrix expressions. Compatible physical bounds are refined to the union grid, and lower-degree operands are elevated exactly before matching coefficients are combined. The corresponding MATLAB overload names are `plus`, `minus`, `uplus`, and `uminus`.

Syntax

```
S = A + B
S = A + M
S = M + A
D = A - B
D = A - M
D = M - A
P = +A
N = -A
```

Here `M` is a finite real scalar or size-compatible matrix. A scalar broadcasts to the matrix payload size. The first example makes the degree alignment visible.

Examples and output

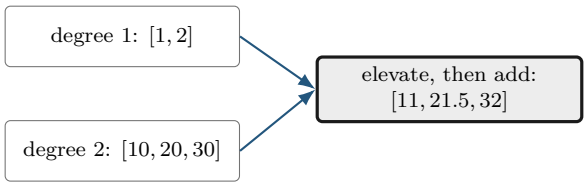
```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B;
sumDegree = S.Degree
sumCoefficients = cell2mat(S.coeffs(1))
```

```
sumDegree =
     2
sumCoefficients =
    11.0000   21.5000   32.0000
```

The middle coefficient of the elevated A is $(1+2)/2 = 1.5$, so the middle sum is $1.5 + 20 = 21.5$. Subtraction and unary signs act immediately on the same local coefficients.

```
D = 5 - A;
N = -A;
differenceCoefficients = cell2mat(D.coeffs(1))
negativeAtOne = N.evaluate(1)
positiveClass = class(+A)
```

```
differenceCoefficients =
     4     3
negativeAtOne =
     -2
positiveClass =
    'pdmat'
```



When grids have the same outer bounds but different interior nodes, the union grid is used before

addition or subtraction. For example, `pdmat({[0 1]},...)+pdmat({[0 .5 1]},...)` has grid `[0 .5 1]`; each operand is represented exactly on both resulting cells.

4.6.2 Ordered Matrix Multiplication

Use `mtimes` for a matrix product. The inner payload dimensions must match. Two parameter-dependent factors produce the degree sum; a numeric factor is constant and does not raise the degree.

Syntax

```
K = A * B
K = A * M
K = M * A
```

For one parameter, fix physical cell c and let $A = \sum_i B_i^m A^{(c)}[i]$ and $D = \sum_j B_j^n D^{(c)}[j]$. Then $K = AD$ has coefficients

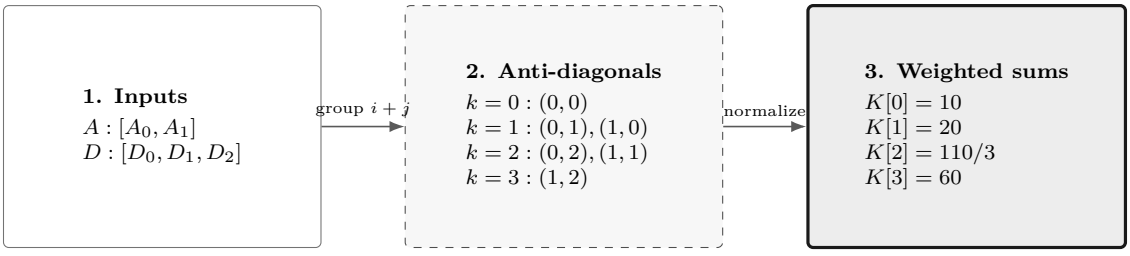
$$K^{(c)}[k] = \sum_{i+j=k} \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}} A^{(c)}[i] D^{(c)}[j].$$

The left/right matrix order is never exchanged. It is worthnoting that the above formula is a weighted convolution of the local coefficients, not a simple sum of products. The binomial weights are the Bernstein basis evaluations at the local coordinate $\alpha = i/(m+n)$ for each anti-diagonal $i + j = k$.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
K = A * B;
productDegree = K.Degree
productCoefficients = cell2mat(K.coeffs(1))
```

```
productDegree =
     3
productCoefficients =
   10.0000   20.0000   36.6667   60.0000
```



Coefficient labels add under the weighted convolution; payload matrices multiply in the written order. The three panels make the convolution readable without crossing contribution arrows.

For tensor data, label addition and binomial weights are applied independently in each parameter direction. If $\mathbf{m}, \mathbf{n} \in \mathbb{N}^{\ell}$ are direction-wise basis degrees, the ℓ-dimensional ordered convolution is

$$K^{(\mathbf{c})}[\mathbf{k}] = \sum_{\mathbf{i}+\mathbf{j}=\mathbf{k}} \left(\prod_{s=1}^{\ell} \frac{\binom{m_s}{i_s}\binom{n_s}{j_s}}{\binom{m_s+n_s}{k_s}} \right) A^{(\mathbf{c})}[\mathbf{i}] D^{(\mathbf{c})}[\mathbf{j}].$$

Two degree-one factors in two parameters produce degree two and $3^2 = 9$ coefficients per physical cell.

Remarks

Incompatible operands for `+`, `-`, and `*` raise `pdmat:InvalidAddition`, `pdmat:InvalidSubtraction`, or `pdmat:InvalidMultiplication`. Parameter counts or outer bounds that cannot be merged raise `pdmat:MixedGrid`. Coefficient algebra on a function-only object raises `pdmat:FunctionOnlyAlgebra`; construct the function with an explicit verified `Degree` when coefficient evidence is required.

4.7 Reshape and Transform Known Matrices

These methods transform every local matrix coefficient in the same way. They do not transpose, reshape, or concatenate the physical grid. Except for the documented evaluator/display/shape paths, structural methods require coefficient evidence.

4.7.1 Shape, Equality, And Display Protocols

Shape queries describe the matrix payload, not the grid. Equality compares coefficient-backed functions after compatible grid refinement and degree elevation. The protocol methods support ordinary MATLAB indexing syntax and dot access; they are not a second storage API.

Syntax

```
sz = size(A)
d = size(A, dim)
n = height(A)
n = width(A)
n = length(A)
n = numel(A)
n = ndims(A)
tf = isequal(A, B, ...)
idx = end(A, k, n)
n = numArgumentsFromSubscript(A, S, ctx)
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
matrixSize = size(A)
rowCount = height(A)
columnCount = width(A)
```

```
longestDimension = length(A)
elementCount = numel(A)
dimensionCount = ndims(A)
equalsUnaryPlus = isequal(A,+A)
```

```
matrixSize =
     2     3
rowCount =
     2
columnCount =
     3
longestDimension =
     3
elementCount =
     6
dimensionCount =
     2
equalsUnaryPlus =
  logical
   1
```

A function-only object can use shape queries and compares its stored metadata and function handle. A coefficient-backed equality that cannot form a common comparison grid raises `pdmat:InvalidEquality`. The detailed multi-line `display` transcript is shown in section 4.5.2.

4.7.2 Transpose And Conjugate Transpose

Both transpose forms act on every coefficient. Because `pdmat` payloads are finite and real, conjugation does not change their values.

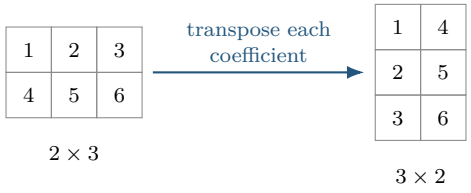
Syntax

```
T = transpose(A)
H = ctranspose(A)
T = A.'
H = A'
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
T = A.';
H = A';
transposeSize = size(T)
formsEqual = isequal(T,H)
transposeAtZero = T.evaluate(0)
```

```
transposeSize =
     3     2
formsEqual =
  logical
   1
transposeAtZero =
     1     4
     2     5
     3     6
```



4.7.3 Vectorization, Reshape, And Squeeze

Use these methods to change the matrix view while preserving column-major element order, the parameter grid, the degree, and every coefficient value. `reshape` accepts exactly two positive dimensions with at most one inferred `[]` dimension. `squeeze` retains the package's two-dimensional matrix convention.

Syntax

```
V = vec(A)
R = reshape(A, [m n])
R = reshape(A, m, n)
S = squeeze(A)
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
V = vec(A);
R = reshape(A, [3 2]);
S = squeeze(A);
vectorSize = size(V)
```

```
reshapeSize = size(R)
squeezeSize = size(S)
vectorAtZero = V.evaluate(0).'
```

```
vectorSize =
     6     1
reshapeSize =
     3     2
squeezeSize =
     2     3
vectorAtZero =
     1     4     2     5     3     6
```

Malformed or element-count-changing reshape requests raise `pdmat:InvalidReshape`.

4.7.4 Diagonals And Trace

`diag` extracts a selected diagonal from a matrix, or builds a square diagonal matrix from a vector. A finite integer offset `k` may not select an empty diagonal. `trace` returns a 1-by-1 `pdmat`.

Syntax

```
d = diag(A)
d = diag(A, k)
D = diag(v)
D = diag(v, k)
t = trace(A)
```

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
d = diag(A);
D = diag(d);
t = trace(A);
diagonalAtZero = d.evaluate(0)
rebuiltAtZero = D.evaluate(0)
traceAtZero = t.evaluate(0)
```

```
diagonalAtZero =
     1
     4
rebuiltAtZero =
     1     0
     0     4
traceAtZero =
     5
```

Invalid offsets or empty selections raise `pdmat:InvalidDiag`.

4.7.5 Triangular Parts And Reductions

Triangular extraction preserves or zeros entries relative to a selected diagonal. Reductions act along matrix dimensions; `"all"` is supported by `sum` and `mean`.

Syntax

```
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
S = sum(A)
S = sum(A, dim)
S = sum(A, "all")
M = mean(A)
M = mean(A, dim)
M = mean(A, "all")
C = cumsum(A)
C = cumsum(A, dim)
```

First inspect the two triangular results.

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
L = tril(A);
U = triu(A);
lowerAtZero = L.evaluate(0)
upperAtZero = U.evaluate(0)
```

```
lowerAtZero =
     1     0
     3     4
upperAtZero =
     1     2
     0     4
```

Then apply reductions to the same coefficient matrix.

```
rowSum = sum(A, 2);
allMean = mean(A, "all");
running = cumsum(A, 2);
rowSumAtZero = rowSum.evaluate(0)
allMeanAtZero = allMean.evaluate(0)
runningAtZero = running.evaluate(0)
```

```
rowSumAtZero =
     3
     7
allMeanAtZero =
    2.5000
runningAtZero =
     1     3
     3     7
```

Invalid triangular offsets raise `pdmat:InvalidTriangularPart`. Invalid reduction calls raise `pdmat:InvalidSum`, `pdmat:InvalidMean`, or `pdmat:InvalidCumsum`.

4.7.6 Reordering And Rotation

These methods reorder rows or columns inside every coefficient; they never reverse the scheduling grid.

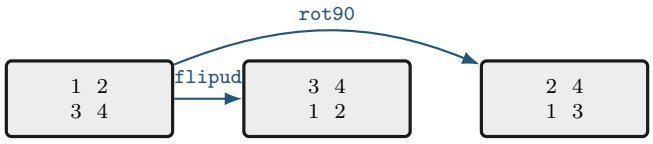
Syntax

```
B = flip(A)
B = flip(A, dim)
B = flipud(A)
B = fliplr(A)
B = rot90(A)
B = rot90(A, k)
```

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
F = flipud(A);
R = rot90(A);
flippedAtZero = F.evaluate(0)
rotatedAtZero = R.evaluate(0)
```

```
flippedAtZero =
     3     4
     1     2
rotatedAtZero =
     2     4
     1     3
```



Invalid dimensions or quarter-turn counts raise `pdmat:InvalidFlip` or `pdmat:InvalidRot90`.

4.7.7 Concatenation And Block Assembly

Concatenation joins payload matrices after compatible grid refinement and degree elevation. Numeric blocks are promoted to constant known data. `cat` supports only dimensions 1 and 2.

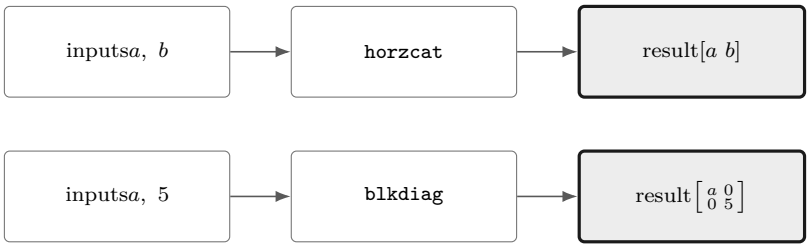
Syntax

```
H = horzcat(A, B, ...)
V = vertcat(A, B, ...)
H = [A, B]
V = [A; B]
C = cat(dim, A, B, ...)
D = blkdiag(A, B, ...)
```

Examples and output

```
a = pdmat({[0 1]}, {1, 2}, Degree=1);
b = pdmat({[0 1]}, {10, 20}, Degree=1);
H = [a, b];
V = [a; b];
D = blkdiag(a, 5);
horizontalAtZero = H.evaluate(0)
verticalAtZero = V.evaluate(0)
blockDiagonalAtZero = D.evaluate(0)
```

```
horizontalAtZero =
     1    10
verticalAtZero =
     1
    10
blockDiagonalAtZero =
     1     0
     0     5
```



Incompatible blocks or syntax raise `pdmat:InvalidConcatenation`; unsupported dimensions raise `pdmat:UnsupportedCatDimension`; invalid block-diagonal inputs raise `pdmat:InvalidBlkdiag`. Function-only operands raise `pdmat:FunctionOnlyAlgebra`.

4.7.8 Repetition

`repmat` tiles the matrix payload. It accepts a positive scalar `m`, interpreted as `[m m]`, two positive counts, or a two-element count vector. Grid and Bernstein degree remain unchanged.

Syntax

```
R = repmat(A, m)
R = repmat(A, m, n)
R = repmat(A, [m n])
```

Examples and output

```
v = pdmat({[0 1]}, {[1 2], [3 4]}, Degree=1);
R = repmat(v, [2 2]);
repeatedSize = size(R)
repeatedDegree = R.Degree
repeatedAtZero = R.evaluate(0)
```

```
repeatedSize =
     2     4
repeatedDegree =
     1
repeatedAtZero =
     1     2     1     2
     1     2     1     2
```

Invalid repetition shapes raise `pdmat:InvalidRepmat`.

4.8 Index and Assign `pdmat` Entries

Parenthesis indexing selects a matrix block from every coefficient. Assignment writes the selected block into every coefficient after compatible promotion and degree alignment. It does not select a physical cell; use `coeffs` for cell inspection.

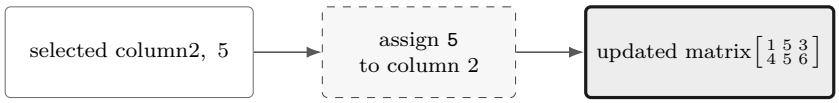
Syntax

```
B = A(rows, cols)
B = A(1:end, end)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end);
A(:, 2) = 5;
selectedSize = size(lastCol)
assignedSize = size(A)
selectedAtZero = lastCol.evaluate(0)
assignedAtZero = A.evaluate(0)
```

```
selectedSize =
     2     1
assignedSize =
     2     3
selectedAtZero =
     3
     6
assignedAtZero =
     1     5     3
     4     5     6
```



`subsref`, `subsasgn`, `end`, and `numArgumentsFromSubscript` implement this two-index MATLAB protocol. Invalid selectors raise `pdmat:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdmat:UnsupportedAssignment`; an incompatible right-hand side raises `pdmat:InvalidAssignment`. Slicing and assignment require coefficient evidence and otherwise raise `pdmat:FunctionOnlyAlgebra`.

4.9 Boundaries and Related APIs

1. Function-only objects created without `Degree` do not have Bernstein coefficient evidence for `bernsteinTable` or coefficient algebra.
2. Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.
3. Grid refinement is supported for matching parameter dimensions and matching grid bounds.

Related APIs. `pdmat`, `evaluate`, `bernsteinTable`, `plot`, `pdbase`.

Chapter 5

pdvar: Decision Matrix Functions

`pdvar` creates continuous cell-local Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. It does not choose solvers or call `optimize`. Comparison overloads is created and handled by `pdlmi` objects, of which the member function `toYalmip` later converts them to YALMIP constraints.

5.1 API Map

Task	Entry
Construct decisions	`pdvar`
Inspect coefficients	`bernsteinTable`
Convert assigned coefficients	`value`
Differentiate rates	`rhodiff`
Affine/mixed algebra	`+`, `-`, `*` with known data
Matrix/index methods	structural methods, indexing and assignment
Build LMIs	`<=`, `>=`, `==`

5.2 Construct pdvar Decisions

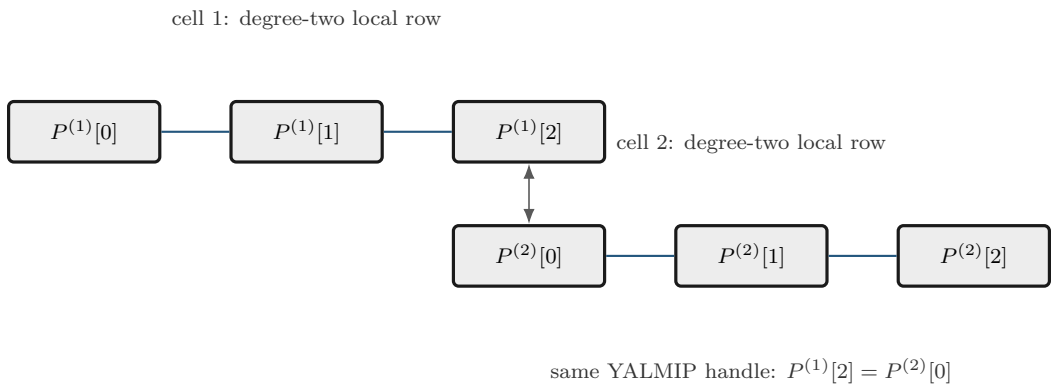
Syntax

```
P = pdvar(n, gridVectors)
P = pdvar(n, n, gridVectors)
P = pdvar(n, m, gridVectors)
P = pdvar(..., "full")
P = pdvar(..., "symmetric")
P = pdvar(..., Degree=0)
P = pdvar(..., Degree=1)
P = pdvar(..., Degree=d)
P = pdvar(..., RateBounds=rb)
```

Arguments

- `pdvar(n, gridVectors)` creates an $n \times n$ square variable using scalar-size shorthand; for example, `pdvar(2, {[0 1]})`. Square variables default to symmetric YALMIP coefficients.
- `pdvar(n, n, gridVectors)` is the explicit square-size form; for example, `pdvar(2, 2, {[0 1]})`. It has the same default symmetric structure as the shorthand.

- `pdvar(n, m, gridVectors)` creates an $n \times m$ variable. Rectangular variables default to full YALMIP coefficients; for example, `pdvar(2, 3, {[0 1]})`.
- `"full"` and `"symmetric"` explicitly choose the YALMIP coefficient structure. `"symmetric"` requires a square size.
- `Degree=1` is the default continuous Bernstein decision variable. Every finite nonnegative integer `Degree=d` is supported. `Degree=0` creates one parameter-independent decision matrix shared across the grid; `d >= 1` creates continuous piecewise Bernstein data whose neighboring cells share complete control faces.
- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter; use a two-row matrix for two parameters, as shown in the rate-metadata example below.



For positive degree, neighboring physical cells reuse complete boundary-face YALMIP handles; no extra continuity LMI is generated.

Examples and output

The square shorthand creates a 2×2 continuous decision object. The default degree is one and the object carries no rate metadata.

```
yalmip('clear')
Ps = pdvar(2, {[0 1]});
P= Ps.coeffs([1]);
P{1}
symmetricCoefficient = ishermitian(P{1})
variableClass = class(Ps)
matrixSize = size(Ps)
degree = Ps.Degree
rateDependent = Ps.HasRateDependence
```

```
Linear matrix variable 2x2 (symmetric, real, 3 variables)
Coefficient range: 1 to 1

symmetricCoefficient =
 logical
  1
```

```
variableClass =
    'pdvar'

matrixSize =
     2     2

degree =
     1

rateDependent =
  logical
   0
```

Use two dimension arguments for a rectangular full variable.

```
yalmip('clear')
Pf = pdvar(2, 3, {[0 1]});
variableClass = class(Pf)
matrixSize = size(Pf)
degree = Pf.Degree
```

```
variableClass =
    'pdvar'

matrixSize =
     2     3

degree =
     1
```

The structure flag makes the coefficient structure explicit. This is useful when a square variable should use full, not symmetric, coefficients.

```
yalmip('clear')
Pfull = pdvar(2, 2, {[0 1]}, "full");
P = Pfull.coeffs([1]);
P{1}
fullCoefficient = ishermitian(P{1})
variableClass = class(Pfull)
matrixSize = Pfull.MatrixSize
degree = Pfull.Degree
```

```
Linear matrix variable 2x2 (full, real, 4 variables)
Coefficient range: 1 to 1

fullCoefficient =
 logical
  0

variableClass =
    'pdvar'

matrixSize =
     2     2

degree =
     1
```

Use `Degree=0` for a parameter-independent decision variable stored with grid metadata.

```
yalmip('clear')
P0 = pdvar(1, [0 1 2], Degree=0);
variableClass = class(P0)
degree = P0.Degree
cellCount = P0.ncell()
```

```
variableClass =
    'pdvar'

degree =
     0

cellCount =
     2
```

Rate bounds are metadata for later `rhodiff(P)` calls. `pdlmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

```
yalmip('clear')
Pr = pdvar(1, {[0 1], [10 20]}, RateBounds=[−1 2; −3 4]);
variableClass = class(Pr)
rateDependent = Pr.HasRateDependence
rateBounds = Pr.RateBounds
```

```
variableClass =
    'pdvar'

rateDependent =
 logical
  1

rateBounds =
     -1     2
     -3     4
```

Remarks

- Missing or misplaced dimensions and grids raise `pdvar:InvalidInput`; invalid sizes and degrees raise `pdvar:InvalidMatrixSize` or `pdvar:InvalidDegree`.
- Option failures use `pdvar:InvalidOptions`, `pdvar:UnknownOption`, or `pdvar:UnsupportedOption`.
- A symmetric nonsquare request raises `pdvar:InvalidStructure`; grid failures use `pdvar:InvalidGrid` or `pdvar:InvalidGridVector`.
- `RateBounds` must be a finite ℓ-by-2 matrix with each lower bound no greater than its upper bound; violations raise `pdbase:InvalidRateBounds`.

5.3 Inspect Object Properties

`pdvar` declares no additional class-owned properties. It exposes the read-only inherited `pdbase` properties listed in section 3.3; their private set access prevents user assignment. In particular, decision coefficients are inspected through `P.LocalValues`, while `P.Degree`, `P.IsContinuous`, and `P.RateBounds` describe their representation and rate metadata.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=2, RateBounds=[−1 1]);
degree = P.Degree
isContinuous = P.IsContinuous
rateBounds = P.RateBounds
firstCellCoefficients = P.coeffs(1)
```

```
degree =
     2

isContinuous =
 logical
  1

rateBounds =
     -1     1

firstCellCoefficients =
1x3 cell array
    {[2x2 sdpvar]}    {[2x2 sdpvar]}    {[2x2 sdpvar]}
```

Use public algebra to create a modified value; direct assignments such as `P.Degree = 1` are not supported.

5.4 `rhodiff`

Syntax

```
D = rhodiff(P)
D = rhodiff(P, rb)
```

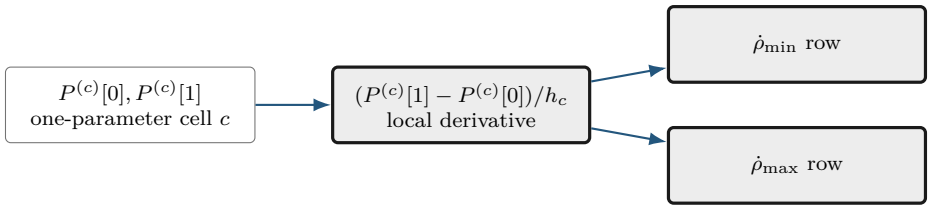
Arguments

- `P` is a `pdvar` without existing rate-vertex rows.
- `rb` is a finite ℓ-by-2 numeric matrix of lower and upper parameter-rate bounds, with each lower bound no greater than its upper bound. It must equal `P.RateBounds` when the object already carries nonempty bounds.
- Without `rb`, `rhodiff(P)` requires nonempty `P.RateBounds`.
- `D` is a discontinuous `pdvar` with one coefficient row per rate vertex. In one parameter a degree-m derivative has degree $m-1$, while a degree-zero derivative remains zero; multivariate partials are elevated into the implementation's common tensor degree.

For a one-parameter degree-one coefficient pair $P^{(c)}[0], P^{(c)}[1]$ on the physical cell $c = [\rho_c, \rho_{c+1}]$, define the positive cell length explicitly by $h_c := \rho_{c+1} - \rho_c$. The rate-dependent derivative row is then

$$\dot{P}(\rho, \dot{\rho}) = \dot{\rho}\, \frac{P^{(c)}[1] - P^{(c)}[0]}{h_c}.$$

This is the one-dimensional special case: the two rows correspond to the lower and upper allowed scheduling rates.



Differentiation changes local coefficients; rate substitution creates rows. Neither operation adds physical cells.

For ℓ parameters, work cell by cell with $h_s^{(c)} := \rho_s^{(c_s+1)} - \rho_s^{(c_s)} > 0$ in direction s of the physical hypercube $c = (c_1, \ldots, c_\ell)$. Thus nonuniform grids use a distinct step length in every direction and cell:

$$\dot{P}^{(c)}(\rho, \dot{\rho}) = \sum_{s=1}^{\ell} \dot{\rho}_s\, \frac{\partial P^{(c)}}{\partial \rho_s}(\rho).$$

`RateBounds` fixes a lower/upper box for $\dot{\boldsymbol{\rho}}$, so `rhodiff` evaluates this affine expression at its 2^ℓ vertices and stores one row per vertex. Supply those bounds explicitly as `rhodiff(P, rb)`, or initialize `P` with `RateBounds=rb`; if both are present they must match. Without bounds there is no finite vertex set to store, so `rhodiff(P)` raises `pdvar:MissingRateBounds`. Section A.9.1 gives the exact hypercube-local row and coefficient layout.

Remarks

Unsupported call arity or differentiating an expression that already contains rate-vertex rows raises `pdvar:InvalidDiff`. Explicit malformed rate bounds raise `pdvar:InvalidRateBounds`. Calling `rhodiff(P)` without stored bounds raises `pdvar:MissingRateBounds`; explicit bounds that disagree with `P.RateBounds` raise `pdvar:RateBoundsMismatch`.

Examples and output

When rate bounds live on the object, `rhodiff(P)` uses them directly.

```
yalmip('clear')
P = pdvar(2, {[0 1 2]}, "symmetric", RateBounds=[−1 1]);
D = rhodiff(P);
derivativeClass = class(D)
derivativeDegree = D.Degree
isContinuous = D.IsContinuous
coefficientTableSize = size(D.coeffs(1))
rateBounds = D.RateBounds
```

```
derivativeClass =
    'pdvar'
derivativeDegree =
     0
isContinuous =
  logical
   0
coefficientTableSize =
     2     1
rateBounds =
     -1     1
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

```
yalmip('clear')
Q = pdvar(1, {[0 1], [10 20]}, RateBounds=[−1 1; −2 2]);
E = rhodiff(Q);
% Rows 1:4 use [−1 −2], [−1 2], [1 −2], and [1 2], respectively.
derivativeClass = class(E)
derivativeDegree = E.Degree
coefficientTableSize = size(E.coeffs([1 1]))
rateVertexCount = coefficientTableSize(1)
coefficientsPerRateVertex = coefficientTableSize(2)
rateBounds = E.RateBounds
```

```
derivativeClass =
    'pdvar'
derivativeDegree =
     1
coefficientTableSize =
     4     4
rateVertexCount =
     4
coefficientsPerRateVertex =
     4
rateBounds =
     -1     1
     -2     2
```

5.5 bernsteinTable

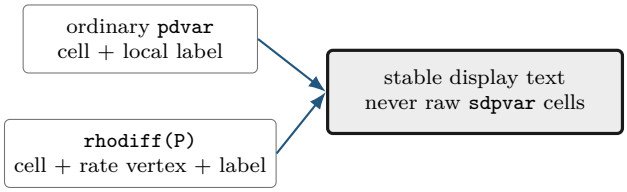
Syntax

```
T = bernsteinTable(P)
T = bernsteinTable(P, cellSubs)
T = bernsteinTable(P, "oneLine")
T = bernsteinTable(P, cellSubs, "oneLine")
```

Arguments

- `P` is a `pdvar` object.

- `cellSubs` selects one physical cell; omit it to list every cell.
- `"oneLine"` is the only supported text option. It returns one compact Bernstein expression per selected cell.
- `T` is a table. Ordinary rows contain the local label, basis, physical-node flag, and symbolic or numeric value. A rate-dependent derivative also includes `RateVertexIndex` and `RateVertex`.



Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]});
bernsteinTable(P, "oneLine")
```

```
    CellSubscript                    Expression
    -------------    ---------------------------------------------

        {[1]}        "(1-alpha)*internal(10) + alpha*internal(11)"
```

For `D = rhodiff(P)`, `bernsteinTable(D, cellSubs, "oneLine")` emits one row for every selected physical cell and rate-box vertex, rather than one row per local Bernstein coefficient. `RateVertexIndex` and `RateVertex` identify the lower/upper rate tuple; their tensor order is defined in Section A.9.1.

```
yalmip('clear')
P = pdvar(1, {[0 1], [0 1]}, RateBounds=[-1 1; -2 2]);
D = rhodiff(P);
T = bernsteinTable(D, [1 1], "oneLine");
vertexIndices = T.RateVertexIndex
rateVertices = vertcat(T.RateVertex{:})
expressionCount = height(T)
```

```
vertexIndices =
     1
     2
     3
     4

rateVertices =
     -1    -2
     -1     2
     1     -2
     1      2

expressionCount =
     4
```

Remarks

`bernsteinTable` is an inspection method, not a solver call. Text options other than `"oneLine"`, repeated selectors, invalid physical-cell selectors, or inconsistent rate-vertex rows raise `pdvar:InvalidBernsteinTableInput`.

5.6 Build Affine Algebra with Known Data

A `pdvar` result must remain affine in the underlying YALMIP decisions. The following blocks separate affine sums from products so the supported boundary is visible beside each example.

5.6.1 Signs, Addition, And Subtraction

Use this family to combine compatible decision expressions, coefficient-backed known data, finite real numeric data, and affine `sdpvar` matrices. Operands with the same physical bounds are aligned on a common grid and degree. The corresponding overload names are `plus`, `minus`, `uplus`, and `uminus`.

Syntax

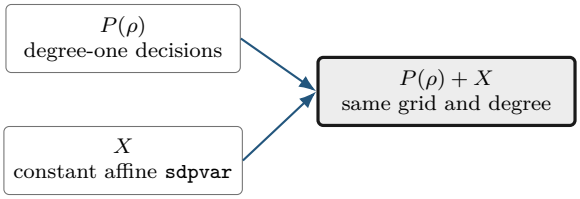
```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P + X
R = X + P
R = P - Q
R = P - A
R = A - P
R = P - M
R = M - P
R = +P
R = -P
```

Here `A` is coefficient-backed `pdmat`, `M` is numeric, and `X` is affine `sdpvar`. A numeric scalar broadcasts to the payload size; an affine `sdpvar` is promoted as parameter-independent data.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
X = sdpvar(2, 2, 'full');
S = P + X;
D = eye(2) - P;
N = -P;
sumClass = class(S)
sumSize = size(S)
sumDegree = S.Degree
sumContainsDecision = S.ContainsDecision
differenceClass = class(D)
negativeClass = class(N)
```

```
sumClass =
    'pdvar'
sumSize =
     2     2
sumDegree =
     1
sumContainsDecision =
  logical
   1
differenceClass =
    'pdvar'
negativeClass =
    'pdvar'
```



A nonlinear symbolic operand such as `x*x` is rejected. Rate-vertex rows from `rhodiff` may be added to compatible ordinary expressions; the ordinary rows are broadcast over the active rate vertices.

5.6.2 Multiplication By Known Or Numeric Data

The `mtimes` overload supports multiplication only when decision dependence occurs on at most one side. A coefficient-backed `pdmat` or finite real numeric matrix may multiply a `pdvar` from either side. A scalar `pdvar` may scale a known matrix. Two decision-bearing factors, including `P*Q` and `P*x` for a decision `sdpvar x`, are nonlinear and rejected.

Syntax

```
R = A * P
R = P * A
R = M * P
R = P * M
```

The first example shows numeric left and right matrix products without exposing incidental YALMIP variable identifiers.

Examples and output

```
yalmip('clear')
P = pdvar(2, 1, {[0 1]}, "full");
L = [1 2] * P;
R = P * [4 5];
S = 3 * P;
leftSize = size(L)
rightSize = size(R)
scaledSize = size(S)
degrees = [L.Degree R.Degree S.Degree]
```

```
leftSize =
     1     1
rightSize =
     2     2
scaledSize =
     2     1
degrees =
     1     1     1
```

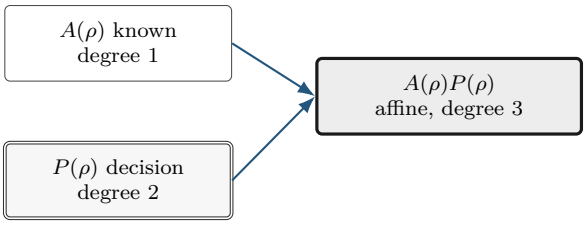
A known parameter-dependent factor adds its Bernstein degree through ordered coefficient convolution. On one cell, a quadratic decision and affine known factor produce

$$(PA)[k] = \sum_{i+j=k} \frac{\binom{2}{i}\binom{1}{j}}{\binom{3}{k}} P[i]A[j], \qquad k = 0, 1, 2, 3.$$

The same backend rule is documented at `bernProd` in section 3.6.4 and derived step by step in Appendix A.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=2);
A = pdmat({[0 1]}, {eye(2), 2*eye(2)}, Degree=1);
L = A * P;
R = P * A;
knownLeftClass = class(L)
knownLeftDegree = L.Degree
knownRightClass = class(R)
knownRightDegree = R.Degree
coefficientsPerCell = numel(L.coeffs(1))
```

```
knownLeftClass =
    'pdvar'
knownLeftDegree =
     3
knownRightClass =
    'pdvar'
knownRightDegree =
     3
coefficientsPerCell =
     4
```



A derivative object may be multiplied by numeric data or a matching-grid known object while preserving every rate row. Products with rate dependence on both sides, products of a derivative with an ordinary decision expression, and products involving metadata-only rate dependence are rejected.

Two decision-bearing factors are also rejected: their product would be quadratic in YALMIP decision coefficients. The supported boundary is numeric or known `pdmat` data multiplying one affine `pdvar` expression on either side, with the written matrix order preserved.

Remarks

Incompatible affine operands raise `pdvar:InvalidAddition` or `pdvar:InvalidSubtraction`. Bad dimensions, two decision-bearing factors, nonlinear symbolic data, or unsupported rate combinations raise `pdvar:InvalidMultiplication`. Incompatible parameter bounds raise `pdvar:MixedGrid`; a function-only `pdmat` operand raises `pdvar:FunctionOnlyAlgebra`.

5.7 Reshape and Transform Decision Matrices

Every method below transforms each local affine YALMIP payload while preserving the physical grid, degree, and compatible rate rows. The examples report stable classes, dimensions, and metadata rather than YALMIP's internal variable numbers.

5.7.1 Shape, Equality, And MATLAB Protocols

Shape queries describe the matrix payload. `isequal` is an exact identity check: grid, degree, matrix size, metadata, and local symbolic payloads must already match; it does not merge or elevate operands.

Syntax

```
sz = size(P)
d = size(P, dim)
n = height(P)
n = width(P)
n = length(P)
n = numel(P)
n = ndims(P)
tf = isequal(P, Q, ...)
idx = end(P, k, n)
n = numArgumentsFromSubscript(P, S, ctx)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
matrixSize = size(P)
rowCount = height(P)
columnCount = width(P)
longestDimension = length(P)
elementCount = numel(P)
dimensionCount = ndims(P)
equalsUnaryPlus = isequal(P,+P)
```

```
matrixSize =
     2     3
rowCount =
     2
columnCount =
     3
longestDimension =
     3
elementCount =
     6
dimensionCount =
     2
equalsUnaryPlus =
  logical
   1
```

`end`, `subsref`, `subsasgn`, and `numArgumentsFromSubscript` support the two-index and dot-access forms shown in section 5.8; they do not expose physical-cell storage.

5.7.2 Transpose And Conjugate Transpose

Both forms transpose every local affine payload without breaking the YALMIP coefficient graph.

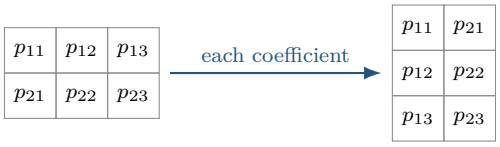
Syntax

```
T = transpose(P)
H = ctranspose(P)
T = P.'
H = P'
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
T = P.';
H = P';
transposeSize = size(T)
conjugateTransposeSize = size(H)
transposeClass = class(T)
conjugateTransposeClass = class(H)
```

```
transposeSize =
     3     2
conjugateTransposeSize =
     3     2
transposeClass =
    'pdvar'
conjugateTransposeClass =
    'pdvar'
```



5.7.3 Vectorization, Reshape, And Squeeze

`vec` uses MATLAB column-major order. `reshape` accepts two positive dimensions with at most one inferred `[]` dimension and preserves the element count. `squeeze` retains the two-dimensional package convention.

Syntax

```
V = vec(P)
R = reshape(P, [m n])
R = reshape(P, m, n)
S = squeeze(P)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
V = vec(P);
R = reshape(P, [3 2]);
S = squeeze(P);
vectorSize = size(V)
reshapeSize = size(R)
squeezeSize = size(S)
```

```
vectorSize =
     6     1
reshapeSize =
     3     2
squeezeSize =
     2     3
```

Malformed or size-changing reshape requests raise `pdvar:InvalidReshape`.

5.7.4 Diagonals And Trace

`diag` extracts a diagonal or constructs a diagonal matrix from a vector; the selected offset may not be empty. `trace` returns a scalar decision expression.

Syntax

```
d = diag(P)
d = diag(P, k)
D = diag(v)
D = diag(v, k)
t = trace(P)
```

Examples and output

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
d = diag(P);
D = diag(d);
t = trace(P);
diagonalSize = size(d)
rebuiltSize = size(D)
traceSize = size(t)
```

```
diagonalSize =
     3     1
rebuiltSize =
     3     3
traceSize =
     1     1
```

Invalid offsets or empty selections raise `pdvar:InvalidDiag`.

5.7.5 Triangular Parts And Reductions

Triangular methods zero the complementary part of every coefficient. Reductions act on matrix dimensions; `"all"` is available for `sum` and `mean`.

Syntax

```
L = tril(P)
L = tril(P, k)
U = triu(P)
U = triu(P, k)
S = sum(P)
S = sum(P, dim)
S = sum(P, "all")
M = mean(P)
M = mean(P, dim)
M = mean(P, "all")
C = cumsum(P)
C = cumsum(P, dim)
```

First form the two triangular views.

Examples and output

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
L = tril(P);
U = triu(P);
lowerSize = size(L)
upperSize = size(U)
degrees = [L.Degree U.Degree]
```

```
lowerSize =
     3     3
upperSize =
     3     3
degrees =
     1     1
```

Then reduce a rectangular expression.

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
rowSum = sum(P, 2);
allMean = mean(P, "all");
running = cumsum(P, 2);
rowSumSize = size(rowSum)
allMeanSize = size(allMean)
runningSize = size(running)
```

```
rowSumSize =
     2     1
allMeanSize =
     1     1
runningSize =
     2     3
```

Invalid triangular offsets raise `pdvar:InvalidTriangularPart`. Invalid reduction calls raise `pdvar:`

`InvalidSum`, `pdvar:InvalidMean`, or `pdvar:InvalidCumsum`.

5.7.6 Reordering And Rotation

These methods rearrange matrix entries inside every symbolic coefficient; they do not reverse the parameter grid.

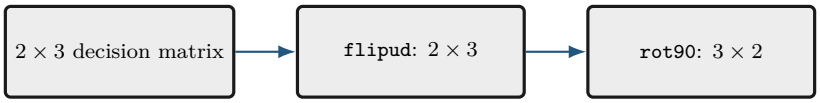
Syntax

```
Q = flip(P)
Q = flip(P, dim)
Q = flipud(P)
Q = fliplr(P)
Q = rot90(P)
Q = rot90(P, k)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
F = flipud(P);
R = rot90(P);
flippedSize = size(F)
rotatedSize = size(R)
flippedClass = class(F)
rotatedClass = class(R)
```

```
flippedSize =
     2     3
rotatedSize =
     3     2
flippedClass =
    'pdvar'
rotatedClass =
    'pdvar'
```



Invalid calls raise `pdvar:InvalidFlip` or `pdvar:InvalidRot90`.

5.7.7 Concatenation And Block Assembly

Assembly accepts compatible `pdvar`, coefficient-backed `pdmat`, affine `sdpvar`, and finite real numeric blocks. It aligns grids and degrees while preserving an affine expression. `cat` supports only matrix dimensions 1 and 2.

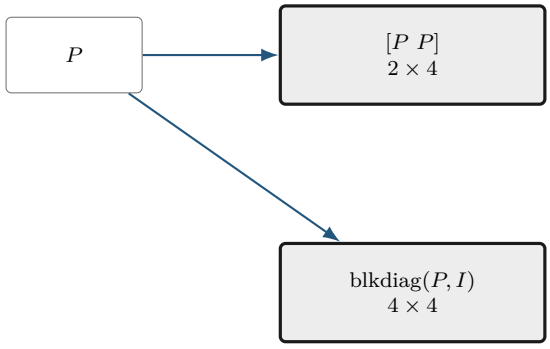
Syntax

```
H = horzcat(P, Q, ...)
V = vertcat(P, Q, ...)
H = [P, Q]
V = [P; Q]
C = cat(dim, P, Q, ...)
D = blkdiag(P, Q, ...)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
H = [P, P];
V = [P; P];
D = blkdiag(P, eye(2));
horizontalSize = size(H)
verticalSize = size(V)
blockDiagonalSize = size(D)
```

```
horizontalSize =
     2     4
verticalSize =
     4     2
blockDiagonalSize =
     4     4
```



Incompatible blocks or rate metadata raise `pdvar:InvalidConcatenation`; unsupported dimensions raise
`pdvar:UnsupportedCatDimension`; invalid block-diagonal inputs raise `pdvar:InvalidBlkdiag`. Incompatible bounds raise `pdvar:MixedGrid`; function-only known blocks raise `pdvar:FunctionOnlyAlgebra`.

5.7.8 Repetition

`repmat` tiles the matrix payload while retaining the parameter metadata. It accepts a positive scalar, two positive counts, or a two-element count vector.

Syntax

```
Q = repmat(P, m)
Q = repmat(P, m, n)
Q = repmat(P, [m n])
```

Examples and output

```
yalmip('clear')
P = pdvar(1, 2, {[0 1]}, "full");
Q = repmat(P, [2 2]);
repeatedSize = size(Q)
repeatedDegree = Q.Degree
containsDecision = Q.ContainsDecision
```

```
repeatedSize =
     2     4
repeatedDegree =
     1
containsDecision =
  logical
   1
```

Invalid repetition shapes raise `pdvar:InvalidRepmat`.

5.8 Index and Assign `pdvar` Entries

Parenthesis indexing selects a matrix block from every symbolic coefficient. Assignment writes a compatible numeric, `pdvar`, coefficient-backed `pdmat`, or affine `sdpvar` block into that position. It does not select one physical cell.

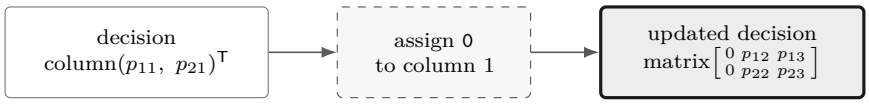
Syntax

```
Q = P(rows, cols)
Q = P(1:end, end)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
lastCol = P(:, end);
P(:, 1) = 0;
selectedSize = size(lastCol)
assignedSize = size(P)
assignedClass = class(P)
assignedDegree = P.Degree
containsDecision = P.ContainsDecision
```

```
selectedSize =
     2     1
assignedSize =
     2     3
assignedClass =
    'pdvar'
assignedDegree =
     1
containsDecision =
  logical
   1
```



Invalid selectors raise `pdvar:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdvar:UnsupportedAssignment`; incompatible or nonlinear right-hand sides raise `pdvar:InvalidAssignment`.

5.9 `value`

Syntax

```
A = value(P)
rows = value(rhodiff(P))
```

`value` converts assigned `pdvar` coefficients into known, coefficient-backed `pdmat` data. An ordinary expression returns one `pdmat`, preserving its grid, matrix size, degree, local coefficient order, and continuity metadata. A rate-vertex expression created by `rhodiff` returns a 1-by-2^{ℓ} cell array of `pdmat` objects in `helper.combRows(RateBounds)` lower/upper vertex order. The returned `pdmat` objects do not carry rate metadata. Every symbolic coefficient must have an assigned finite real YALMIP value; otherwise the method raises `pdvar:UnassignedValue`.

Examples and output

```
yalmip('clear')
P = pdvar(1, [0 1], Degree=2);
c = P.coeffs(1);
assign([c{:}], [1 2 4]);
A = value(P);
assignedCoefficients = A.coeffs(1)
```

```
assignedCoefficients =
    {[1]}    {[2]}    {[4]}
```

5.10 Comparisons To `pdlmi`

The `le`, `ge`, and `eq` overloads implement `<=`, `>=`, and `==`. Inequalities select either semidefinite or entry-wise meaning once for the complete original residual; equality is always direct coefficient equality.

Syntax

```
C = P <= 0
C = P >= 0
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

Arguments

- `rhs` may be compatible numeric, coefficient-backed `pdmat`, affine `sdpvar`, or `pdvar` data. Ordinary operands use the same grid refinement, degree alignment, and promotion as subtraction.
- For `<=` and `>=`, every coefficient in every physical cell and rate row is scanned before assembly. The complete relation is semidefinite only if every original coefficient is square Hermitian. Numeric Hermitian testing uses the inclusive rule $\|A - A^{\mathsf{T}}\|_\infty \leq 10^{-10}$; symbolic coefficients use YALMIP's `ishermitian`. If any coefficient fails, the complete inequality is entry-wise and warning `pdlmi:ElementwiseInequality` is issued once. Modes are never mixed coefficient by coefficient or entry by entry.
- `==` accepts rectangular and nonsymmetric residuals without that warning. It equates corresponding coefficient entries directly. Ordinary rows compare with ordinary rows; derivative rows compare only with compatible derivative rows.
- `C` is a `pdlmi` wrapper. A comparison assembles constraints but does not solve an optimization problem.

Examples and output

Comparison creates a `pdlmi` wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

```
yalmip('clear')
P = pdvar(2, {[0 0.5 1]}, "symmetric");
Cneg = P <= 0;
negativeWrapperClass = class(Cneg)
negativeConstraintCount = numel(Cneg.Constraints)
Cpos = P >= 0;
positiveWrapperClass = class(Cpos)
positiveConstraintCount = numel(Cpos.Constraints)
```

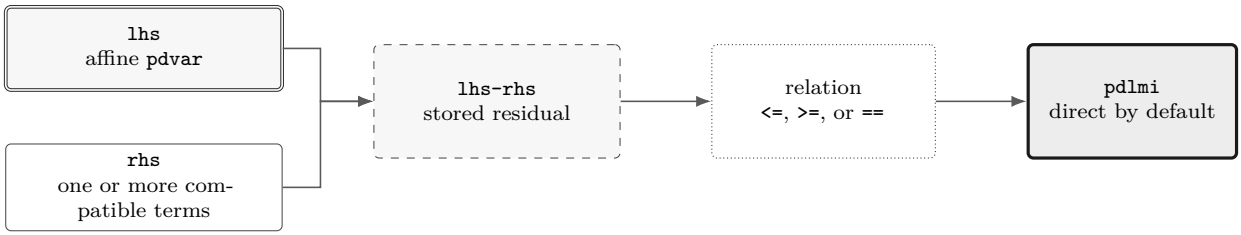
```
negativeWrapperClass =
    'pdlmi'
negativeConstraintCount =
     4
positiveWrapperClass =
    'pdlmi'
positiveConstraintCount =
     4
```

A rectangular residual selects entry-wise inequality globally. Equality is entry-wise by definition and is warning-free.

```
yalmip('clear')
V = pdvar(3, 2, {[0 1]}, "full");
warnId = "pdlmi:ElementwiseInequality";
warnState = warning("off", warnId);
restoreWarning = onCleanup(@() warning(warnState));
Centry = V >= 0;
Ceq = V == sdpvar(3, 2, 'full');
entrywiseConstraintGroups = numel(Centry.Constraints)
equalityConstraintGroups = numel(Ceq.Constraints)
identity = V == V;
identityConstraints = identity.toYalmip()
clear restoreWarning
```

```
entrywiseConstraintGroups =
     2
equalityConstraintGroups =
     2
identityConstraints =
     []
```

Each group above corresponds to one local coefficient and contains six scalar entries in MATLAB column-major order. Under Pólya, Putinar, or Full Box, each entry of an entry-wise inequality receives an independent scalar certificate. Equality is direct-only: constructor certificate options and `applyPolya`, `applyPutinar`, or `applyFullBoxPreorder` raise `pdlmi:UnsupportedEqualityCertificate` before validating an order. Mixed ordinary/derivative row kinds raise `pdvar:InvalidEqualityRows`. Use overloaded `==` to build a modeling equality; use `isequal` only for MATLAB structural comparison.



5.11 Boundaries and Related APIs

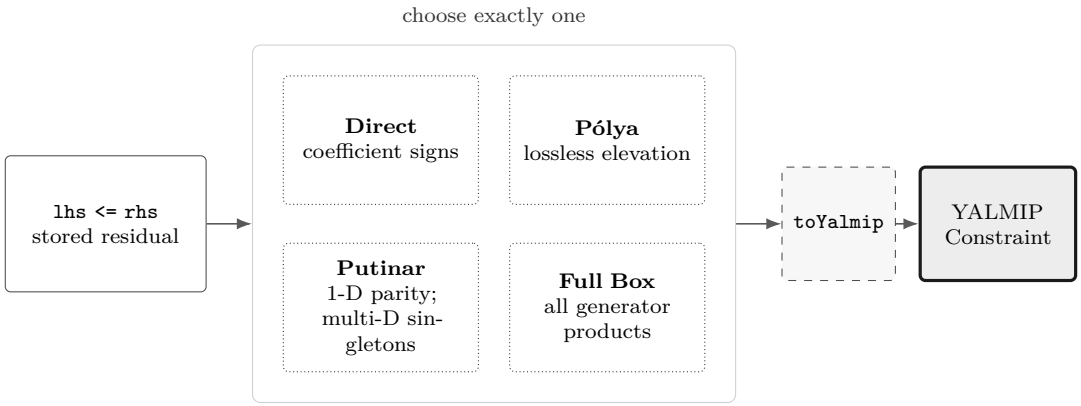
1. Constructor-created `pdvar` objects support every finite nonnegative integer degree. Higher degree increases the number of Bernstein control matrices per cell.
2. Products may contain decision variables on at most one side; nonlinear decision products such as `P * Q` are outside this slice.
3. Function-only `pdmat` operands cannot enter `pdvar` algebra unless they were constructed with explicit Bernstein evidence.
4. `rhodiff` of an existing rate-vertex derivative expression is not part of the current public workflow.

Related APIs. `pdvar`, `rhodiff`, `bernsteinTable`, `mtimes`, `pdlmi`, `pdmat`.

Chapter 6

`pdlmi`: Finite Certificate Assembly

`pdlmi` stores YALMIP constraints assembled from `pdvar` expressions. Square Hermitian inequality families use semidefinite constraints; other inequalities use entry-wise scalar constraints; equality uses direct coefficient equations. Direct assembly is the default, while cell-local Pólya elevation, a dimension-aware Putinar selector, and the full box Bernstein–Gram preordering are explicit inequality alternatives. It is the package boundary between Bernstein objects and ordinary YALMIP solver calls.



The four certificate paths are alternatives. Selecting Pólya, Putinar, or full-box rebuilding replaces, rather than compounds, the previous selection.

6.1 API Map

Task	Entry
Construct wrapper	`pdlmi`, `<=/>=/==`
Count constraints	direct coefficient constraints
Select Pólya elevation	`UsePolya`, `PolyaDegree`, and `applyPolya`
Select Putinar certificate	`UsePutinar`, `PutinarOrder`, and `applyPutinar`
Select full box preordering	`UseFullBoxPreorder`, `FullBoxOrder`, and `applyFullBoxPreorder`
Export to YALMIP	`toYalmip`
Solve	solver-facing use, Examples

6.2 Inspect Certificate State

All `pdlmi` properties have private set access and are readable with dot notation. `Constraints` contains the assembled YALMIP entries; `Residual` and `Relation` retain the original comparison for rebuilding. The remaining properties record the selected certificate state.

Property	Meaning
`Constraints`	Stored finite YALMIP constraint entries.
`Residual`, `Relation`	Original `pdvar` residual and one of `"<="`, `">="`, or `"=="`.
`UsePolya`, `PolyaDegree`	Pólya selector and nonnegative degree increment.
`UsePutinar`, `PutinarOrder`	Putinar selector and absolute Gram order.
`UseFullBoxPreorder`, `FullBoxOrder`	Full-box selector and absolute Gram order.

For example, `C.Relation`, `C.Residual.Degree`, and `C.UsePutinar` inspect a wrapper. Apply methods return a rebuilt value; assigning these properties is unsupported.

6.3 Construct `pdlmi` Comparisons

Syntax

```
C = pdlmi(expr, relation)
C = pdlmi(expr, "==")
C = pdlmi(expr, relation, "UsePolya")
C = pdlmi(expr, relation, UsePolya=true, PolyaDegree=d)
C = pdlmi(expr, relation, "UsePolya", "PolyaDegree", d)
C = pdlmi(expr, relation, "UsePutinar")
C = pdlmi(expr, relation, UsePutinar=true, PutinarOrder=r)
C = pdlmi(expr, relation, PutinarOrder=r)
C = pdlmi(expr, relation, "UseFullBoxPreorder")
C = pdlmi(expr, relation, UseFullBoxPreorder=true)
C = pdlmi(expr, relation, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=true, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=false, FullBoxOrder=r)
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

Arguments

- `expr` is a `pdvar` residual; `relation` is exactly `"<="`, `">="`, or `"=="`. Inequalities are classified globally from the original coefficient family. Equality is entry-wise direct coefficient assembly and accepts rectangular or nonsymmetric residuals.
- `UsePolya` is a logical scalar option with default `false`. Its bare form, `"UsePolya"`, and `UsePolya=true` without a degree select increment one.

- `PolyaDegree=d` is a finite nonnegative integer scalar. It elevates the residual by `d` in every parameter direction before assembling constraints. Supplying it without `UsePolya` enables Pólya and issues `pdlmi:ImplicitUsePolya`; `UsePolya=false` with positive `d` raises `pdlmi:ConflictingPolyaOptions`.
- `UsePutinar` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select the dimension-appropriate Putinar certificate at the smallest admissible order.
- `PutinarOrder=r` is a finite nonnegative integer scalar and is an absolute Bernstein Gram order, not a degree increment. For residual degree m, its default and minimum are $\lfloor m/2 \rfloor$ in one parameter and $\lceil m/2 \rceil$ in two or more parameters. Supplying the order without `UsePutinar` enables Putinar assembly. Pairing `UsePutinar=false` with any explicit order, including zero, raises `pdlmi:ConflictingPutinarOptions`.
- `UseFullBoxPreorder` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select full-box assembly at the smallest admissible order.
- `FullBoxOrder=r` is a finite nonnegative integer scalar and is an absolute order, not a degree increment. Supplying it without `UseFullBoxPreorder` enables full-box assembly. When full-box assembly is enabled, the minimum is $\lfloor m/2 \rfloor$ for a one-parameter residual of degree m, and $\lceil m/2 \rceil$ for two or more parameters. Explicitly pairing `UseFullBoxPreorder=false` with `FullBoxOrder=r` suppresses that automatic selection: `C` remains direct and records `r` only as inspectable metadata; the order is not applied to assembly.
- Pólya, Putinar, and full-box selection are mutually exclusive and apply only to inequalities. Any recognized certificate option on an equality raises `pdlmi:UnsupportedEqualityCertificate`. Enabling more than one inequality family raises `pdlmi:ConflictingRelaxations`.
- `C` is direct coefficient-wise assembly unless one opt-in inequality mode is selected. Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and accept compatible numeric, `pdmat`, affine `sdpvar`, and `pdvar` right-hand sides.

The inspectable properties `Constraints`, `Residual`, and `Relation` expose the assembled finite constraints and the original comparison data used for rebuilding. `UsePolya`, `PolyaDegree`, `UsePutinar`, `PutinarOrder`, `UseFullBoxPreorder`, and `FullBoxOrder` report the selected mode and order metadata. Their set access is private. Comparison-created direct objects have all three selectors false and all three numeric properties zero. The full-box explicit-false combination described above is the exception: it retains `FullBoxOrder=r` while `UseFullBoxPreorder` remains false and the stored constraints remain direct. Putinar deliberately has no corresponding exception.

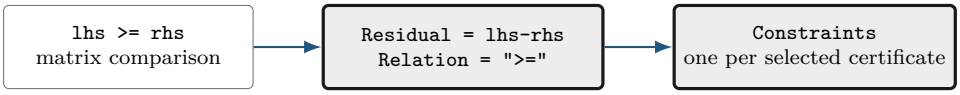
Examples and output

The direct constructor is equivalent to the comparison path when the relation and options are explicit.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = pdlmi(P, ">=", UsePolya=false, PolyaDegree=0);
wrapperClass = class(C)
constraintCount = numel(C.Constraints)
usePolya = C.UsePolya
```

```
polyaDegree = C.PolyaDegree
usePutinar = C.UsePutinar
putinarOrder = C.PutinarOrder
useFullBox = C.UseFullBoxPreorder
fullBoxOrder = C.FullBoxOrder
```

```
wrapperClass =
    'pdlmi'
constraintCount =
     2
usePolya =
  logical
   0
polyaDegree =
     0
usePutinar =
  logical
   0
putinarOrder =
     0
useFullBox =
  logical
   0
fullBoxOrder =
     0
```



Remarks

1. A non-`pdvar` residual raises `pdlmi:InvalidExpression`; an unsupported relation raises `pdlmi:InvalidRelation`.

2. Malformed, unknown, or duplicate options raise `pdlmi:InvalidOptions`, `pdlmi:UnknownOption`, or `pdlmi:DuplicateOption`; nonlogical selectors use their corresponding `InvalidUse...` identifier.

3. Malformed Pólya increments, Putinar orders, and full-box orders raise, respectively, `pdlmi:InvalidPolyaDegree`, `pdlmi:InvalidPutinarOrder`, and `pdlmi:InvalidFullBoxOrder`. An integer below an active Gram-family minimum raises `pdlmi:PutinarOrderTooLow` or `pdlmi:FullBoxOrderTooLow`.

4. Explicit false plus a Putinar order raises `pdlmi:ConflictingPutinarOptions`; explicit false plus a positive Pólya increment raises `pdlmi:ConflictingPolyaOptions`. The full-box explicit-false form remains direct and retains the supplied order as metadata.

5. An inequality with any nonsquare or non-Hermitian original coefficient is assembled entry-wise and emits `pdlmi:ElementwiseInequality`; it is not rejected. Numeric Hermitian testing accepts equality at the inclusive 10^{-10} infinity-norm tolerance.

6. Equality is direct-only. Every `apply...` method raises `pdlmi:UnsupportedEqualityCertificate` before validating its supplied increment or order.

6.4 Assemble Direct Coefficient Constraints

For each physical cell, local Bernstein coefficient, and rate-vertex row if present, `pdlmi` creates one stored YALMIP constraint entry. For an ordinary expression with N_c physical cells and N_b Bernstein coefficients per cell, the number of entries is $N_c N_b$; with N_v rate vertices it is $N_c N_b N_v$. A semidefinite-mode entry is one matrix constraint. An entry-wise-mode entry is the column-major vector of scalar matrix entries compared independently. An equality entry is the corresponding direct coefficient equation and may be rectangular.

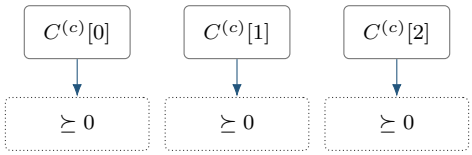
The idea is the Bernstein convex-hull property. After orienting the relation as a positive-semidefinite target

$$S_{\mathbf{c},v}(\boldsymbol{\alpha}) = \sum_{\mathbf{i}} B^m_{\mathbf{i}}(\boldsymbol{\alpha}) C^{(\mathbf{c},v)}[\mathbf{i}],$$

the semidefinite direct certificate requires $C^{(\mathbf{c},v)}[\mathbf{i}] \succeq 0$ for every physical cell $\mathbf{c}$, active rate vertex v, and local label $\mathbf{i}$. This is sufficient, not necessary.

For an entry-wise inequality, replace each matrix coefficient by `C(:)` and impose the oriented scalar sign on every component. The global classification is made before Pólya elevation or Gram assembly, so every coefficient and every certificate in one wrapper uses the same mode. Putinar and Full Box create one independent scalar Gram certificate for each column-major entry. Direct equality simply imposes `C(:) == 0`; it does not use a positivity certificate or an implicit tolerance.

The diagram specializes to one parameter, degree two, and one ordinary (non-rate) row; therefore c is a scalar one-based cell index and $i \in \{0, 1, 2\}$.



Syntax

```
C = pdlmi(expr, relation)
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

The constructor and comparison forms above create direct coefficient constraints when no inequality certificate is selected. For an inequality, each stored coefficient is tested in the oriented positive-semidefinite or entry-wise mode; for an equality, each coefficient entry is constrained directly to

zero. The number of stored constraints is therefore the number of physical cells times the number of local Bernstein coefficients, with an additional factor for active rate vertices.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
C = P >= 0;

constraintCount = numel(C.Constraints)
usePolya = C.UsePolya
usePutinar = C.UsePutinar
useFullBox = C.UseFullBoxPreorder
```

```
constraintCount =
     3
usePolya =
  logical
   0
usePutinar =
  logical
   0
useFullBox =
  logical
   0
```

Here the degree-two, one-parameter residual has three local Bernstein coefficients, so direct assembly stores three coefficient constraints. The wrapper remains direct because no certificate selector was enabled; a call such as `C.applyPolya(1)` returns a new wrapper with an elevated coefficient family and leaves `C` unchanged.

6.5 Select Pólya Assembly with `applyPolya`

With `UsePolya=true`, `pdlmi` first elevates the stored residual by `PolyaDegree` in every parameter direction, then constrains every elevated local coefficient and every active rate row. Thus, if the original degree is m, the count is $N_c(m+d+1)^\ell$ for ordinary expressions and $N_c(m+d+1)^\ell N_v$ for rate-vertex expressions. An increment of zero is valid and has the same coefficient count as direct assembly. For an entry-wise inequality, elevation preserves the global mode and the oriented sign is applied independently to every column-major scalar entry of each elevated coefficient. Equality wrappers reject this method.

Syntax

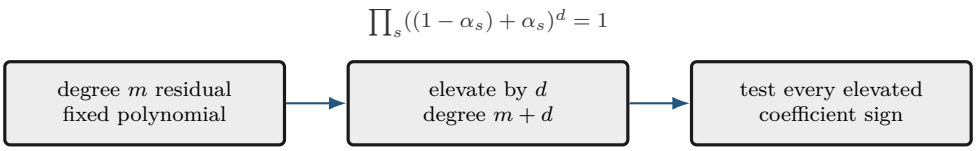
```
Cpolya = C.applyPolya()
Cpolya = C.applyPolya(d)
```

`applyPolya` is a value-class method: it returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from `C.Residual`. Selecting a new increment therefore replaces rather than compounds a previous selection. The no-argument form uses one; an invalid increment raises `pdlmi:InvalidPolyaDegree`.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=1);
direct = P >= 0;
polya = direct.applyPolya(2);
directDegree = direct.PolyaDegree
usePolya = polya.UsePolya
polyaDegree = polya.PolyaDegree
constraintCount = numel(polya.Constraints)
```

```
directDegree =
     0
usePolya =
  logical
   1
polyaDegree =
     2
constraintCount =
     4
```



Pólya degree is a uniform representation increment, not a new decision degree, a grid refinement, or a Gram order.

The mathematical background and the limits of fixed-increment coefficient positivity are developed in Chapter B.

6.6 Select Putinar Assembly with `applyPutinar`

For an entry-wise inequality, the constructions below are applied independently to each scalar matrix entry in MATLAB column-major order. Gram variables are not shared between entries. Equality wrappers reject Putinar selection.

The Putinar selector is dimension-aware. With one parameter it deliberately uses the same exact even/odd Markov–Lukács assembly as `applyFullBoxPreorder`. For selected absolute order r, the even form matches at degree $2r$,

$$F = \mathcal{Z}_r^{\mathsf{T}} Q_0 \mathcal{Z}_r + \alpha(1-\alpha)\mathcal{Z}_{r-1}^{\mathsf{T}} Q_1 \mathcal{Z}_{r-1},$$

where the second block is omitted at $r = 0$. The odd form matches at degree $2r+1$,

$$F = (1-\alpha)\mathcal{Z}_r^{\mathsf{T}} Q_L \mathcal{Z}_r + \alpha \mathcal{Z}_r^{\mathsf{T}} Q_U \mathcal{Z}_r.$$

All present Gram matrices are positive semidefinite. The one-parameter default and minimum are $r = \lfloor m/2 \rfloor$ for residual degree m.

For two or more parameters, the selector instead uses the fixed-order, box-specific singleton-generator Bernstein–Gram quadratic module. After normalizing the relation so that $F(\boldsymbol{\alpha})$ is positive semidefinite, it represents

$$F(\boldsymbol{\alpha}) = S_0(\boldsymbol{\alpha}) + \sum_{s=1}^{\ell} \alpha_s(1-\alpha_s)S_s(\boldsymbol{\alpha}).$$

Only singleton box generators appear in this multivariate branch: products such as $\alpha_1(1-\alpha_1)\alpha_2(1-\alpha_2)$ belong to the full-box preordering, not this module.

Syntax

```
Cputinar = C.applyPutinar()
Cputinar = C.applyPutinar(r)
```

Here `r` is an absolute Bernstein Gram order. The no-argument form uses the dimension-dependent minima above; an explicit order must satisfy the same applicable bound. In the multivariate branch, the unweighted block S_0 uses tensor basis degree $r\mathbf{1}$, while S_s uses degree $r\mathbf{1} - \boldsymbol{e}_s$. A nominal block with a negative component is omitted, which occurs for every multiplier at $r = 0$. The target is degree-elevated without changing its represented polynomial, and the multivariate Gram sum is matched exactly at tensor degree $2r$.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
putinar = direct.applyPutinar();
directUsesPutinar = direct.UsePutinar
directOrder = direct.PutinarOrder
directConstraintCount = numel(direct.Constraints)
selectedUsesPutinar = putinar.UsePutinar
selectedOrder = putinar.PutinarOrder
selectedConstraintCount = numel(putinar.Constraints)
```

```
directUsesPutinar =
  logical
   0
directOrder =
     0
directConstraintCount =
     3
selectedUsesPutinar =
  logical
   1
selectedOrder =
     1
selectedConstraintCount =
     5
```

For this one-parameter scalar degree-two cell, order one uses the exact even Markov–Lukács form: an unweighted Gram block of dimension two and one generator-weighted block of dimension one, followed by three exact coefficient identities. The result is identical to `direct.applyFullBoxPreorder()`.

Each physical cell and each active rate-vertex row receives independent copies of these PSD blocks and identities.

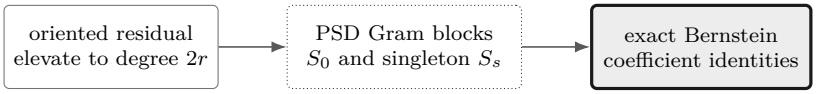
Remarks

For the multivariate backend, the smallest useful worked case is $\ell = 2$. Let $g_s = \alpha_s(1 - \alpha_s)$. At absolute order r, Putinar uses exactly the singleton-generator module

$$F = S_0 + g_1 S_1 + g_2 S_2,$$

where S_0 has Bernstein basis degree (r, r), S_1 has degree $(r - 1, r)$, and S_2 has degree $(r, r - 1)$. Each S_j is a PSD Gram form. MATLAB elevates the sign-normalized target and all present terms to tensor degree $(2r, 2r)$, then creates one exact coefficient match per local Bernstein label. These PSD blocks and equalities are assembled independently for every physical cell, active rate row, and scalar matrix entry; they are never shared across those axes.

The diagram below specializes to the even one-parameter example above.



The method changes the finite certificate, not the stored residual, decision degree, or physical grid.

`applyPutinar` is a value-class method: it returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from the original `Residual` and `Relation`. Reapplication replaces any earlier Pólya, Putinar, or full-box selection. It adds no implicit margin and makes no solver call.

Remarks

---

1. Malformed orders raise `pdlmi:InvalidPutinarOrder`; an integer below $\lfloor m/2 \rfloor$ in one parameter, or below $\lceil m/2 \rceil$ in two or more parameters, raises `pdlmi:PutinarOrderTooLow`.

2. A malformed selector raises `pdlmi:InvalidUsePutinar`; explicit false plus any supplied order raises `pdlmi:ConflictingPutinarOptions`.

3. Simultaneous relaxation families raise `pdlmi:ConflictingRelaxations`; expression, relation, and option validation in section 6.3 still applies. Global inequality classification is unchanged: the family is semidefinite only when every original coefficient across all cells and rate rows is square Hermitian. Otherwise the complete family uses independent entry-wise scalar Putinar certificates in MATLAB column-major order.

6.7 Select Full Box Bernstein–Gram Assembly with `applyFullBoxPreorder`

The full-box mode is a cell-local positive-semidefinite representation in the forward local coordinates $\alpha_s \in [0, 1]$. It is a box-specific Bernstein–Gram preordering, not a general Putinar certificate, general-domain SOS interface, or full-space SOS representation. It adds no implicit positivity margin. For an entry-wise inequality, every column-major scalar entry receives an independent copy of the selected Gram construction. Equality wrappers reject full-box selection.

Syntax

```
Cbox = C.applyFullBoxPreorder()
Cbox = C.applyFullBoxPreorder(r)
```

The no-argument form selects the smallest admissible absolute order. The explicit form selects `r`; it does not add `r` to the residual degree. Both forms are value-class selectors: they return a new `pdlmi`, leave `C` unchanged, and rebuild from `C.Residual`. Reapplication therefore replaces an earlier full-box order or a Pólya or Putinar selection rather than compounding it.

The idea is to represent the oriented positive target as a sum of PSD Gram forms multiplied by functions that are nonnegative on the local box:

$$S(\boldsymbol{\alpha}) = \sum_{J \subseteq \{1,\dots,\ell\}} \left(\prod_{s \in J} \alpha_s (1 - \alpha_s) \right) \mathcal{Z}_J(\boldsymbol{\alpha})^{\mathsf{T}} Q_J \mathcal{Z}_J(\boldsymbol{\alpha}), \qquad Q_J \succeq 0.$$

In one parameter the implementation uses the even/odd Markov–Lukács forms, with minimum absolute order $\lfloor m/2 \rfloor$ for residual degree m. For two or more parameters it uses every subset product of the ℓ box generators and minimum order $\lceil m/2 \rceil$. The selected order fixes the dense Bernstein Gram bases; it is not added to m.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
box = direct.applyFullBoxPreorder();
directConstraintCount = numel(direct.Constraints)
useFullBox = box.UseFullBoxPreorder
fullBoxOrder = box.FullBoxOrder
fullBoxConstraintCount = numel(box.Constraints)
```

```
directConstraintCount =
     3
useFullBox =
  logical
   1
fullBoxOrder =
     1
fullBoxConstraintCount =
     5
```

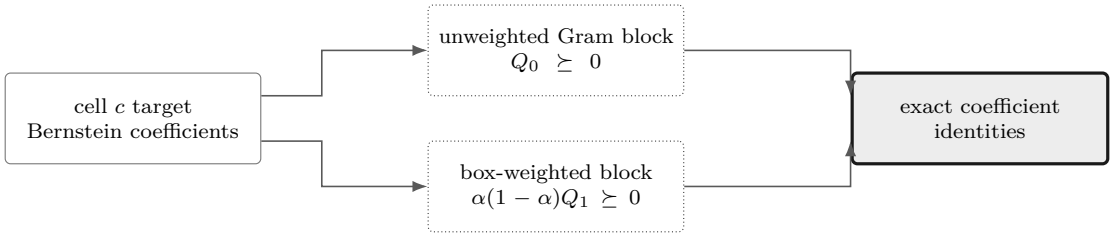
At this order the scalar degree-two cell has two PSD Gram constraints followed by three exact Bernstein coefficient identities.

Remarks

For $\ell = 2$, Full Box differs from Putinar by retaining the product generator. With $g_s = \alpha_s(1 - \alpha_s)$, its exact backend form is

$$F = S_0 + g_1 S_1 + g_2 S_2 + g_1 g_2 S_{12}.$$

The Bernstein basis degrees are respectively (r, r), $(r-1, r)$, $(r, r-1)$, and $(r-1, r-1)$; each present term is a PSD Gram block. All four terms are elevated and coefficient-matched at tensor degree $(2r, 2r)$. Thus the additional S_{12} block is precisely the two-generator distinction from the singleton Putinar module, not a change to the residual, grid, or decision degree. The implementation builds the PSD blocks and exact equalities independently for each cell, rate row, and scalar matrix entry.



Appendix B derives the one-dimensional parity forms, explains the multivariate subset masks, and distinguishes this implemented box preordering from the multivariate singleton-generator Putinar branch, full-space SOS, and general polynomial parsers. Assembly is independent for every physical cell and every active rate-vertex row; Gram variables are not shared across either boundary. For `>=`, the method represents the residual. For `<=`, it negates the target before forming the positive-semidefinite representation. Within each cell and row, the stored constraints are ordered as all PSD Gram blocks followed by exact equalities matching every Bernstein coefficient. Consequently, the number of stored constraints is not the direct coefficient count.

Remarks

---

1. Malformed orders raise `pdlmi:InvalidFullBoxOrder`; an admissible integer below the minimum raises `pdlmi:FullBoxOrderTooLow`. The constructor rejects malformed selectors with `pdlmi:InvalidUseFullBoxPreorder`.
2. Simultaneous Pólya, Putinar, or full-box selection raises `pdlmi:ConflictingRelaxations`; expression, relation, and option validation in section 6.3 still applies.
3. Global inequality classification is unchanged: the family is semidefinite only when every original coefficient across all cells and rate rows is square Hermitian. Otherwise the complete family uses independent entry-wise scalar Full Box certificates in MATLAB column-major order.

6.8 Export with `toYalmip`

Syntax

```
F = toYalmip(C)
F = C.toYalmip()
```

Arguments

`toYalmip` concatenates the stored constraint cells into a YALMIP constraint array for direct, Pólya, Putinar, and full-box objects. It has no package-specific options and does not add a solver wrapper, objective, margin, or diagnostic layer. The output is suitable for `optimize`, concatenation with

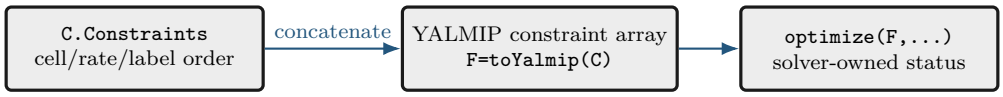
other YALMIP constraints, or ordinary YALMIP inspection.

Examples and output

The function-call and dot-call forms return equivalent YALMIP constraint arrays.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
wrapperClass = class(C)
storedConstraintCount = numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
firstIsConstraint = isa(F1, "lmi") || isa(F1, "constraint")
firstConstraintCount = length(F1)
secondIsConstraint = isa(F2, "lmi") || isa(F2, "constraint")
secondConstraintCount = length(F2)
```

```
wrapperClass =
    'pdlmi'
storedConstraintCount =
     2
firstIsConstraint =
  logical
   1
firstConstraintCount =
     2
secondIsConstraint =
  logical
   1
secondConstraintCount =
     2
```



After this boundary, the package is no longer assembling Bernstein data; YALMIP owns solver choice, diagnostics, and optimization status.

6.9 Boundaries and Related APIs

`pdvar`, `rhodiff`, `applyPolya`, `applyPutinar`, `applyFullBoxPreorder`, `toYalmip`, `optimize`.

6.9.1 Solve with YALMIP

`pdlmi` does not own solver settings or wrap `optimize`. After `toYalmip`, use ordinary YALMIP:

```
opts = sdpsettings("solver", solverName, "verbose", 0);
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);
```

Chapter 7

Public Helper Utilities

The `+helper` namespace contains exactly nine implementation utilities whose ownership genuinely crosses class boundaries. They are developer-facing building blocks, not an alternative modeling API: ordinary users should enter through `pdbase`, `pdmat`, `pdvar`, and `pdlmi`. Their signatures and current consumers are documented below so maintainers can distinguish shared contracts from class-local implementation details.

7.1 `helper.bernTbl`

Purpose. Build the shared detailed or one-line Bernstein inspection table used by `pdmat` and `pdvar`.

Syntax

```
T = helper.bernTbl(obj, errId, valFcn, exprFcn, rateVerts)
T = helper.bernTbl(..., cellSubs)
T = helper.bernTbl(..., "oneLine")
```

Description

`obj` supplies `cells`, `lbls`, and `coeffs`; `valFcn` maps each coefficient to a table value and `exprFcn` maps it to expression text. `rateVerts` is empty for ordinary data or has one row per active rate vertex. The optional `cellSubs` selects one physical cell; `"oneLine"` returns one expression per cell and rate row.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
T = helper.bernTbl(A, "demo:Invalid", @(x) x, ...
    @(x) string(mat2str(x)), [], "oneLine")
```

```
T =
    CellSubscript          Expression
    -------------    ----------------------
        {[1]}        "(1-alpha)*1 + alpha*2"
```

Remarks

Only one cell selector and the case-insensitive `"oneLine"` option are accepted. Invalid selectors, unknown text, repeated selectors, or rate-row count mismatches raise the caller-owned `errId`, preserving the public class error namespace.

7.2 helper.cellGet

Purpose. Read one flat coefficient leaf from a nested physical-cell tree.

Syntax

```
leaf = helper.cellGet(vals, subs)
```

Description

`vals` is addressed as `vals{i1}{i2}...{i_ell}` and `subs` supplies one index per level. The returned `leaf` is the selected flat coefficient cell or rate-row table; no copy, reshaping, or multi-index cell interpretation is introduced.

Examples and output

```
vals = {{{10, 20}}, {{30, 40}}};
leaf = helper.cellGet(vals, [2 1])
```

```
leaf =
  1x2 cell array
    {[30]}    {[40]}
```

Remarks

The helper deliberately relies on ordinary MATLAB cell indexing. Callers validate the subscript length and bounds before traversal.

7.3 helper.chk

Purpose. Apply shared validation predicates while retaining a caller-owned error identifier and message.

Syntax

```
val = helper.chk(val, errId, msg, tags...)
val = helper.chk(val, errId, msg, tags..., Name, Value)
```

Description

Predicate tags are `numeric`, `real`, `cell`, `struct`, `nonempty`, `scalar`, `vector`, `matrix`, `finite`, `integer`, `positive`, `nonnegative`, `increasing`, and `rowbounds`. Named tests are `Size`, `Numel`, `MinNumel`, `Min`, and `Max`. Successful validation returns `val` unchanged.

Examples and output

```
validated = helper.chk([1 2], "demo:BadValue", "bad value", ...
    "numeric", "real", "finite", "Size", [1 2])
```

```
validated =
     1     2
```

Remarks

A failed data predicate raises `errId`. An unknown predicate, or a named test without a value, raises `helper:InvalidValidatorCall`; these errors signal a validator-programming defect rather than bad caller data.

7.4 `helper.combRows`

Purpose. Enumerate Cartesian combinations in the ordering shared by physical cells, local labels, and rate vertices.

Syntax

```
rows = helper.combRows(vecs)
```

Description

`vecs` is a cell array containing one vector per tensor axis. `rows` contains every Cartesian combination, with earlier dimensions varying more slowly.

Examples and output

```
rows = helper.combRows({0:1, 10:11})
```

```
rows =
     0    10
     0    11
     1    10
     1    11
```

Remarks

This ordering is part of the storage contract. Reordering the rows would misalign tensor labels, physical-cell traversal, and derivative rate vertices.

7.5 helper.isZero

Purpose. Prove the distinct zero notions used by known-data and affine algebra fast paths.

Syntax

```
tf = helper.isZero(val, "num")
tf = helper.isZero(val, "add", matrixSize)
tf = helper.isZero(vals, "vals")
tf = helper.isZero(obj, "obj")
```

Description

`"num"` requires a nonempty finite real numeric zero; `"add"` additionally enforces scalar expansion or the supplied matrix shape; `"vals"` recursively checks nested, symbolic, and rate-structured payloads; and `"obj"` accepts coefficient-backed `pdmat` or `pdvar` evidence.

Examples and output

```
numericZero = helper.isZero(zeros(2), "add", [2 2])
A = pdmat({[0 1]}, {0, 0}, Degree=1);
objectZero = helper.isZero(A, "obj")
```

```
numericZero =
  logical
   1
objectZero =
  logical
   1
```

Remarks

Function-only `pdmat` placeholders are not coefficient evidence and therefore do not prove object zero. Invalid modes raise `helper:InvalidZeroMode`; the wrong mode-specific arity raises `helper:InvalidZeroCall`.

7.6 helper.mapVals

Purpose. Map one payload function across every leaf of a nested `LocalValues` tree.

Syntax

```
mapped = helper.mapVals(vals, fcn, grid)
```

Description

`grid` determines the nested physical-cell shape and `fcn` is applied with `cellfun` to every payload in every selected leaf. The result retains the original physical-cell tree and coefficient ordering.

Examples and output

```
vals = {{{1, 2}}, {{3, 4}}};
mapped = helper.mapVals(vals, @(x) 10*x, {[0 1 2], [0 1]});
secondLeaf = helper.cellGet(mapped, [2 1])
```

```
secondLeaf =
  1x2 cell array
    {[30]}    {[40]}
```

Remarks

The mapping preserves nesting but does not validate payload semantics; the owning class remains responsible for matrix sizes, symbolic affinity, and rate-row structure.

7.7 `helper.matSubs`

Purpose. Normalize supported two-dimensional payload subscripts into explicit row and column index vectors.

Syntax

```
[rows, cols] = helper.matSubs(subs, sz, errId)
```

Description

`subs` is a two-element cell array. Each element may be a finite positive integer vector, a matching logical vector, or `":"`; `sz=[m n]` supplies the available matrix size. Errors use the caller-owned `errId`.

Examples and output

```
[rows, cols] = helper.matSubs({":", 2}, [3 4], "demo:BadSub")
```

```
rows =
     1     2     3
cols =
     2
```

Remarks

The helper rejects linear indexing, empty or out-of-range numeric indices, and logical selectors whose lengths do not match the indexed dimension. It never permits array growth.

7.8 helper.mkGrid

Purpose. Validate tensor-grid vectors and construct the shared `GridInfo` record.

Syntax

```
info = helper.mkGrid(grid)
info = helper.mkGrid(grid, owner)
```

Description

`grid` is a nonempty cell array of finite, real, strictly increasing vectors with at least two nodes each. `owner` defaults to `"pdbase"` and prefixes validation identifiers. The result contains `Vectors`, Cartesian-product `Points`, per-axis `Bounds`, and `NumNodes`.

Examples and output

```
info = helper.mkGrid({[0 1], [10 20]}, "demo");
bounds = info.Bounds
points = info.Points
```

```
bounds =
     0     1
    10    20
points =
     0    10
     0    20
     1    10
     1    20
```

Remarks

Malformed containers use `owner + ":InvalidGrid"`; malformed individual axes use `owner + ":InvalidGridVector"`. Cell counts are derived by consumers and are not duplicated in `GridInfo`.

7.9 helper.mkNest

Purpose. Allocate recursive physical-cell storage and construct each leaf from its complete cell subscript.

Syntax

```
vals = helper.mkNest(nCell, mkLeaf)
vals = helper.mkNest(nCell, mkLeaf, prefix)
```

Description

`nCell` gives the positive cell count per parameter direction and `mkLeaf` receives one completed one-based subscript row. `prefix` is recursion state used internally; ordinary callers omit it. The result is addressed as `vals{i1}{i2}...{i_ell}`.

Examples and output

```
vals = helper.mkNest([2 1], @(subs) {sum(subs)});
secondLeaf = helper.cellGet(vals, [2 1])
```

```
secondLeaf =
  1x1 cell array
    {[3]}
```

Remarks

`mkLeaf` owns the leaf payload. The helper supplies deterministic physical-cell subscripts but does not infer coefficient count, matrix size, or rate semantics.

Chapter 8

Worked Solver Examples

8.1 Deterministic Putinar And Full-Box Selection

This example selects default Putinar and full-box certificates and an explicit full-box order without solving an optimization problem. The reported values depend only on the public assembly contract, not on incidental YALMIP decision-variable identifiers.

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
putinarDefault = direct.applyPutinar();
boxDefault = direct.applyFullBoxPreorder();
boxHigher = boxDefault.applyFullBoxPreorder(2);
polya = direct.applyPolya(1);
boxFromPolya = polya.applyFullBoxPreorder();
putinarFromBox = boxHigher.applyPutinar();
Fbox = toYalmip(boxDefault);

directState = [direct.UseFullBoxPreorder, direct.FullBoxOrder, ...
    numel(direct.Constraints)]
defaultState = [boxDefault.UseFullBoxPreorder, boxDefault.FullBoxOrder, ...
    numel(boxDefault.Constraints)]
putinarState = [putinarDefault.UsePutinar, putinarDefault.PutinarOrder, ...
    numel(putinarDefault.Constraints)]
higherState = [boxHigher.FullBoxOrder, numel(boxHigher.Constraints)]
replacementState = [boxFromPolya.UsePolya, ...
    boxFromPolya.UseFullBoxPreorder, boxFromPolya.FullBoxOrder]
putinarReplacementState = [putinarFromBox.UsePutinar, ...
    putinarFromBox.UseFullBoxPreorder, putinarFromBox.PutinarOrder]
exportedConstraintCount = length(Fbox)
```

```
directState =
     0     0     3
defaultState =
     1     1     5
putinarState =
     1     1     5
higherState =
     2     7
replacementState =
     0     1     1
putinarReplacementState =
     1     0     1
exportedConstraintCount =
     5
```

For the degree-two scalar residual, order one uses two PSD Gram blocks followed by three exact coefficient identities. Order two matches a degree-four target and stores two PSD blocks followed by

five identities. The original `direct` object remains unchanged, and the selection made from `polya` replaces Pólya rather than elevating its already assembled constraints. Likewise, selecting Putinar from `boxHigher` rebuilds from the original residual and clears the full-box selection.

8.2 Solve for a Parameter-Dependent Lyapunov Matrix

The first solver smoke test in `+tests/+pdlmi/test_solver_smoke.m` contrasts a constant Lyapunov matrix with a degree-one, parameter-dependent one. It solves the same decay and positivity constraints in both cases. The degree-one solution is then materialized as known `pdmat` data and plotted. `pdvar` is a decision-expression class and does not itself provide `plot`; applying `value` to its local YALMIP payloads after a successful solve supplies the numeric Bernstein coefficients for `pdmat`.

```
yalmip('clear')
grid = {[0 1]};
A = pdmat(grid, @(x) (1 - x) * [-1 -1; 1 -1] + ...
    x * [-1 -10; 0.1 -1], Degree=1);
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);

Pc = pdvar(2, grid, Degree=0); CconstantDecay = Pc * A + A' * Pc <= -1e-10 * eye(2);
    CconstantPositive = Pc >= eye(2); solConstant = optimize([CconstantDecay.toYalmip, ...
    CconstantPositive.toYalmip], [], opts);

Pd = pdvar(2, grid); CdependentDecay = Pd * A + A' * Pd <= -1e-10 * eye(2);
    CdependentPositive = Pd >= eye(2); solDependent = optimize([CdependentDecay.toYalmip,
    ... CdependentPositive.toYalmip], [], opts); assert(solDependent.problem == 0,
    solDependent.info)

Pplot = pdmat(grid, ...
    {cellfun(@value, Pd.LocalValues{1}, UniformOutput=false)}, ...
    Degree=Pd.Degree);
figure(Name="Parameter-dependent Lyapunov matrix")
plot(Pplot, SamplesPerCell=40, LineWidth=1.5)
title("Solved parameter-dependent Lyapunov matrix")
```

The figure contains one curve for each entry of the solved 2-by-2 matrix $P(\rho)$, sampled on the parameter interval. It is a diagnostic visualization of the feasible solution, not a certificate beyond the coefficient-wise constraints supplied to the solver. The constant-case result is retained to exercise the comparison in the smoke test; only a MOSEK solve is used there to certify its intended infeasibility distinction.

8.3 Solve a Rate-Dependent Block LMI

This example is adapted from `+tests/+pdlmi/test_solver_smoke.m`. The goal is to find a parameter-dependent $P(\rho)$ and scalar γ such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \qquad P(\rho) \succeq 0.$$

The derivative term $\dot{P}$ is built with `rhodiff(P, [-1 1])`. The comparison operators create `pdlmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP. The second listing assigns the solver- and tolerance-dependent numeric value of γ to `gammaValue`. For this model, `numel(E1.Constraints)` returns 6, and `numel(E2.Constraints)` returns 2.

```
yalmip('clear')
grid = linspace(0, 1, 2);
A = pdmat(grid, @(x) [−1, 0.5; −1, −2] + ...
    x * [−1.3, −20; 2, −10], Degree=1);
B = pdmat(grid, @(x) [1, −4; −1, −1] + ...
    x * [2.2, 0.5; −6, −5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = pdvar(2, grid);
diffP = rhodiff(P, [−1 1]);
gamma = pdvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
      B' * P, −gamma * eye(2), Dmat';
      Cmat, Dmat, −gamma * eye(2)] <= 0;
E2 = P >= 0;
decayConstraintCount = numel(E1.Constraints)
positivityConstraintCount = numel(E2.Constraints)
```

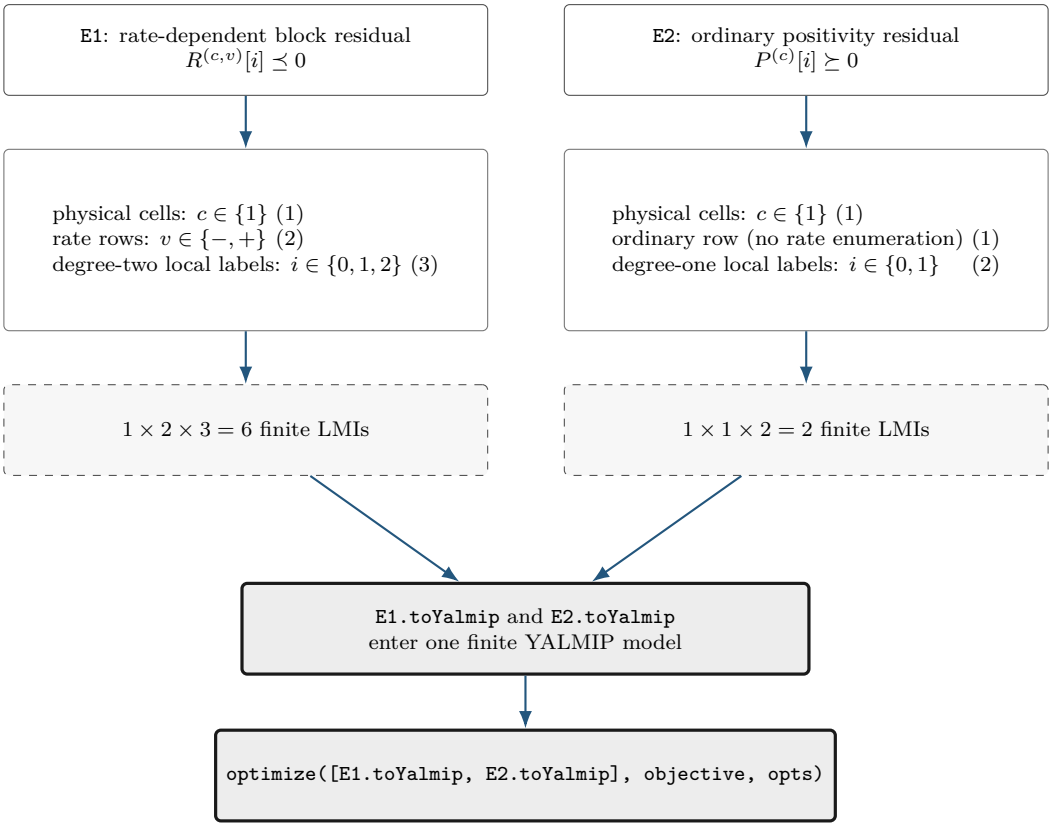
```
decayConstraintCount =
     6

positivityConstraintCount =
     2
```

```
objective = gamma.LocalValues{1}{1};
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
gammaValue = value(objective);
assert(isfinite(gammaValue))
gammaValue
```

The constraint counts above follow directly from the three independent assembly axes: physical-cell identity, rate-row identity, and local Bernstein label. In this tested example the two wrappers use

the same physical grid but different rate-row and degree structures.



The numbers (6) and (2) are certificate-assembly counts: they state how many finite semidefinite constraints represent `E1` and `E2`. They are not evidence that the optimization succeeds. Feasibility is checked separately by `sol.problem`, and the finite objective is checked after the shared `optimize` call.

Appendix A

Bernstein Representation and Exact Operations

This appendix develops the representation used by the package from the one-dimensional degree-one case to cell-local tensor polynomials. The reader needs only scalar polynomials, matrices, and elementary calculus. Every conversion, elevation, product, derivative, and subdivision below is exact; none is a sampling approximation. Farouki gives a broader historical and numerical account [1].

Each main construction follows the same reading path: *Basic idea*, *Formula*, *Worked example*, and *Visual conclusion*. The extra staging is intentional: first identify what the operation means, then read its indices, and only afterward connect it to package storage.

A.1 Convex Combinations Before Polynomials

Basic idea. A convex combination of numbers or matrices has the form

$$X = \sum_{i=0}^{m} w_i X_i, \qquad w_i \geq 0, \qquad \sum_{i=0}^{m} w_i = 1.$$

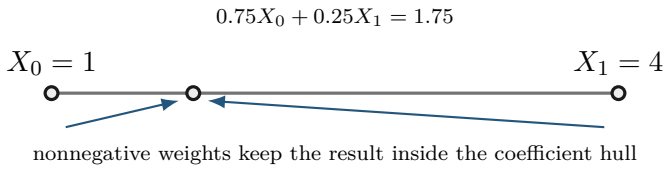
For scalars, X lies between the smallest and largest X_i. For symmetric matrices, if every $X_i \succeq 0$, then $X \succeq 0$, because

$$z^{\mathsf{T}} X z = \sum_i w_i \, z^{\mathsf{T}} X_i z \geq 0 \quad \text{for every } z.$$

Formula. Bernstein bases are useful because their basis values are precisely such weights at every point of a cell.

Worked example. Take $X_0 = 1$, $X_1 = 4$, and weights 0.75 and 0.25. Their combination is 1.75, which remains between the two inputs.

Visual conclusion.



The implication is one-way. A polynomial may be positive everywhere while one Bernstein coefficient is negative; Appendix B uses that fact to motivate stronger certificates.

A.2 Degree-One Interpolation On One Physical Cell

Basic idea. Let a physical cell be $[\rho_1^{(c)}, \rho_1^{(c+1)}]$, with width $h_c = \rho_1^{(c+1)} - \rho_1^{(c)} > 0$. Its forward local coordinate is

$$\alpha = \frac{\rho_1 - \rho_1^{(c)}}{h_c} \in [0, 1].$$

Formula. The degree-one Bernstein basis is

$$B_0^1(\alpha) = 1 - \alpha, \qquad B_1^1(\alpha) = \alpha.$$

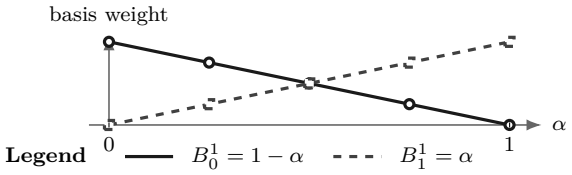
Therefore

$$M_c(\rho) = (1 - \alpha)C^{(c)}[0] + \alpha C^{(c)}[1].$$

At the lower physical face, $\alpha = 0$ and the value is $C^{(c)}[0]$; at the upper face, $\alpha = 1$ and the value is $C^{(c)}[1]$. Between them the matrix follows the straight segment joining those two coefficient matrices.

Worked example. For example, $C^{(c)}[0] = 2$ and $C^{(c)}[1] = 4$ give $M_c(0.25) = 0.75(2) + 0.25(4) = 2.5$, exactly as in section 4.4.

Visual conclusion.



A.3 Higher Degree And The Partition Of Unity

Basic idea. Higher degree adds interior shape controls while keeping the same physical cell. It does not add physical grid nodes.

Formula. For degree m, define

$$B_i^m(\alpha) = \binom{m}{i}(1 - \alpha)^{m-i}\alpha^i, \qquad i = 0, \ldots, m.$$

Each basis function is nonnegative on $[0, 1]$. The binomial theorem gives

$$\sum_{i=0}^{m} B_i^m(\alpha) = ((1 - \alpha) + \alpha)^m = 1.$$

Thus

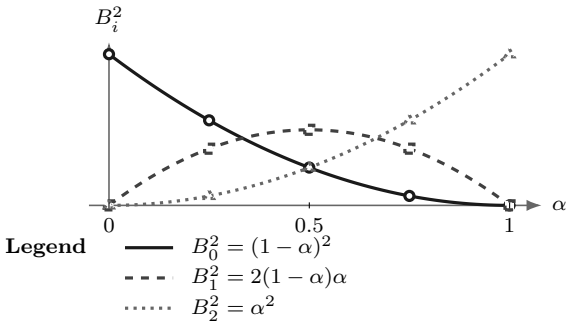
$$M_c(\alpha) = \sum_{i=0}^{m} B_i^m(\alpha)C^{(c)}[i]$$

is a convex combination of its local coefficient matrices.

Worked example. For degree two,

$$B_0^2 = (1-\alpha)^2, \qquad B_1^2 = 2(1-\alpha)\alpha, \qquad B_2^2 = \alpha^2.$$

Visual conclusion.



At every α, the three plotted heights are nonnegative and sum to one.

The label i identifies a basis position inside the selected physical cell. It is not a physical grid-node subscript.

A.4 Worked Conversion: $1 + 2\alpha + 3\alpha^2$

Consider

$$p(\alpha) = 1 + 2\alpha + 3\alpha^2.$$

Seek degree-two Bernstein coefficients b_0, b_1, b_2:

$$p = b_0(1-\alpha)^2 + 2b_1(1-\alpha)\alpha + b_2\alpha^2.$$

Matching powers of α gives

$$b_0 = 1, \qquad b_1 = 2, \qquad b_2 = 6.$$

Indeed,

$$1B_0^2 + 2B_1^2 + 6B_2^2 = (1 - 2\alpha + \alpha^2) + (4\alpha - 4\alpha^2) + 6\alpha^2 = 1 + 2\alpha + 3\alpha^2.$$

The three values $1, 2, 6$ are coefficients of basis functions, not samples at three equally spaced physical points. Only the two endpoint coefficients are endpoint values. At $\alpha = 1/2$,

$$p(1/2) = \tfrac{1}{4}(1) + \tfrac{1}{2}(2) + \tfrac{1}{4}(6) = 2.75.$$

```
p = pdmat({[0 1]}, {1, 2, 6}, Degree=2);
valueAtHalf = p.evaluate(0.5)
T = bernsteinTable(p, "oneLine")
```

```
valueAtHalf =
    2.7500
T =
    CellSubscript                    Expression
    -------------    ------------------------------------------

        {[1]}        "(1-alpha)^2*1 + 2(1-alpha)alpha*2 + alpha^2*6"
```

A.5 Exact Power–Bernstein Conversion

More generally, use brackets for algebraic coefficient labels as well:

$$p(\alpha) = \sum_{j=0}^{n} a[j]\alpha^j = \sum_{k=0}^{n} b[k]B_k^n(\alpha).$$

Power coefficients convert to Bernstein coefficients by

$$b[k] = \sum_{j=0}^{k} \frac{\binom{k}{j}}{\binom{n}{j}} a[j], \qquad k = 0, \ldots, n.$$

The inverse map is

$$a[j] = \binom{n}{j} \sum_{i=0}^{j} (-1)^{j-i} \binom{j}{i} b[i], \qquad j = 0, \ldots, n.$$

These are changes of basis inside the same degree-n polynomial space. For matrix polynomials they act entrywise. For tensor polynomials they act along one coordinate axis at a time.

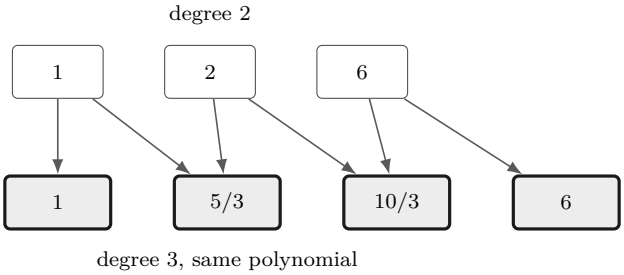
A.6 Lossless Degree Elevation

In one parameter, a polynomial on physical cell c with degree m has a Bernstein representation of every higher degree. One elevation step gives

$$\widehat{C}^{(c)}[0] = C^{(c)}[0], \qquad \widehat{C}^{(c)}[k] = \frac{k}{m+1} C^{(c)}[k-1] + \left(1 - \frac{k}{m+1}\right) C^{(c)}[k], \qquad \widehat{C}^{(c)}[m+1] = C^{(c)}[m].$$

For the worked coefficients $[1, 2, 6]$, elevation from degree two to degree three produces

$$\left[1, \frac{5}{3}, \frac{10}{3}, 6\right].$$



Direct elevation from m to $M \geq m$ is

$$\widehat{C}^{(c)}[k] = \sum_{i=\max(0,k-M+m)}^{\min(m,k)} C^{(c)}[i] \frac{\binom{m}{i}\binom{M-m}{k-i}}{\binom{M}{k}}.$$

The new coefficients are fixed convex combinations of the old ones. Elevation therefore changes representation but adds no decision freedom. Constructing a new higher-degree `pdvar` does enlarge the decision parameterization.

A.7 Physical Tensor Grids And Local Tensor Labels

Suppose direction s has physical nodes

$$\rho_s^{(1)} < \rho_s^{(2)} < \cdots < \rho_s^{(k_s)}.$$

The counts k_s may differ. Thus $\mathcal{G} = \mathcal{G}_1 \times \cdots \times \mathcal{G}_\ell$ has $\prod_s k_s$ physical nodes. A physical hypercube is identified separately by the one-based multi-index $\mathbf{c} = (c_1, \ldots, c_\ell)$. On that hypercube,

$$\alpha_s = \frac{\rho_s - \rho_s^{(c_s)}}{\rho_s^{(c_s+1)} - \rho_s^{(c_s)}}, \qquad s = 1, \ldots, \ell.$$

For the package's scalar degree m,

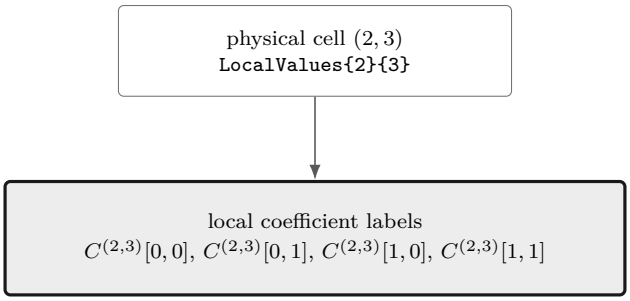
$$C^{(\mathbf{c})}(\boldsymbol{\rho}) = \sum_{\mathbf{i} \in \{0,\ldots,m\}^\ell} B_{\mathbf{i}}^m(\boldsymbol{\alpha}) C^{(\mathbf{c})}[\mathbf{i}], \qquad B_{\mathbf{i}}^m = \prod_{s=1}^{\ell} B_{i_s}^m(\alpha_s).$$

There are $\prod_s (k_s - 1)$ physical hypercubes and $(m+1)^\ell$ zero-based local Bernstein labels per hypercube. Physical node and hypercube indices remain one-based.

```
obj = pdbase({[0 1 2], [10 20 30 40]}, [1 1], 1);
physicalCellCount = obj.ncell()
coefficientsPerCell = obj.ncoeff()
localLabels = obj.lbls()
```

```
physicalCellCount =
     6
coefficientsPerCell =
     4
localLabels =
     0     0
     0     1
     1     0
     1     1
```

The nested tree `LocalValues{c1}{c2}...{cell}` follows physical-cell coordinates. Only after reaching a leaf does the flat coefficient cell follow the `lbls` order. Earlier dimensions vary more slowly in both `cells` and `lbls`.



Select the physical-cell leaf first; only then read the flat local coefficient labels in `lbls` *order.*

A.8 Products As Ordered Weighted Convolution

Basic idea. Multiplying two Bernstein polynomials creates every ordered pair of input coefficients. Pairs with the same label sum contribute to the same output. For matrix payloads, "ordered" matters: the left factor stays on the left.

Formula. First consider one parameter and fix the one-based physical cell c. Let

$$P = \sum_{i=0}^{m} B_i^m P^{(c)}[i], \qquad Q = \sum_{j=0}^{n} B_j^n Q^{(c)}[j].$$

The basis-product identity

$$B_i^m B_j^n = \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{i+j}} B_{i+j}^{m+n}$$

gives

$$(PQ)^{(c)}[k] = \sum_{i+j=k} P^{(c)}[i] Q^{(c)}[j] \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}}.$$

This is an ordered matrix convolution. In general $P^{(c)}[i]Q^{(c)}[j] \neq Q^{(c)}[j]P^{(c)}[i]$.

Worked example. Take quadratic coefficients $P = [1, 2, 6]$ and affine coefficients $Q = [2, 4]$. The anti-diagonals give

$$\begin{aligned} R[0] &= 1(2) = 2, \\ R[1] &= \big(1(4) + 2\,2(2)\big)/3 = 4, \\ R[2] &= \big(2\,2(4) + 6(2)\big)/3 = 28/3, \\ R[3] &= 6(4) = 24. \end{aligned}$$

```
P = pdmat({[0 1]}, {1, 2, 6}, Degree=2);
Q = pdmat({[0 1]}, {2, 4}, Degree=1);
R = P * Q;
productDegree = R.Degree
productCoefficients = cell2mat(R.coeffs(1))
```

```
productDegree =
     3
productCoefficients =
    2.0000    4.0000    9.3333   24.0000
```

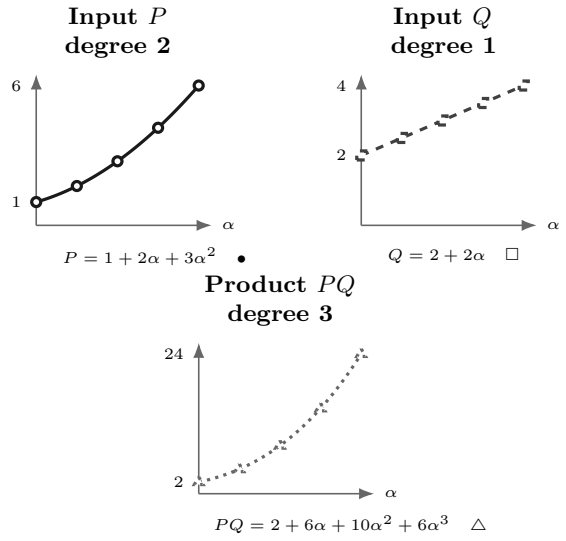
The product degree in the transcript is three: degrees add, so $2 + 1 = 3$. The four outputs are coefficients in that cubic Bernstein basis, not values sampled at four points.

The same calculation can be checked as a function identity. On $\alpha \in [0, 1]$,

$$P(\alpha) = 1 + 2\alpha + 3\alpha^2, \qquad Q(\alpha) = 2 + 2\alpha,$$

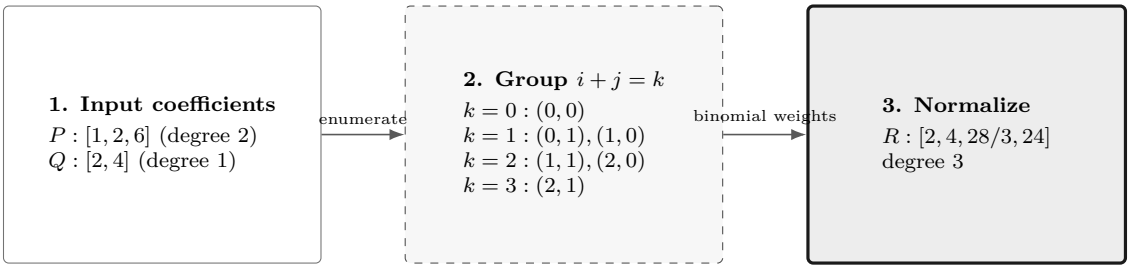
and

$$P(\alpha)Q(\alpha) = 2 + 6\alpha + 10\alpha^2 + 6\alpha^3.$$



Direct formula labels sit outside the plot regions. Solid/circle, dashed/square, and dotted/triangle treatments distinguish the three functions without relying on color.

Visual conclusion.



Only the panel-to-panel arrows encode flow. Contributions are listed inside their anti-diagonal group, so no edges cross labels or equations.

This is the algorithm behind protected `bernProd` (section 3.6.4) and public `pdmat`/`pdvar` multiplication (sections 4.6.2 and 5.6).

For tensor labels,

$$(PQ)^{(\mathbf{c})}[\mathbf{k}] = \sum_{\mathbf{i}+\mathbf{j}=\mathbf{k}} P^{(\mathbf{c})}[\mathbf{i}]Q^{(\mathbf{c})}[\mathbf{j}] \prod_{s=1}^{\ell} \frac{\binom{m}{i_s}\binom{n}{j_s}}{\binom{m+n}{k_s}}.$$

Labels add componentwise; physical-cell identity does not change.

A.9 Differentiation And Rate Vertices

Basic idea. Differentiate inside each physical cell. Adjacent control differences produce the derivative coefficients; multiplying by a bounded rate then creates one stored row per rate-box vertex. Value continuity of P does not force the cell-local derivatives to match across a face.

Formula. For one parameter, on physical cell c of width h_c,

$$\frac{\partial P}{\partial \rho} = \frac{m}{h_c} \sum_{i=0}^{m-1} (C^{(c)}[i+1] - C^{(c)}[i]) B_i^{m-1}(\alpha).$$

Worked example. For the worked polynomial $[1, 2, 6]$ on $[0, 2]$, $m = 2$ and $h = 2$, so the derivative coefficients are

$$\left[\frac{2}{2}(2-1), \frac{2}{2}(6-2)\right] = [1, 4].$$

This represents $dp/d\rho = 1 + 3\alpha$.

In ℓ dimensions, differentiation with respect to ρ_s replaces each coefficient by

$$\frac{m}{h_s}\left(C^{(\mathbf{c})}[\mathbf{i} + \mathbf{e}_s] - C^{(\mathbf{c})}[\mathbf{i}]\right),$$

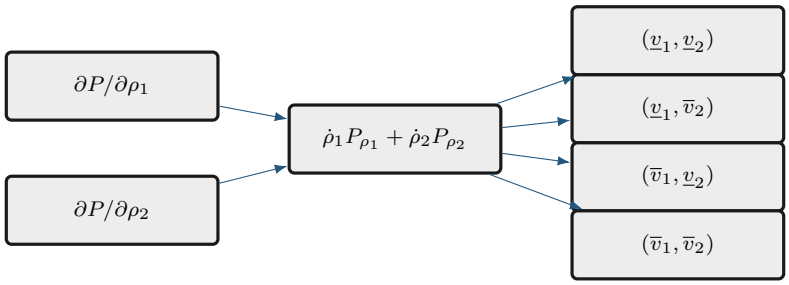
reduces the degree along direction s, and leaves the other label components unchanged. The implementation elevates partial derivatives to a common tensor degree before adding them.

Finally,

$$\dot{P} = \sum_{s=1}^{\ell} \frac{\partial P}{\partial \rho_s} \dot{\rho}_s.$$

This matrix expression is affine in $\dot{\boldsymbol{\rho}}$. Every value inside the rate box is therefore a convex combination of the 2^ℓ vertex matrices. Because the positive- and negative-semidefinite cones are convex, the requested semidefinite sign holds on the whole rate box whenever it holds at every vertex. `rhodiff` stores one row for each vertex.

Visual conclusion.



A.9.1 Rate-Vertex Row Storage

Let $\texttt{RateBounds}(s,:) = [\underline{\dot\rho}_s \;\; \overline{\dot\rho}_s]$. The rate box has the finite vertex set

$$\mathcal{V} = \prod_{s=1}^{\ell} \{\underline{\dot\rho}_s, \overline{\dot\rho}_s\}, \qquad |\mathcal{V}| = 2^\ell.$$

For a derivative object $D = \texttt{rhodiff}(P, \texttt{RateBounds})$ and one physical hypercube $c = (c_1, \ldots, c_\ell)$, the leaf $\texttt{D.LocalValues}\{c_1\}\cdots\{c_\ell\}$ is a 2^ℓ-by-$(\texttt{D.Degree}+1)^\ell$ cell array. Its rows enumerate $\mathcal{V}$; its columns enumerate the local Bernstein labels in `combRows` order. Each entry already contains the rate-weighted sum $\sum_s v_s\, \partial P^{(c)}/\partial\rho_s$ at that row's vertex v. The rows add rate alternatives inside one existing hypercube; they do not add physical cells.

For $\ell = 2$, the lower/upper tensor order is

row	$\dot\rho_1$	$\dot\rho_2$
1	$\underline{\dot\rho}_1$	$\underline{\dot\rho}_2$
2	$\underline{\dot\rho}_1$	$\overline{\dot\rho}_2$
3	$\overline{\dot\rho}_1$	$\underline{\dot\rho}_2$
4	$\overline{\dot\rho}_1$	$\overline{\dot\rho}_2$

The last parameter direction changes fastest. This is the same tensor ordering used by the package's coefficient and cell traversal, so `pdlmi` can constrain every rate row without reconstructing the rate box.

A.10 de Casteljau Evaluation And Exact Subdivision

de Casteljau recursion evaluates a Bernstein polynomial using only affine combinations. For one parameter, on a fixed physical cell c, begin with $C^{(c,0)}[i] = C^{(c)}[i]$ and define

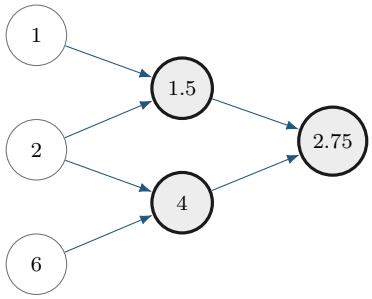
$$C^{(c,r)}[i] = (1-\tau)C^{(c,r-1)}[i] + \tau C^{(c,r-1)}[i+1], \qquad r = 1, \ldots, m.$$

The value at $\alpha = \tau$ is $C^{(c,m)}[0]$.

For $[1, 2, 6]$ and $\tau = 1/2$, the recursion is

$$[1, 2, 6] \longrightarrow [1.5, 4] \longrightarrow [2.75].$$

The left edge of this triangle gives exact coefficients $[1, 1.5, 2.75]$ on the left subinterval; the right edge gives $[2.75, 4, 6]$ on the right.



One recursion triangle supplies both the evaluation and the two child-cell coefficient lists.

Tensor evaluation or subdivision applies the same recursion along one parameter direction at a time. Subdivision changes the physical-cell partition and can tighten coefficient hulls while preserving the polynomial. The package uses exact coefficient transformations internally where documented, but it does not currently expose a public de Casteljau, subdivision, or adaptive-refinement API.

A.11 What Changes, And What Does Not

The following distinctions prevent several common modeling mistakes.

Operation	Changes	Preserves
Basis conversion	coefficient coordinates	polynomial, degree, physical cell
Degree elevation	Bernstein degree and coefficient list	polynomial, physical grid, decision freedom
Grid subdivision	physical-cell partition and local coefficients	polynomial on the original domain
Higher-degree `pdvar`	number of decision coefficients	grid and matrix size
Multiplication	polynomial degree and coefficients	physical-cell identity
Differentiation	degree and coefficient values	physical-cell partition before rate rows

These representation facts support the finite certificates in the next appendix. They do not by themselves select a solver or prove that one certificate is necessary.

A.12 Further Reading

For a concise orientation before the primary paper by Farouki [1], see the Bernstein polynomial overview and the de Casteljau overview. They are background references for the basis and recursion; the package's cell-local storage and certificate behavior are defined by this manual and the source implementation.

Appendix B

Polynomial-Matrix Certificates on Hypercubes

This appendix starts with positive semidefinite matrices, builds SOS and Gram representations, and then explains the four certificate choices implemented by `pdlmi`. Background constructions are included so a new reader can recognize the mathematical landscape; only explicitly marked methods are package APIs.

As in the preceding appendix, each major certificate is revealed as *Basic idea* → *Formula* → *Worked example* → *Visual conclusion*. Keep two questions separate while reading: what polynomial matrix is represented, and what finite cone is being used to certify its sign?

For a residual $F \preceq 0$, define $S = -F$. For $F \succeq 0$, define $S = F$. The common problem is

$$S(\boldsymbol{\alpha}) \succeq 0 \quad \text{for every } \boldsymbol{\alpha} \in [0,1]^{\ell}.$$

All Gram constraints below prove non-strict inequalities. A strict model must state its own margin, for example $S - \varepsilon I \succeq 0$. The package adds no hidden margin.

B.1 Positive Semidefinite Matrices And LMIs

Basic idea. A real symmetric matrix Q is positive semidefinite when

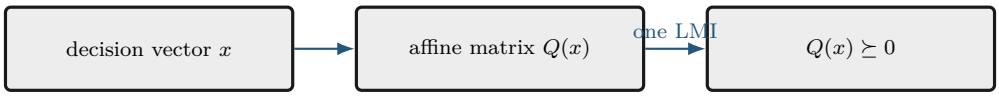
$$z^{\mathsf{T}} Q z \geq 0 \qquad \text{for every vector } z.$$

Formula. An LMI is an affine matrix expression constrained to the PSD cone:

$$Q(x) = Q_0 + x_1 Q_1 + \cdots + x_r Q_r \succeq 0.$$

The unknowns enter affinely, so this is a semidefinite program. A polynomial matrix sign condition appears infinite because it asks for one PSD matrix at every point. A certificate replaces those pointwise conditions by finitely many PSD matrices and affine coefficient identities.

Visual conclusion.



B.2 SOS And Gram Matrices

Basic idea. A scalar polynomial is a sum of squares (SOS) if

$$p(z) = q_1(z)^2 + \cdots + q_s(z)^2.$$

Every SOS polynomial is nonnegative. Choose a polynomial basis vector $v_d(z)$. The Gram form

$$p(z) = v_d(z)^\mathsf{T} Q v_d(z), \qquad Q \succeq 0,$$

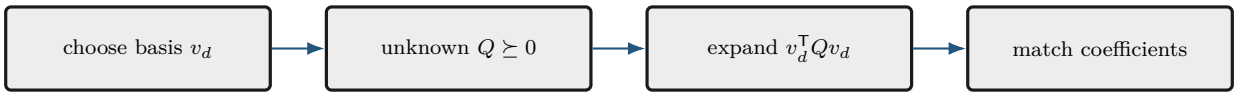
is SOS because a factorization $Q = L^\mathsf{T} L$ yields $p = \|L v_d\|_2^2$. Expanding the Gram form and matching every coefficient of p gives equations linear in the entries of Q. Thus an SOS search is a finite PSD constraint plus affine equalities.

Formula. For a symmetric n-by-n polynomial matrix, set

$$Z_d(z) = v_d(z) \otimes I_n, \qquad S(z) = Z_d(z)^\mathsf{T} Q Z_d(z), \quad Q \succeq 0.$$

Then $y^\mathsf{T} S(z) y = (Z_d y)^\mathsf{T} Q (Z_d y) \geq 0$ for every y, so $S(z) \succeq 0$. Matrix-SOS relaxations are developed in [9].

Visual conclusion.



B.3 Full-Space SOS

An unweighted Gram form proves $S(z) \succeq 0$ on all of $\mathbb{R}^\ell$. That can be much stronger than positivity only on a box. For example, $\alpha(1-\alpha)$ is nonnegative on $[0,1]$ but negative outside it. The package does not expose a general full-space SOS parser; this construction is background for understanding weighted certificates.

B.4 Direct Bernstein Coefficients

Basic idea. If every local coefficient matrix has the requested sign, every convex combination of those coefficients has the same sign. This yields a small, transparent sufficient certificate.

Formula. On physical cell $\mathbf{c}$ and one rate row, write

$$S(\boldsymbol{\alpha}) = \sum_{\mathbf{i}} B_{\mathbf{i}}^m(\boldsymbol{\alpha}) C^{(\mathbf{c})}[\mathbf{i}].$$

Because the basis weights are nonnegative and sum to one,

$$C^{(\mathbf{c})}[\mathbf{i}] \succeq 0 \ \forall \mathbf{i} \quad \Longrightarrow \quad S(\boldsymbol{\alpha}) \succeq 0.$$

This is the package default. It introduces no Gram variables and creates one constraint per physical cell, rate row, and local label. The converse is false.

Worked example. Consider

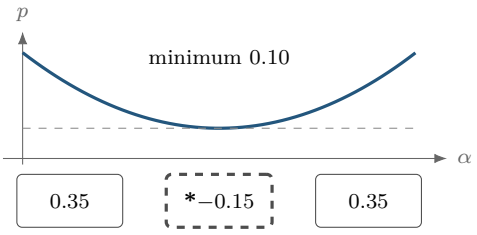
$$p(\alpha) = \alpha^2 - \alpha + 0.35 = (\alpha - 0.5)^2 + 0.10.$$

It is strictly positive, with minimum 0.10, but its degree-two Bernstein coefficients are

$$[\,0.35,\ -0.15,\ 0.35\,].$$

The negative middle coefficient defeats the direct certificate without making the polynomial negative.

Visual conclusion.



One negative Bernstein coefficient does not imply a negative curve.

The same polynomial has the PSD Bernstein Gram representation

$$\zeta_1 = \begin{bmatrix} 1-\alpha \\ \alpha \end{bmatrix}, \qquad Q = \begin{bmatrix} 0.35 & -0.15 \\ -0.15 & 0.35 \end{bmatrix} \succeq 0, \qquad p = \zeta_1^{\mathsf{T}} Q \zeta_1.$$

The eigenvalues of Q are 0.20 and 0.50. A complete Gram certificate can therefore succeed when coefficient-wise signs fail.

B.5 Pólya As Lossless Elevation

Basic idea. Pólya selection does not change the residual polynomial. It rewrites that same polynomial at a higher Bernstein degree and then retries the direct coefficient test.

Formula. Classical Pólya theory concerns homogeneous polynomials strictly positive on a simplex [12]. For one box coordinate,

$$\lambda_0 = 1 - \alpha, \quad \lambda_1 = \alpha, \quad \lambda_0 + \lambda_1 = 1.$$

For ℓ coordinates, the implemented multiplier is

$$\prod_{s=1}^{\ell} (\lambda_{s,0} + \lambda_{s,1})^d = 1.$$

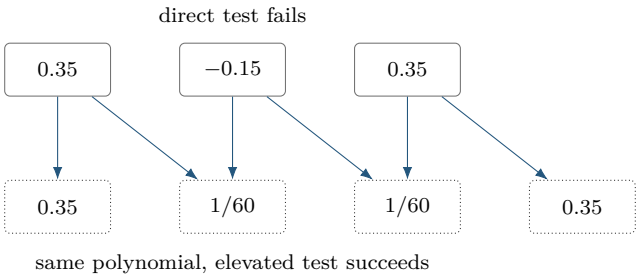
Regrouping is exactly Bernstein degree elevation: the represented residual does not change.

Worked example. For the worked coefficients,

$$[\,0.35, -0.15, 0.35\,] \longrightarrow \left[0.35, \frac{1}{60}, \frac{1}{60}, 0.35\right]$$

after one elevation. Every new coefficient is positive, so the elevated coefficient test succeeds.

Visual conclusion.



`applyPolya(d)` applies one scalar increment in every parameter direction and then coefficient constraints to every cell and rate row. It adds neither Gram variables nor decision freedom. A fixed-level failure is inconclusive; the package makes no tensor-box completeness claim.

B.6 Exact Interval Markov–Lukács Forms

Basic idea. On one interval, endpoint factors can encode the domain exactly. Even and odd target degrees require different weighted Gram patterns, so parity is a structural choice rather than a cosmetic one.

Formula. In one variable, interval nonnegativity has exact parity-specific descriptions [6, 7]. If $\deg p \le 2r$,

$$p \ge 0 \text{ on } [0,1] \iff p = s_0 + \alpha(1-\alpha)s_1,$$

where s_0, s_1 are SOS of degrees at most $2r$ and $2r-2$. If $\deg p \le 2r+1$,

$$p \ge 0 \text{ on } [0,1] \iff p = (1-\alpha)s_L + \alpha s_U,$$

where s_L, s_U are SOS of degree at most $2r$.

For polynomial matrices, with

$$\mathcal{Z}_d(\alpha) = \begin{bmatrix} B_0^d I_n & \cdots & B_d^d I_n \end{bmatrix}^{\mathsf{T}},$$

the even and odd forms are

$$S = \mathcal{Z}_r^{\mathsf{T}} Q_0 \mathcal{Z}_r + \alpha(1-\alpha)\mathcal{Z}_{r-1}^{\mathsf{T}} Q_1 \mathcal{Z}_{r-1}, \qquad Q_0, Q_1 \succeq 0,$$

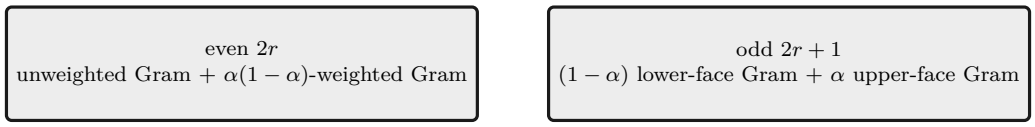
and

$$S = (1-\alpha)\mathcal{Z}_r^{\mathsf{T}} Q_L \mathcal{Z}_r + \alpha \mathcal{Z}_r^{\mathsf{T}} Q_U \mathcal{Z}_r, \qquad Q_L, Q_U \succeq 0.$$

The second even block is absent at $r = 0$. See [8, 9] for polynomial-matrix context.

Worked example. A cubic residual has degree $2r+1$ with $r = 1$. It therefore uses two degree-one Gram bases: one multiplied by $1-\alpha$ and one by α. This is the one-dimensional full-box pattern used for the cubic residual in the introductory DPD-LMI example.

Visual conclusion.



Partition Q into n-by-n blocks Q_{ij}. The Bernstein coefficient at label k of $\alpha^a(1-\alpha)^b \mathcal{Z}_d^{\mathsf{T}} Q \mathcal{Z}_d$, expressed at total degree $2d+a+b$, is

$$\mathcal{G}_k^{d,a,b}(Q) = \frac{1}{\binom{2d+a+b}{k}} \sum_{\substack{0\le i,j\le d \\ i+j=k-a}} \binom{d}{i}\binom{d}{j} Q_{ij}.$$

Equating this affine map to each target coefficient gives exact linear identities; $Q \succeq 0$ supplies the LMIs. For one parameter, `applyFullBoxPreorder` implements these exact parity forms. Its minimum absolute order is $\lfloor m/2 \rfloor$.

B.7 Putinar Modules And Schmüdgen Preorderings

For

$$K = \{z : g_1(z) \ge 0, \dots, g_q(z) \ge 0\},$$

a Putinar quadratic module has the form

$$\sigma_0 + \sum_{j=1}^{q} \sigma_j g_j, \qquad \sigma_j \text{ SOS}.$$

Under the Archimedean hypothesis, every polynomial strictly positive on K has such a representation at some degree [10]. Compactness alone is not that theorem statement. The package implements one box-specific, fixed-order selector, not a general-domain or general-generator parser.

For one parameter, `applyPutinar(r)` dispatches to the same exact even/odd Markov–Lukács forms as `applyFullBoxPreorder(r)`. Its absolute default and minimum are $r = \lfloor m/2 \rfloor$. Even targets use the unweighted and $\alpha(1-\alpha)$-weighted Gram blocks and match at degree $2r$; odd targets use the two endpoint-weighted Gram blocks and match at degree $2r+1$.

For $\ell \ge 2$, use the local box generators

$$g_s(\boldsymbol{\alpha}) = \alpha_s(1-\alpha_s),$$

and `applyPutinar(r)` uses

$$F = S_0 + \sum_{s=1}^{\ell} g_s S_s, \qquad S_0 = \mathcal{Z}_{r\mathbf{1}}^{\mathsf{T}} Q_0 \mathcal{Z}_{r\mathbf{1}}, \quad S_s = \mathcal{Z}_{r\mathbf{1}-\boldsymbol{e}_s}^{\mathsf{T}} Q_s \mathcal{Z}_{r\mathbf{1}-\boldsymbol{e}_s},$$

where every present $Q_s \succeq 0$. In this multivariate branch the order is absolute and satisfies $r \ge \lceil m/2 \rceil$. Exact identities match the sign-normalized target, elevated without changing it, at tensor degree $2r$. Negative-degree multiplier blocks are omitted at $r = 0$. Each physical cell and active rate row owns independent Gram and equality blocks; no generator products, implicit margin, or solver call are introduced.

A Schmüdgen full preordering includes every square-free generator product:

$$p = \sum_{\boldsymbol{\nu}\in\{0,1\}^q} \sigma_{\boldsymbol{\nu}} \prod_{j=1}^{q} g_j^{\nu_j}, \qquad \sigma_{\boldsymbol{\nu}} \text{ SOS}.$$

Strict positivity on a compact basic closed semialgebraic set has such a representation [11]. A finite truncation is still a sufficient certificate, and up to 2^q multiplier blocks may be required.

B.8 The Implemented Multivariate Full Box Preordering

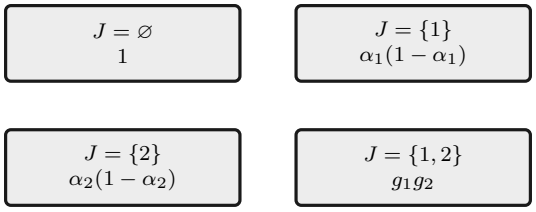
Basic idea. For several parameters, use every square-free product of the box generators. Each subset selects one weighted Gram block, and exact coefficient identities make the sum equal the stored residual on one physical cell and rate row.

Formula. For the hypercube, use

$$g_s(\boldsymbol{\alpha}) = \alpha_s(1-\alpha_s), \qquad s = 1, \dots, \ell.$$

Worked example. For two parameters, the four subset products are $1, g_1, g_2, g_1g_2$.

Visual conclusion.



For $\ell \geq 2$ and absolute order r, define

$$\mathcal{Z}_{\boldsymbol{d}} = \left(\bigotimes_{s=1}^{\ell} \begin{bmatrix} B_0^{d_s} & \cdots & B_{d_s}^{d_s} \end{bmatrix}^{\mathsf{T}} \right) \otimes I_n.$$

The implemented form is

$$S = \sum_{J\subseteq\{1,\dots,\ell\}} \left(\prod_{s\in J} g_s \right) \mathcal{Z}_{r\mathbf{1}-\boldsymbol{\chi}_J}^{\mathsf{T}} Q_J \mathcal{Z}_{r\mathbf{1}-\boldsymbol{\chi}_J}, \qquad Q_J \succeq 0.$$

Terms with negative tensor degree are omitted. A present block has dimension

$$n \prod_{s=1}^{\ell} \left(r + 1 - \mathbf{1}_{s\in J}\right).$$

Every physical cell and active rate row gets independent dense Gram blocks and exact coefficient identities. The minimum order is $\lceil m/2 \rceil$ for $\ell \geq 2$. This is a specific dense Bernstein realization of the complete box preordering, not a full-space SOS, Putinar-only, sparse, or general-domain parser. A fixed multivariate order is sufficient rather than exact.

Finite SDP example: $\ell = 2$, $m = 3$. Let $(a, b) \in [0, 1]^2$ be the local coordinates of one cell, and suppose the sign-normalized residual is

$$F(a, b) = \sum_{i=0}^{3} \sum_{k=0}^{3} B_i^3(a) B_k^3(b) F_{i,k} \succeq 0.$$

The default order is $r = \lceil 3/2 \rceil = 2$. After exact elevation to degree $(4, 4)$, the implementation introduces four independent PSD Gram matrices and enforces the identity

$$\begin{aligned} F(a, b) &= \mathcal{Z}_{(2,2)}^{\mathsf{T}} Q_{\varnothing} \mathcal{Z}_{(2,2)} + a(1-a) \mathcal{Z}_{(1,2)}^{\mathsf{T}} Q_1 \mathcal{Z}_{(1,2)} \\ &\quad + b(1-b) \mathcal{Z}_{(2,1)}^{\mathsf{T}} Q_2 \mathcal{Z}_{(2,1)} \\ &\quad + a(1-a) b(1-b) \mathcal{Z}_{(1,1)}^{\mathsf{T}} Q_{12} \mathcal{Z}_{(1,1)}, \\ Q_{\varnothing} \succeq 0, &\qquad Q_1 \succeq 0, \qquad Q_2 \succeq 0, \qquad Q_{12} \succeq 0. \end{aligned}$$

For an n-by-n residual, these blocks have dimensions $9n$, $6n$, $6n$, and $4n$. If

$$F(a, b) = \sum_{p=0}^{4} \sum_{q=0}^{4} B_p^4(a) B_q^4(b) \widetilde{F}_{p,q},$$

then $\widetilde{F}_{0,0} = F_{0,0}$ and $\widetilde{F}_{1,0} = F_{0,0}/4 + 3F_{1,0}/4$. The finite SDP has the four PSD-LMI constraints above and one exact matrix equality for each of the $5 \times 5 = 25$ elevated Bernstein labels. With $[Q]_{u,v}$ denoting the n-by-n Gram block indexed by labels u, v, two illustrative equalities are

$$\begin{aligned} F_{0,0} &= [Q_{\varnothing}]_{(0,0),(0,0)}, \\ \frac{1}{4} F_{0,0} + \frac{3}{4} F_{1,0} &= \frac{1}{2} [Q_{\varnothing}]_{(0,0),(1,0)} + \frac{1}{2} [Q_{\varnothing}]_{(1,0),(0,0)} + \frac{1}{4} [Q_1]_{(0,0),(0,0)}. \end{aligned}$$

The remaining 23 are generated by the same weighted Bernstein coefficient map. Thus the identities are affine equalities, not 25 additional LMIs. A $\preceq 0$ comparison instead matches the negatives of the elevated target coefficients.

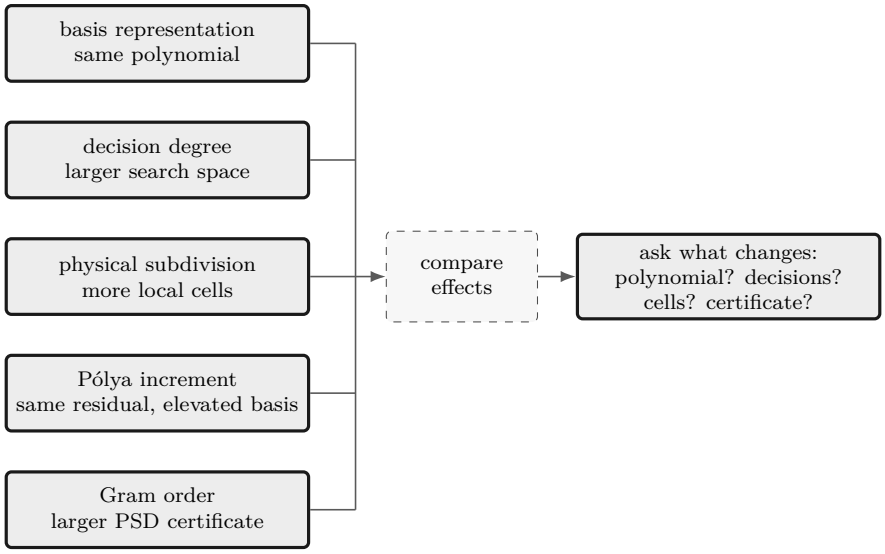
```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
polya = direct.applyPolya(1);
putinar = direct.applyPutinar();
box = direct.applyFullBoxPreorder();
directConstraintCount = numel(direct.Constraints)
polyaState = [polya.PolyaDegree, numel(polya.Constraints)]
putinarState = [putinar.PutinarOrder, numel(putinar.Constraints)]
fullBoxState = [box.FullBoxOrder, numel(box.Constraints)]
```

```
directConstraintCount =
     3
polyaState =
     1     4
putinarState =
     1     5
fullBoxState =
     1     5
```

Selections rebuild from the stored original `Residual` and `Relation`. Reapplying Pólya, Putinar, or full-box replaces the previous selection; the certificates never compound, and the three opt-in families are mutually exclusive.

B.9 Five Different Modeling Choices

These choices act on different parts of the problem.



Choice	Changes	Does not mean
basis conversion	coefficient coordinates	more decision freedom
decision degree	independent decision coefficients	certificate order
subdivision	physical cells and local hulls	higher polynomial degree
Pólya degree	elevated residual coefficients	Gram variables
Putinar/full-box order	Gram bases and PSD block sizes	decision degree

When a certificate fails, do not infer continuous infeasibility. Decide separately whether the model needs a richer decision degree, a finer physical grid, a larger Pólya increment, or a higher Putinar or full-box order.

B.10 Implemented Boundary

Implemented public behavior	Background only
direct coefficient assembly	general full-space SOS parser
uniform `applyPolya([d])`	direction-wise or automatic Pólya search
`applyPutinar([r])`	general-domain/general-generator Putinar interface
`applyFullBoxPreorder([r])`	general-domain Schmüdgen parser
exact one-dimensional parity forms	automatic or adaptive Gram-order search
all cells and active rate rows	sparse Gram selection or adaptive refinement
explicit user margins	package-owned strictness or solver policy

B.11 Further Reading

The primary papers by Putinar [10] and Schmüdgen [11] remain the mathematical authority for their respective theorems. For implementation-oriented background, YALMIP maintains an SOS tutorial, and the Positivstellensatz overview gives a compact map of the surrounding result family. Neither link changes the fixed-order, box-specific package boundary described above.

Bibliography

[1] R. T. Farouki, "The Bernstein polynomial basis: A centennial retrospective," *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012. doi:10.1016/j.cagd.2012.03.001.

[2] M. Zettler and J. Garloff, "Robustness analysis of polynomials with polynomial parameter dependency using Bernstein expansion," *IEEE Transactions on Automatic Control*, vol. 43, no. 3, pp. 425–431, 1998. doi:10.1109/9.661615.

[3] I. Masubuchi, A. Kume, and E. Shimemura, "Spline-type solution to parameter-dependent LMIs," in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 2, 1998. doi:10.1109/CDC.1998.758549.

[4] Y. Xu and F. Jabbari, "Spline-based parameter varying output feedback synthesis with improved L_2 gain," in *2024 IEEE 63rd Conference on Decision and Control*, pp. 1455–1460, 2024. doi:10.1109/CDC56724.2024.10886461.

[5] G. Chesi, "LMI techniques for optimization over polynomials in control: a survey," *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2500–2510, 2010. doi:10.1109/TAC.2010.2046926.

[6] F. Lukács, "Verschärfung des ersten Mittelwertsatzes der Integralrechnung für rationale Polynome," *Mathematische Zeitschrift*, vol. 2, pp. 295–305, 1918. EuDML record.

[7] E. de Klerk, R. Hess, and M. Laurent, Improved convergence rates for Lasserre-type hierarchies of upper bounds for box-constrained polynomial optimization, *SIAM Journal on Optimization*, vol. 27, no. 1, pp. 347–367, 2017. doi:10.1137/16M1065264.

[8] E. M. Aylward, S. M. Itani, and P. A. Parrilo, Explicit SOS decompositions of univariate polynomial matrices and the Kalman–Yakubovich–Popov lemma, in *Proceedings of the 46th IEEE Conference on Decision and Control*, pp. 5660–5665, 2007. author manuscript.

[9] C. W. Scherer and C. W. J. Hol, Matrix sum-of-squares relaxations for robust semi-definite programs, *Mathematical Programming*, vol. 107, pp. 189–211, 2006. doi:10.1007/s10107-005-0684-2.

[10] M. Putinar, "Positive polynomials on compact semi-algebraic sets," *Indiana University Mathematics Journal*, vol. 42, no. 3, pp. 969–984, 1993. JSTOR 24897130.

[11] K. Schmüdgen, "The K-moment problem for compact semi-algebraic sets," *Mathematische Annalen*, vol. 289, pp. 203–206, 1991. doi:10.1007/BF01446568.

[12] V. Powers and B. Reznick, "A new bound for Pólya's theorem with applications to polynomials positive on polyhedra," *Journal of Pure and Applied Algebra*, vol. 164, nos. 1–2, pp. 221–229, 2001. doi:10.1016/S0022-4049(00)00155-9.

Index